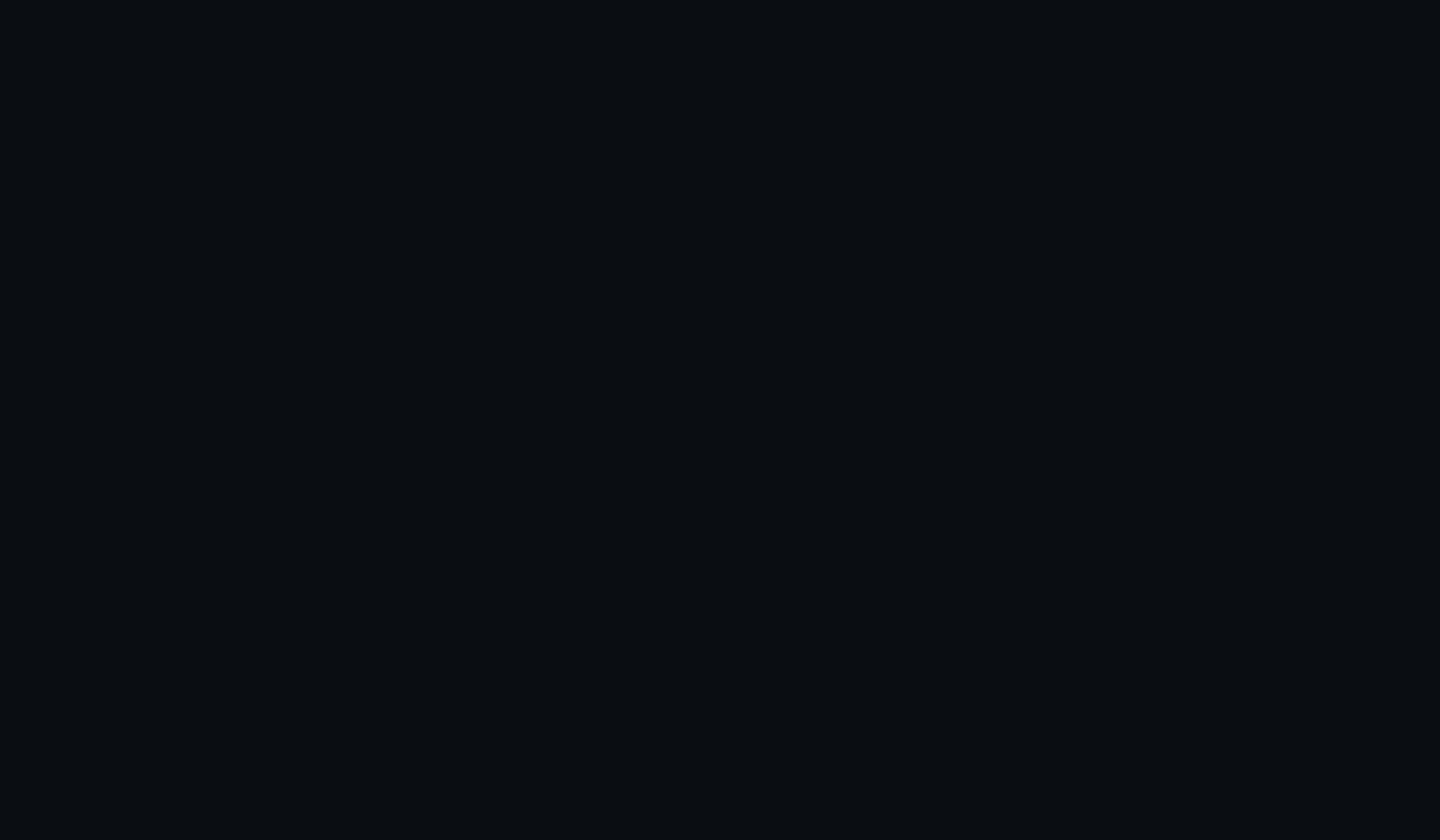


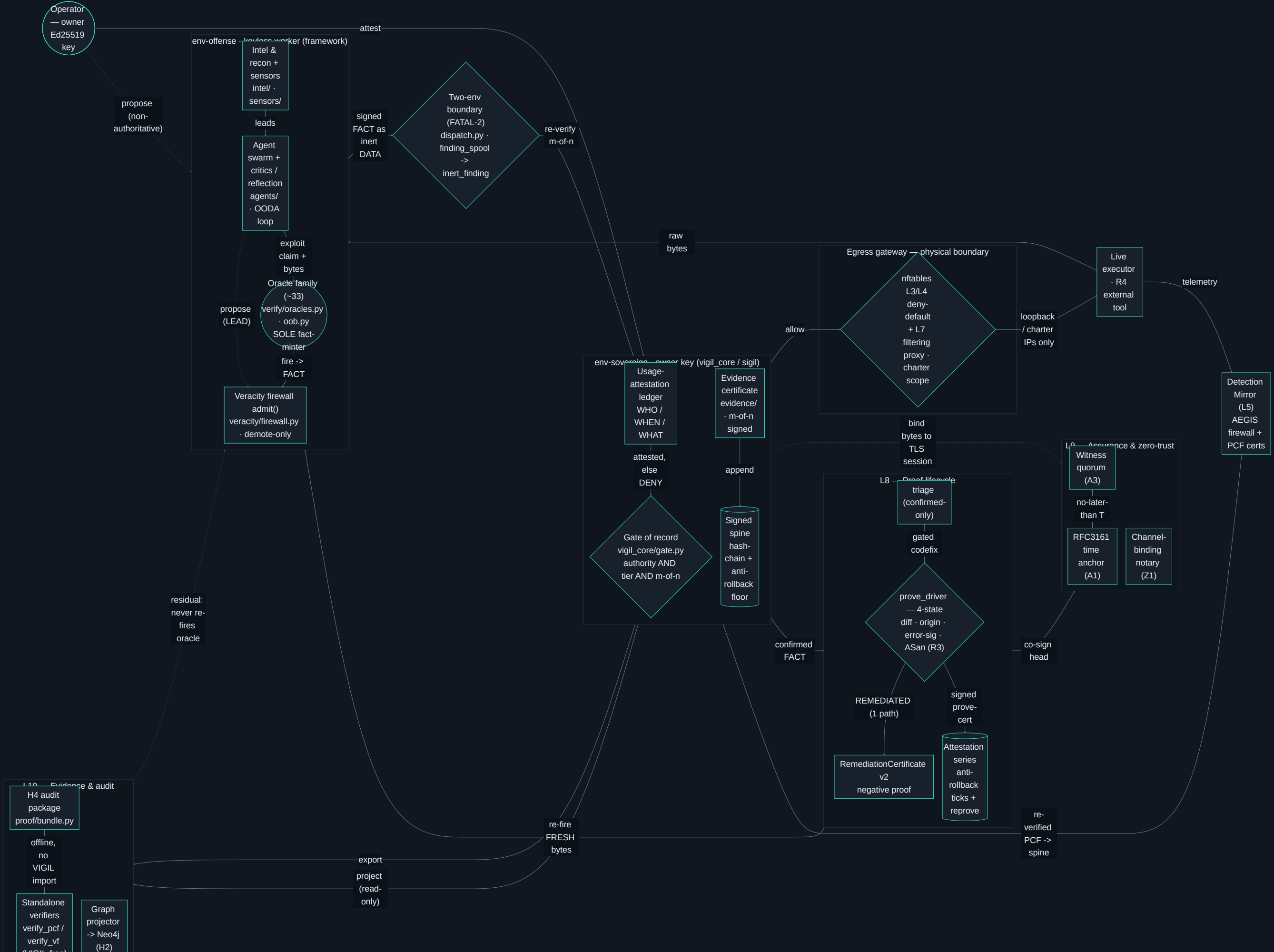
One provable system, end to end

VIGIL fuses an LLM-driven offense engine (redamon-shaped) and a Go-shaped orchestration body onto one deterministic sovereign core: the LLM, its tools, the knowledge-graph, and the cognition governors only **propose** — nothing they say is true until a deterministic CRUCIBLE oracle **re-executes** the evidence and mints a signed FACT, nothing acts until the conjunctive gate (CRUCIBLE authority $\wedge$ WARDEN tier $\wedge$ m-of-n destruction) **authorizes** it, and nothing leaves the box until the egress gate **lets it out** — pinned to loopback for this validation build. Every one of those decisions, and every use of the system itself, lands as an append-only, Ed25519-signed, hash-chained record on one spine, witnessed and offline-verifiable forever.

The same invariant runs through every layer below: L1 is the core substrate that enforces it, L2 is the per-turn loop that exercises it, L3 is the offense “body” bolted onto it, L4 is the live backends that make it real against a loopback target, L5 is its defensive mirror (the AEGIS Detection Mirror proving the same attacks from telemetry), L6 is the auto-patch loop that may act only on a FACT, and L7 names the veracity firewall and trust boundaries that hold across all six.

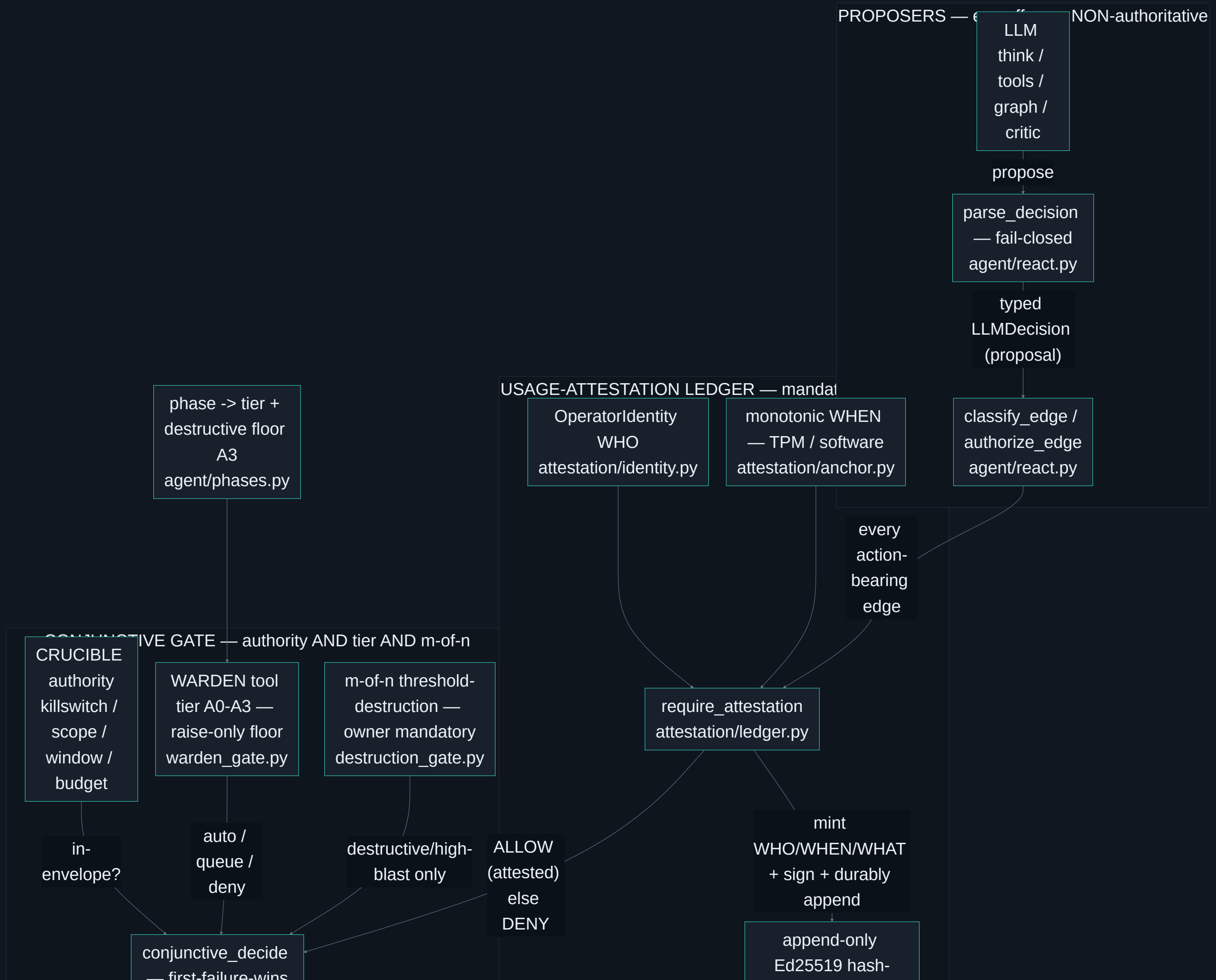
The remaining layers carry a confirmed FACT the rest of the way — and are explicit about where a property is **BUILT and measured** versus a **CAPABILITY** whose last mile is a hardware, service, or third-party fact. **L8** is the proof lifecycle: a FACT becomes a signed, re-executable *remediation* certificate and a continuously re-proven, anti-rollback attestation series (differential / direct-to-origin / error-signature / ASan-binary remediation channels). **L9** operationalizes assurance and closes the zero-trust gap as far as software can: an external RFC 3161 time anchor, a deployable witness quorum, TLS channel-binding, and two *VIGIL-free* standalone verifiers. **L10** is the evidence substrate — certificates, the signed spine, the external audit package, and the read-only graph projection — that authenticates and transports facts but never mints them. **L11** is the offense plane and two-env control plane underneath it all: the oracle family (~33 sole-minters), the intel/sensor inflow, the egress gateway, and the `dispatch` / keyless-worker boundary that keeps offense and sovereign code from ever sharing a process.

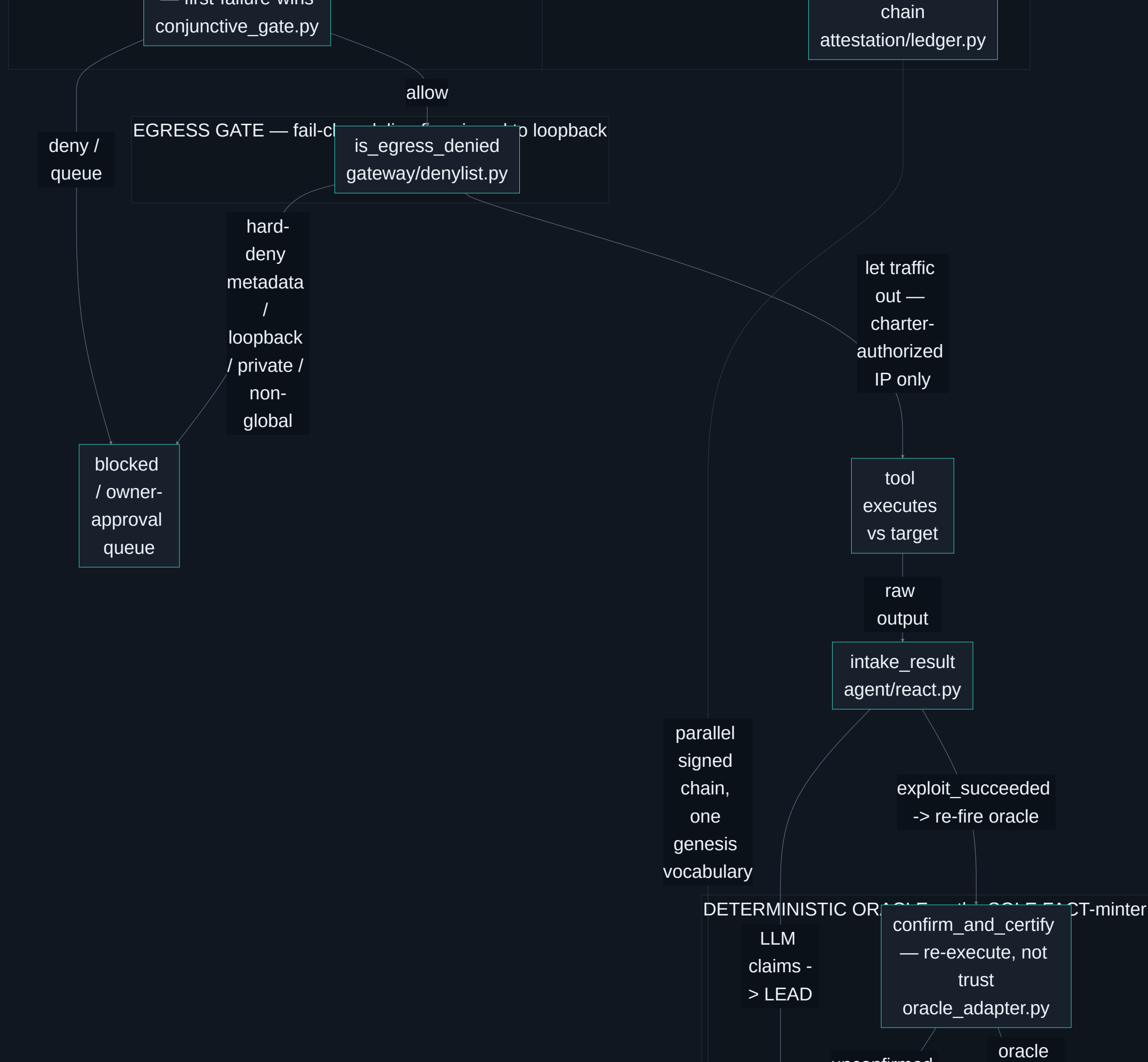




VIGIL L1 — The Sovereign Core: propose → attest → gate → egress → oracle → sign → spine

L1 is the deterministic center every offense action routes through. The LLM/tools/graph/critic only PROPOSE: a raw model response is parsed fail-closed into a non-authoritative LLMDecision (agent/react.py). Before any gated action the always-on usage-attestation LEDGER mints a signed WHO/WHEN/WHAT record and fail-closed DENIES if it cannot (attestation/). Every target-touching edge then crosses the CONJUNCTIVE GATE — CRUCIBLE authority (killswitch/scope/window/budget) $\wedge$ WARDEN tier A0–A3 (phase → tier, raise-only floor) $\wedge$ (for destructive/high-blast) an owner-inclusive m-of-n threshold-destruction authorization — first-failure-wins, any error is a DENY. Only an ALLOW reaches the EGRESS gate (gateway/denylist.py), which hard-denies metadata/loopback/private/non-global egress and lets traffic out only to charter-authorized IPs (live-fire validation is pinned to loopback). A tool's raw output is taken in honestly: LLM claims become LEADs; an exploit_succeeded claim re-fires the deterministic CRUCIBLE ORACLE over the retained context, and ONLY an oracle confirmation on a known bug-class mints a signed, proof-carrying EvidenceCertificate (m-of-n) — a FACT. Facts carry a signed evidence_ref enforced at the type level (agent/state.py); the veracity path is demote-only (unconfirmed → labelled lead, never up). A FACT crosses the two-env boundary as INERT signed JSON DATA only (inert_finding.py; keyless offense worker), is re-verified m-of-n on the sovereign side, and appended to the one append-only Ed25519 signed hash-chain SPINE (vigil_core: crypto/chain/canonical/models), anchored by a monotonic anti-rollback SignedChainHead. On top of the head sit the I2 witnessed transparency log (split-view-resistant under strict majority) and offline-verifiable SCITT/OpenVEX certs with RFC-6962 Merkle inclusion proofs. Every record — findings and attestations alike — is a domain-separated, canonical-JSON, Ed25519-signed link on an append-only chain.





TWO-ENV BOUNDARY — confirmed DATA only

keyless offense
worker — no
owner key
offense_worker.py

cross env-
offense -
> env-
sovereign
as inert
DATA

validate/build
inert finding —
JSON-only
inert_finding.py

re-verify
m-of-n,
then
append

THE ONE SIGNED SPINE — the center

Ed25519
+ m-of-n
verify +
weak-
key reject
crypto.py

canonical
JSON +
domain-
tagged
digests
canonical.py

checkpoint
AgentState into spine
live/spine_vigilcore.py

append-
only
hash-
linked
chain
chain.py
+
models.py

unconfirmed
/ unmapped
(demote-
only)

FIRED +
known
bug-
class

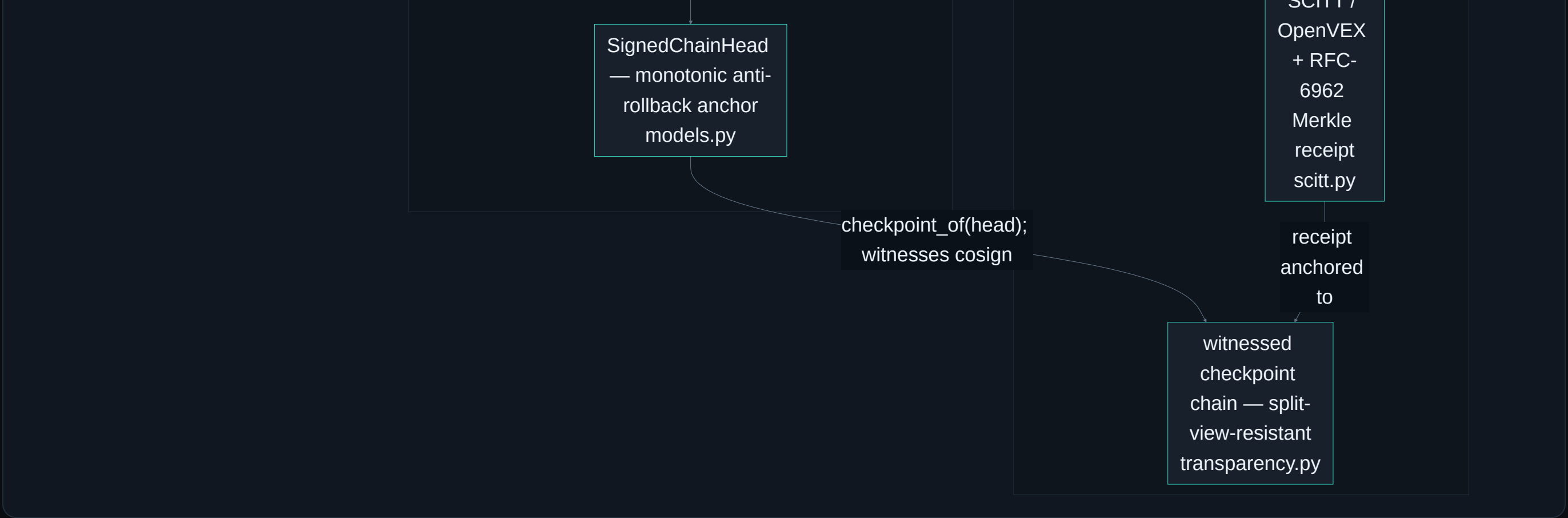
leads store
— proposals
agent/state.py

EvidenceCertificate
+ m-of-n sign
oracle_adapter.py

FACT:
signed
evidence_ref

facts store —
oracle-only
agent/state.py

DSSE m-
of-n +
inclusion
proof



Spine crypto — Ed25519 + m-of-n verify

/home/kali/vigil/packages/core/vigil_core/vigil_core/crypto.py

Ed25519 primitives and threshold verification, vendored verbatim from CRUCIBLE. load_public_key rejects non-canonical ($y \geq p$) and low-order/small-order points before any verify; verify_one returns False on a bad sig and raises only on malformed material; verify_threshold counts DISTINCT trust-root authorizers with a valid signature and compares to the m-of-n threshold. sign/generate_keypair are provisioning-only — the runtime only ever verifies.

FEATURES Ed25519 verify

m-of-n threshold verify over distinct authorizers

non-canonical ($y \geq p$) key rejection

Spine chain — tamper-evident hash-linked ledger

/home/kali/vigil/packages/core/vigil_core/vigil_core/chain.py

Each ChainEntry hashes (seq, prev_hash, cert_digest); build_chain/append_entry extend it; verify_chain rejects a prev_hash mismatch (delete/reorder), an entry_hash mismatch (tamper), or a seq gap; sign_head signs a SignedChainHead over version-conditional bytes; verify_head re-checks the chain, that head_hash/last_seq/entry_count match, the m-of-n head signature, and a prev_highwater anti-rollback. entry_count stays ABSOLUTE (base_count + live window) so pruning never changes the tamper model.

FEATURES hash-linked entries (seq/prev/cert)

Canonical bytes + domain-tagged digests

/home/kali/vigil/packages/core/vigil_core/vigil_core/canonical.py

canonical_json emits deterministic UTF-8 JSON (recursively sorted keys, compact separators); sha256_hex/digest_payload hash it; evidence_signing_bytes prefixes the crucible-evidence-v1 domain tag to the canonical certificate bytes. This is the exact byte basis every signer and verifier across the core derives independently.

FEATURES

deterministic canonical JSON (sorted keys, compact)

sha256 digests

frozen crucible-evidence-v1 domain separation

TOOLS

hashlib

json (stdlib)

Integrity + trust models

/home/kali/vigil/packages/core/vigil_core/vigil_core/models.py

Pydantic models with extra='forbid': Signature, AuthorizerKey, TrustRoot (m-of-n with threshold $\leq$ count and unique key_ids, validated at construction), ChainEntry, and SignedChainHead (schema_version, last_seq monotonic, absolute entry_count, head_hash, plus v2 cold-archive prune fields defaulted to the no-prune identity).

FEATURES

TrustRoot m-of-n schema (threshold=1 covers solo owner / FROST group key)

extra='forbid' everywhere

v2 prune fields default to no-prune identity

GENESIS_PREV all-zero sentinel

libsodium small-order/low-order blocklist

strict base64 length checks

IntegrityError (malformed = provisioning error, not attacker path)

TOOLS

cryptography (pyca) Ed25519

base64/binascii

vigil_core.models

IN VARIANT

The runtime NEVER holds authority to mint truth by signing at runtime — it only verifies. Weak/malformed keys fail closed (a low-order key admits a keyless R=identity,S=0 forgery, so it is barred everywhere), so no trust-root poisoning or encoding-ambiguity can forge a signature.

signed head with monotonic last_seq anti-rollback

re-based/pruned live-window verification (genesis_prev override)

absolute entry_count over pruned base + Merkle root

version-conditional head signing bytes

TOOLS

vigil_core.canonical

vigil_core.crypto

vigil_core.models

IN VARIANT

The spine is append-only and tamper-evident: any deletion, reorder, tamper, truncation, or rollback below the accepted high-water fails verification. A v1 head signs byte-identically to pre-v2 so every persisted head verifies unchanged.

IN VARIANT

Signer and verifier derive BYTE-IDENTICAL signing bytes from the same field values, and a domain tag prevents any signature from being replayed across record types. The domain tag is frozen — changing it requires a schema bump that invalidates all existing signatures.

TOOLS

pydantic

IN VARIANT

Trust config is validated fail-closed at construction (no duplicate key_ids, threshold within bounds); a v1 head parses and signs identically under the v2 model so old deployments verify unchanged.

WARDEN phase → tier map

/home/kali/vigil/integration/vigil_integration/agent/phases.py

Pure/deterministic mapping of the offense phase ladder onto WARDEN kernel tiers: informational → A1, exploitation → A2, post_exploitation → A3. can_transition enforces MONOTONE, one-step-at-a-time escalation (refuses downgrade or skip). tool_tier floors a destructive/high-blast tool at A3 regardless of phase.

FEATURES

phase → tier ladder A1/A2/A3

monotone one-step escalation

destructive floor at A3

stdlib-only, import-clean

TOOLS

agent.state.Phase (stdlib enum)

WARDEN tool gate — raise-only tier floor

/home/kali/vigil/integration/vigil_integration/warden_gate.py

decide_tool classifies a tool NAME to A0..A3 via an injected classifier (default: the sigil-kernel binary via subprocess), applies a raise-only floor (max(tier, A2)), and returns auto only if tier<=A1 AND tier<=ceiling, else queue. Empty/unknown name, garbage classifier output, or denylist → DENY (fail-closed A3). WardenGateHooks fails safe: only an AUTO decision runs, everything else raises.

FEATURES

injectable, cached kernel classifier

raise-only floor (default A2) + ceiling (default A1)

hard denylist

fail-closed to A3

Sovereign ReAct interposition

/home/kali/vigil/integration/vigil_integration/agent/react.py

The keystone that re-plumbs redamon's trust-the-LLM ReAct loop through the sovereign core. parse_decision parses a raw response fail-closed into a typed LLMDecision, downgrading a malformed action to the safest valid one (a broken deploy_fireteam NEVER becomes a deploy; total failure pauses for a human). classify_edge/authorize_edge route every target-touching edge through the injected conjunctive gate (queueing escalations/deploys for signed approval; DENY on any structural invalidity or gate error). intake_result turns tool output into LEADs and re-fires the injected ORACLE on an exploit_succeeded claim — only a signed evidence ref yields a FACT.

Conjunctive gate

/home/kali/vigil/integration/vigil_integration/conjunctive_gate.py

conjunctive_decide composes CRUCIBLE authority ∧ WARDEN tier ∧ (if destructive) m-of-n threshold-destruction as injected thunks, first-failure-wins: CRUCIBLE checked first (killswitch has absolute precedence), then WARDEN, then the destruction conjunct. ALLOW only if CRUCIBLE in-envelope AND WARDEN auto AND (if destructive) threshold-authorized; QUEUE if WARDEN needs owner approval; DENY otherwise. build_offense_gate wires the REAL CRUCIBLE authorize_action (loaded verified via the governance trust root) and cross-binds the destruction quorum to the SAME target+engagement the CRUCIBLE half scoped.

INVARIANT

The phase gate is enforced by the WARDEN tier (kernel), not by a Python check the LLM could talk past; escalation is monotone and every forward crossing needs signed approval through the conjunctive gate, never the LLM's say-so.

INVARIANT

An offense tool never auto-runs on a LIVE target — a read-shaped name that classifies low is raised to the floor, and with ceiling A1 every offense tool queues for owner approval. The floor can only RAISE a tier, never lower it; any failure classifies maximally dangerous.

INVARIANT

The LLM/tools ONLY PROPOSE: a bare LLM decision can never escalate, act on a target, or assert a fact. Every authority-bearing edge is gate-authorized; every fact is oracle-minted. No gate wired and no oracle wired both mean fail-closed (deny / no fact).

INVARIANT

ONLY the conjunction authorizes an action — any deny or ANY error in any half is a DENY (never caught-and-continued), a strict 'is True' on the destruction result, and an unrecognized WARDEN outcome can never silently ALLOW. A None trust_root is refused (it would load the authority unsigned). An irreversible action needs the quorum even if WARDEN would auto-allow the class.

Threshold-destruction gate (l4 m-of-n)

/home/kali/vigil/integration/vigil_in
tegration/destruction_gate.py

authorize_destruction is the highest-
consequence gate: five fail-closed
properties — m-of-n threshold
(vigil_core.verify_threshold), MANDATORY
owner from the immutable deployment-time
DestructionAuthority (not a per-call string),
action binding to
(engagement,target,class,action_id), dead-
man's-switch (bounded, policy-capped

CRUCIBLE oracle adapter
— the sole FACT-minter

/home/kali/vigil/integration/vigil_in
tegration/oracle_adapter.py

confirm_and_certify drives CRUCIBLE's
deterministic oracle by RE-EXECUTING
over the finding's retained oracle_context
(baseline/mutated responses, probe rounds,
timing, OOB hits). Returns None unless an
oracle actually fired at >= high confidence
(no assertion-only path). A signed FACT
additionally requires the confirmed

Egress gate — denylist

/home/kali/vigil/gateway/vigil_gatewa
y/denylist.py

Pure, deterministic L3/L4 conscience:
is_egress_denied answers 'may sandbox
egress reach this IP?' fail-closed. Tier-1
HARD-DENY (metadata 169.254.169.254 /
IMDS-over-IPv6, loopback, link-local,
multicast, reserved, TEST-NETs) can
NEVER be re-enabled by charter; Tier-2
private (RFC1918/CGNAT/ULA) is denied
unless the exact resolved IP is charter-

Veracity firewall — Finding
+ AgentState (demote-only)

/home/kali/vigil/integration/vigil_in
tegration/agent/state.py

Finding.status is the veracity: a LEAD is a
proposal, a FACT is oracle-confirmed and
MUST carry a signed evidence_ref
(enforced by a model_validator at the TYPE
level, so even a deserialized/replayed
record can't construct an evidence-less
fact). AgentState keeps SEPARATE facts
and leads stores: record_lead scrubs any

window; nan/expired denied), and single-use (caller-supplied is_consumed, no permissive default). Exact-type checks reject behavior-overriding subclasses; require_destruction_authorization raises DestructionRefused for the privileged executor.

FEATURES m-of-n over distinct authorizers

immutable mandatory-owner set

action-binding
(engagement/target/class/action_id)

dead-man's-switch (max lifetime 900s)

single-use nonce vs spine-derived is_consumed

fresh domain tag (non-replayable)

TOOLS vigil_core.verify_threshold

vigil_core.canonical_json

vigil_core.sign

INVARIANT

A single autonomous, prompt-injectable offense worker can NEVER perform an irreversible/high-blast action on its own authority — it needs an owner-inclusive quorum, bound to this exact action, fresh in time, single-use. Any malformed input or error is a DENY; the mandatory signer set is bound to the provisioned key artifact so it cannot be renamed per call.

Two-env inert-finding seam

/home/kali/vigil/integration/vigil_in
tegration/inert_finding.py

The sovereign side's receiver for a confirmed finding crossing env-

bug_class to be a KNOWN oracle-mapped class — otherwise it stays a labelled lead. On a confirmed+mapped finding it mints and m-of-n signs a proof-carrying EvidenceCertificate binding the oracle_context_digest; certify_to_scitt bridges only a FACT to the SCITT/OpenVEX form. Refuses to sign with zero governance signers.

FEATURES re-execution not trust

high-confidence threshold

known-bug-class honesty invariant

proof-carrying m-of-n EvidenceCertificate

OracleKind .value normalization (anti-tamper)

lazy framework import

TOOLS

framework.v2.verify.confirmation.confirm_finding

framework.v2.evidence.certify (build/sign)

framework.v2.verify.verifier.is_known_bug_class

scitt.mint_finding_statement

INVARIANT

An LLM's say-so NEVER confirms anything: only a deterministic oracle re-firing over retained evidence, on a known bug-class, with an m-of-n governance signature, produces a FACT. Everything else is an honest, replayable LEAD. Facts are re-executed, not string-trusted.

Keyless offense worker
(two-env boundary)

/home/kali/vigil/integration/vigil_in
tegration/offense_worker.py

Runs an offense engagement in env-offense (framework/strix importable) as an

authorized; a final is_global backstop denies anything non-routable. Embedded-IPv4 forms (::ffff:, 6to4, NAT64, ::a.b.c.d) are unwrapped and re-checked. Imported by both the nftables renderer and the forward proxy (re-checks every resolved IP, defeating DNS rebinding).

FEATURES

hard-deny tier (charter-independent)

private-unless-charter-scoped tier

embedded-IPv4 unwrapping (IMDS evasion defense)

is_global routable backstop

pure (no DNS/IO/clock) — reused by nftables + proxy

TOOLS ipaddress (stdlib)

INVARIANT

ONLY the egress gate lets traffic out, and it fail-closes: unparseable → deny, hard-deny beats the allowlist, cloud-metadata/SSRF targets are unreachable even if an operator (or an injection) lists them. Live-fire validation is pinned to loopback via the live executor while this gate governs real target egress.

Usage-attestation ledger

/home/kali/vigil/integration/vigil_in
tegration/attestation/ledger.py

The always-on 'when was this tool used and by whom' core. require_attestation is called

evidence_ref, record_fact refuses an empty ref. The honesty audit in cognition.py can only downgrade a scheduling verdict, never a finding's truth.

FEATURES

type-level fact⇒evidence_ref invariant

separate facts/leads stores

record_lead scrubs evidence

demote-only honesty audit (verdict, not truth)

TOOLS pydantic model_validator

agent.cognition.audit_productivity_claim

INVARIANT

Veracity is demote-only: the LLM and tools can only ADD to leads; facts grow solely through the oracle-confirmation path and only with a non-whitespace signed evidence reference. The invariant is closed even against deserialization and checkpoint-replay paths.

Attestation identity (WHO)

/home/kali/vigil/integration/vigil_in
tegration/attestation/identity.py

Loads-or-creates a persisted (0600) operator Ed25519 keypair, validating a

offense → env-sovereign.

validate_inert_finding parses the inbound blob with json.loads ONLY (never pickle/eval/yaml), size-bounds it (256KiB), enforces exactly {schema,certificate,signatures} with non-empty string identity fields, and verify_signature re-checks the CRUCIBLE governance m-of-n over the exact evidence_signing_bytes using vigil_core alone. to_spine_payload yields the vetted JSON datum the sovereign side appends.

FEATURES

- JSON-only parse (no code execution)
- 256KiB DoS bound + strict shape
- recursion-depth guard
- m-of-n signature re-verify (vigil_core only)
- two-anchor trust model

TOOLS

- json (stdlib)
- vigil_core.verify_threshold
- vigil_core.evidence_signing_bytes

engagement-scoped, ceiling-bounded process holding NO owner signing key — construction refuses an owner key outright. It crosses a confirmed finding to the sovereign side only via inert_finding.build_envelope, and imports neither framework nor strix (duck-types SignedEvidence).

FEATURES

- keylessness enforced at construction
- engagement-scoped, ceiling-bounded (A2 working ceiling)
- inert-DATA-only cross-boundary
- no framework/strix import

TOOLS

- inert_finding.build_envelope

INVARIANT

BEFORE any gated action: it mints a signed UsageAttestation FIRST binding WHO (OperatorIdentity) / WHEN (wall 'at' data field + monotonic anchor) / WHAT (action/target/phase), then fail-closed DENIES if no attestation can be minted (no signer / unbound operator / malformed input), no durable writer is wired, or the durable append fails. record_usage redacts every free string through the one F3 vocabulary before hashing/signing. verify_ledger re-checks the whole presented chain (contiguity, each record hash, non-decreasing monotonic anchor, operator binding, every Ed25519 signature vs an injected trust anchor) and supports expected_head/expected_count pins to catch a truncated tail / total wipe. Ordered by (seq, prev_hash) alone — 'at'/'monotonic' are signed DATA, never ordering keys.

FEATURES

- fail-closed engagement gate (mint-first, writer mandatory)
- append-only hash-chained Ed25519 records
- F3 secret-redaction of all committed strings
- monotonic anti-back-dating check
- external head/count pins (tail-truncation/wipe detection)
- JSONL persistence with torn-tail tolerance
- who/when non-repudiation replay

TOOLS

- vigil_core.canonical/crypto (sha256_hex, evidence_signing_bytes, verify_one)
- tools.mcp_registry._redact_str (F3)
- attestation.anchor / identity / models

INVARIANT

loaded public key through vigil_core load_public_key (rejects weak keys) and round-tripping the private key; derives the fingerprint (sha256 of the raw pubkey) that binds a signature's key_id to the operator. resolve_operator reads OS login, git user.name/user.email (argv-list subprocess, never a shell), and hostname via total injectable readers. operator_signer/operator_key_resolver build the injected signer (key_id == fingerprint) and the matching trust anchor verify_ledger consults.

FEATURES

- persisted 0600 operator keypair (weak-key-validated)
- sha256 pubkey fingerprint binding
- shell-free git identity resolution
- injected signer + trust-anchor resolver

TOOLS

- vigil_core.crypto (generate_keypair/sign/verify_one/load_public_key)
- getpass/socket/subprocess (stdlib)

INVARIANT

A finding crosses as INERT SIGNED DATA only — never code, never a governance action. Structural inertness (JSON-only) plus m-of-n signature re-verification is anchor 1 of the two-anchor trust model (owner-signed spine head is anchor 2). Depends on vigil_core alone, never framework/strix, so the personal core stays offense-free-by-construction.

The powerful, prompt-injectable offense process is a keyless trust domain: without the owner key any governance event it forges carries sig=None and never verifies, so it can never become a confused deputy that escalates its own authority. The two-env boundary is STRUCTURAL (a property of which packages are installed), proven by test_two_env_boundary.py.

A gated action can never proceed unless a signed WHO/WHEN/WHAT record is durably recorded first (mandatory, fail-closed on durability). The ledger is append-only + hash-chained + Ed25519-signed; a tampered/reordered/interior-deleted/forged/back-dated entry fails verification, and no wallclock/RNG touches the chain math (Ed25519 signing is deterministic). Every public function is total.

INVARIANT

The signature's key_id is provably the bound operator's key fingerprint, and only that box's own persisted key verifies — a forged entry signed by an unknown key is rejected. Every substrate read is total (degrades to " , never a crash); key generation uses the CSPRNG once at provisioning and never touches chain math.

Attestation monotonic anchor (WHEN)

/home/kali/vigil/integration/vigil_integration/attestation/anchor.py

read_monotonic_anchor returns a value STRICTLY greater than a persisted floor plus its grounding. TPM-preferred: if /dev/tpm0 and tpm2_readclock are present, reads the hardware monotonic clock via argv-list subprocess (no shell) and uses it when it exceeds the floor (grounded='tpm'); otherwise floor+1 (grounded='software'). The floor is re-persisted so even a grounding switch, a stuck TPM, or a regressed read can never make the anchor go backwards. Total by construction — every branch guarded, never raises.

FEATURES

TPM monotonic clock (injectable probe)

persisted software floor fallback

unified strictly-increasing floor advance

bool-reject on TPM value

best-effort atomic floor persistence

TOOLS

tpm2_readclock (subprocess)

shutil.which / os.replace (stdlib)

Attestation models

/home/kali/vigil/integration/vigil_integration/attestation/models.py

Typed extra='forbid' records: OperatorIdentity (os_login/git_name/git_email/key_fingerprint/hostname) with a fail-closed is_bound requiring the fingerprint AND at least one human handle), MonotonicAnchor (value>=0, grounded), UsageAttestation (seq/prev_hash/operator/action/target/phase/at/machine) with key_id==operator.key_fingerprint), and WhoEntry/WhenEntry replay views. GENESIS_PREV is the same all-zero sentinel as the shared spine.

FEATURES

extra='forbid' typed records

is_bound fail-closed identity test

signature key_id ↔ fingerprint binding

shared genesis vocabulary

TOOLS

pydantic

vigil_core.models.Signature

Witnessed transparency log (I2)

/home/kali/vigil/integration/vigil_integration/transparency.py

Layers transparency guarantees over the signed spine head. checkpoint_of summarizes a SignedChainHead into a public, witnessable Checkpoint; independent Witnesses countersign ONLY after verifying consistent (append-only, count/last_seq only grow, checkpoint meta-chain links back), so an honest stateful witness never equivocates. CheckpointEmitter maintains the linked chain idempotently and gathers countersignatures atomically (decide willing set, then cosign). verify_witnessed proves a quorum signed; verify_split_view_resistant additionally proves a strict-majority (2*threshold>n) of DISTINCTLY-keyed witnesses; is_split lets any client prove a fork after the fact.

FEATURES

public checkpoint summary of the head

consistency (append-only) proof

strict-majority split-view resistance test

distinct-key dedup (base64-malleability safe)

SCITT / OpenVEX offline-verifiable certs

/home/kali/vigil/integration/vigil_integration/scitt.py

Expresses a confirmed-finding EvidenceCertificate as an IETF SCITT-style DSSE signed statement carrying an OpenVEX statement, plus a RECEIPT with a real RFC-6962 Merkle inclusion proof binding it to the transparency log root (and, when anchored, the I2 witnessed checkpoint). DSSE PAE binds payload type to payload so a statement sig can never be replayed as a raw evidence sig. A confirmed finding is status 'affected'; an unconfirmed lead is 'under_investigation' — never asserted affected.

FEATURES

DSSE m-of-n signed statement (PAE domain separation)

OpenVEX portable finding vocabulary

RFC-6962 Merkle inclusion receipt (caller-pinned root, depth-bounded)

affected vs under_investigation honesty

import-clean (vigil_core + transparency only)

TOOLS

INVARIANT

The WHEN anchor can only move forward: an attestation can never be back-dated, so an operator cannot deny an action was recent. A hardware TPM counter cannot be rewound by software; the software fallback still advances in-run even if persistence fails.

INVARIANT

Every attestation field is signed DATA, not a decision input — the chain order is (seq, prev_hash) alone. A malformed/attacker-influenced row either validates into a well-formed record or is rejected; it can never smuggle an unknown field past extra='forbid', and an unbound identity can neither mint nor verify.

idempotent atomic countersignature gather

after-the-fact fork detection (is_split)

TOOLS

vigil_core (verify_threshold, sign, sha256_hex, canonical_json, load_public_key)

INVARIANT

A third party can trust the log WITHOUT trusting its operator. Split-view PREVENTION is honest and CONDITIONAL — it holds only under strict majority of independent, distinctly-keyed witnesses (dedup over DECODED keys; weak keys rejected); below that only per-witness non-equivocation and DETECTION remain. A witness signature is domain-separated so it can never be replayed as a spine/evidence/authority sig.

vigil_core
(verify_threshold/sign/canonical_json/sha256_hex)

transparency (Checkpoint, verify_split_view_resistant)

INVARIANT

A client/regulator/court can verify a finding OFFLINE and FOREVER — no VIGIL service, no network — against the governance trust root, the inclusion proof, and the anchored checkpoint. The honesty invariant is preserved into the portable format: only an oracle-confirmed FACT is 'affected'.

Live signed-spine binder

```
/home/kali/vigil/integration/vigil_in  
tegration/live/spine_vigilcore.py
```

The LIVE drop-in for the framework-free checkpoint layer's injected seams (signer / verify / writer / reader): binds them to a real Ed25519 keypair (vigil_core.crypto) and a real append-only spine FILE (one canonical-JSON hash-chained record per line via vigil_core.chain). rebuild_from re-runs the Finding validator and _facts_store_is_sound on every rebuilt record; a torn tail is dropped and the good prefix survives; a tampered record (bad sig, broken hash-link, forged/edited bytes) is detected by verify_record and the whole-file verify.

FEATURES

real Ed25519 keypair + append-only JSONL spine

hash-chained records (vigil_core.chain)

fact-store soundness re-check on rebuild

torn-tail crash recovery

whole-file tamper audit

TOOLS

vigil_core.crypto

vigil_core.chain

agent.checkpoint

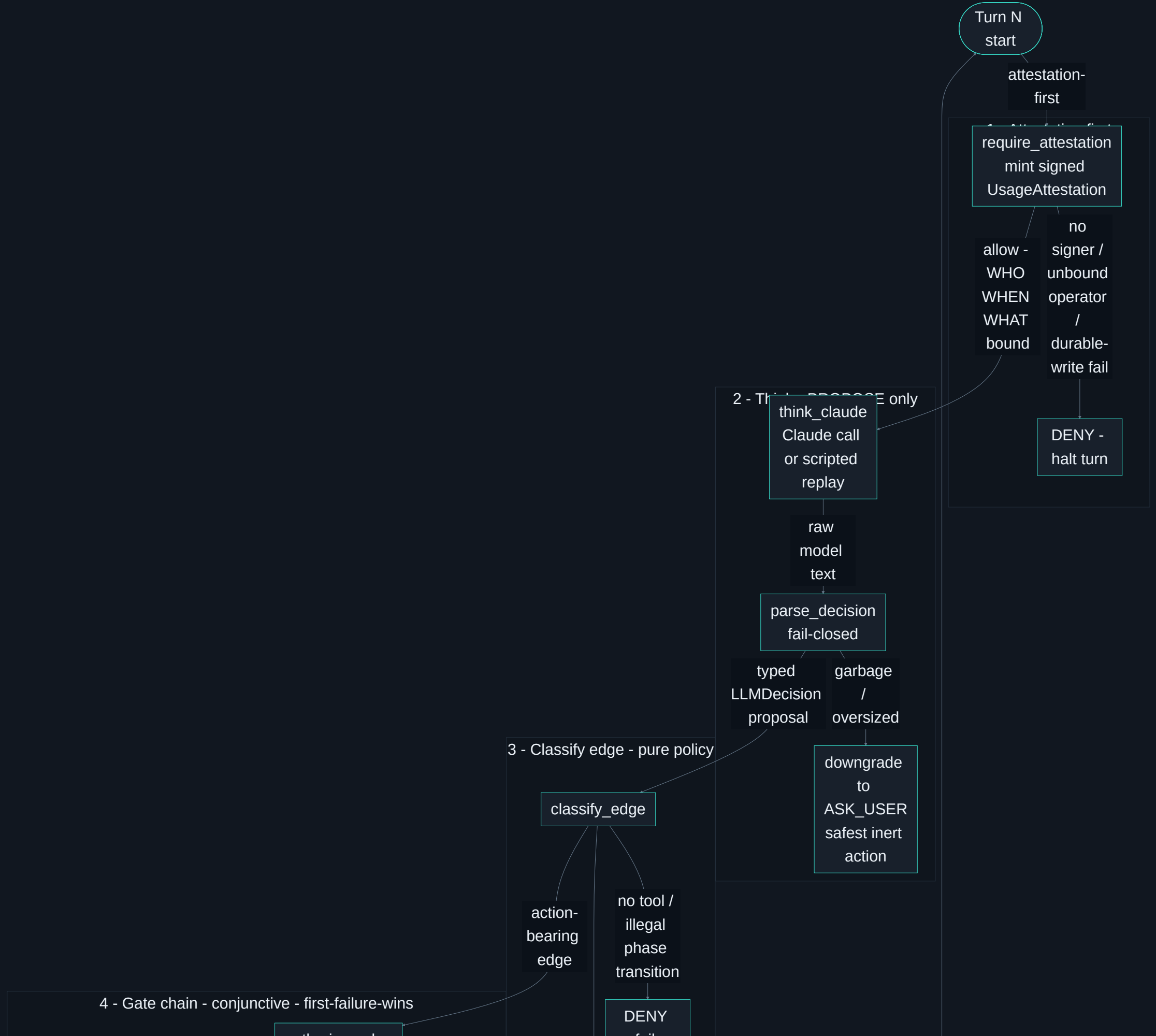
INVARIANT

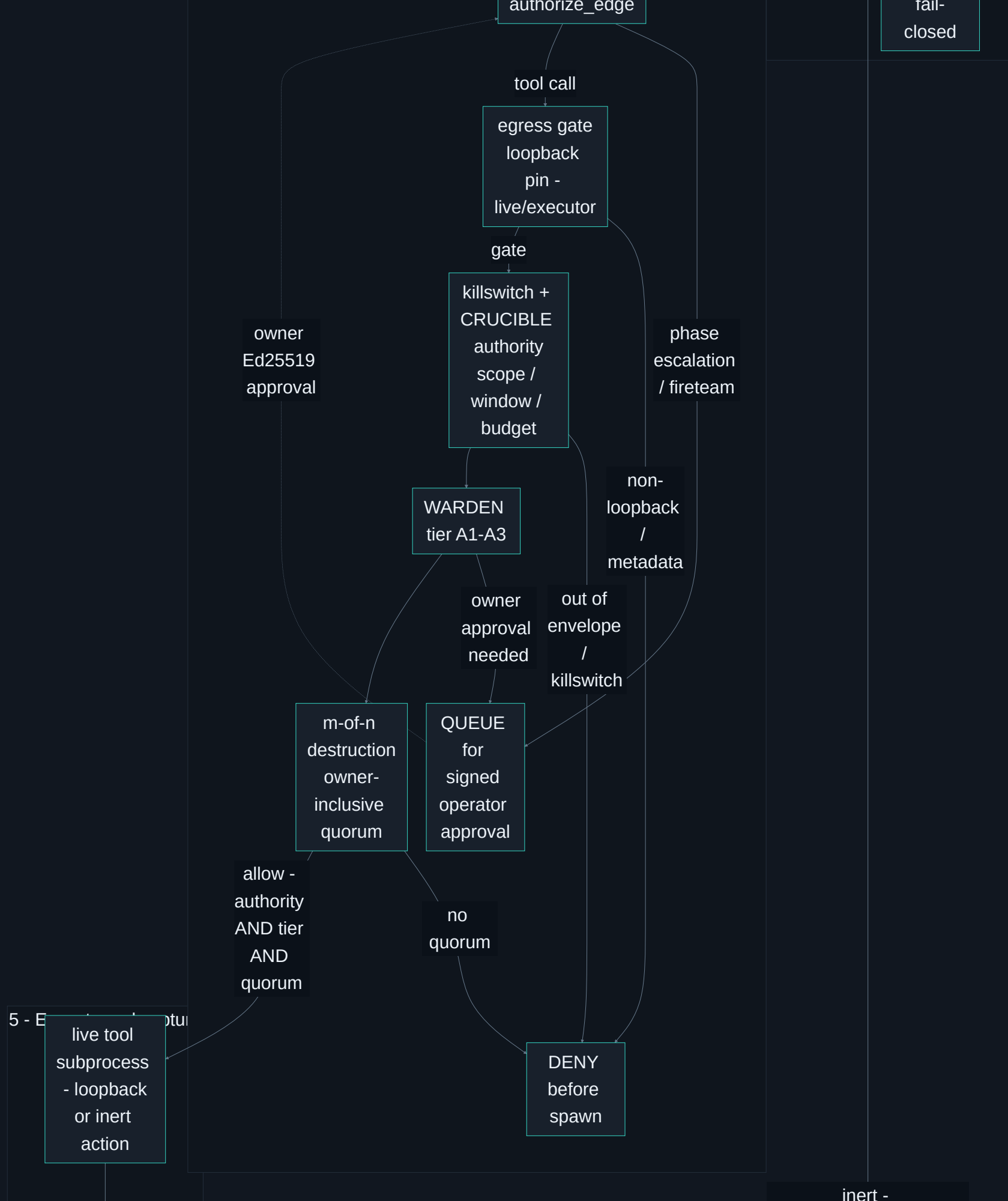
Going live changes NOTHING about the sovereign contract — the binder persists tamper-evidently but makes nothing true and authorizes nothing. An adversary who can write the spine file AND holds the signing key still cannot launder an evidence-less 'fact': a forged snapshot smuggling status='fact' with an empty evidence_ref is skipped and never materializes.

The Sovereign OODA / ReAct Loop — attest, propose, gate, execute, oracle-confirm, project, govern, checkpoint

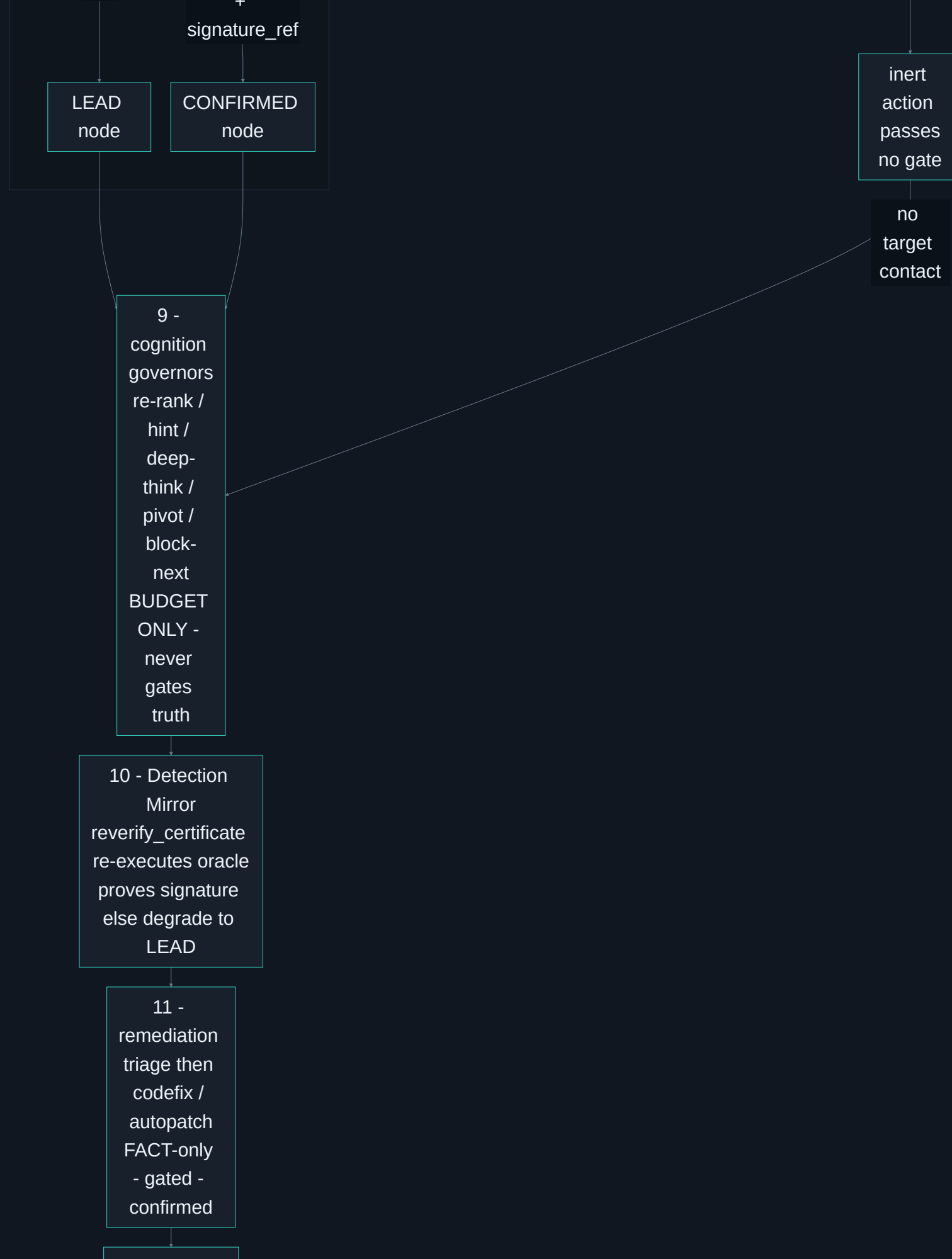
L2 is VIGIL's unified engine turn: a fail-closed OODA cycle where the LLM only PROPOSES and deterministic machinery decides. No single file runs the whole loop — it is assembled at the injection seams of the F2 keystone ``agent/react.py`` (pure, framework-free, fully tested) with the ``live/*`` binders swapping real thunks in for validation. One turn:

(1) mint a signed usage-attestation FIRST — DENY-and-halt if it can't be durably recorded (``attestation/ledger.py``); (2) think — ``live/think_claude.py`` emits an ``LLMDecision`` PROPOSAL (Claude call or scripted replay), parsed FAIL-CLOSED by ``react.parse_decision`` which downgrades garbage to the safest inert ASK_USER; (3) ``react.classify_edge`` — pure policy: inert actions pass, a structurally-invalid edge is DENIED; (4) the conjunctive GATE CHAIN for any target-touching edge — egress loopback-pin (``live/executor.py``) → killswitch+CRUCIBLE authority/scope/window/budget → WARDEN tier A1–A3 → m-of-n owner-inclusive destruction quorum, first-failure-wins, phase-escalation/fireteam QUEUES for a signed-operator approval; (5) execute live tool subprocess pinned to 127.0.0.0/8 (or inert) and capture full stdout/stderr; (6) ORACLE re-fire — ``react.intake_result`` turns every LLM claim into a LEAD and only mints a FACT when the deterministic oracle (``oracle_adapter.py`/`challenge_oracle.py``) re-confirms over the retained output and returns a signed evidence ref; (7) record into SEPARATE facts/leads stores; (8) project to the graph read-model where CONFIRMED requires evidence_ref AND signature_ref (``graph/projector.py``); (9) cognition governors re-rank/hint/pivot/block-next — BUDGET ONLY, never gating truth (``agent/cognition.py``); (10) the Detection Mirror re-executes the oracle over the certificate to PROVE the signature (``detection/base.py``); (11) remediation/autopatch acts on FACTs only, gated (``remediation/*``, ``autopatch/loop.py``); (12) an EMIT-ONLY observability span (``observability/recorder.py``); (13) checkpoint the whole state as a signed, append-only spine-snapshot (``agent/checkpoint.py`` + ``live/spine_vigilcore.py``) and loop. Every fail-closed branch (attestation-DENY, parse → ASK_USER, classify-DENY, gate-DENY, gate-QUEUE-for-signed-approval) is explicit. Marked honestly: the keystone + governors + checkpoint + attestation + detection are LIVE/pure; the executor/think/spine binders are LIVE but loopback-pinned/key-gated for validation; the CRUCIBLE oracle drive, Neo4j graph/triage backend, WARDEN SDK hook adapter, live git/sandbox codefix, and OTel collector are DEFERRED (injected thunks over retained context).









12 -
observability
SpineTracer
span
EMIT-ONLY
- records
never
authorize

13 -
write_checkpoint
signed spine-
snapshot -
append-only
spine_vigilcore

Usage-attestation ledger (attestation-first gate)

`integration/vigil_integration/attestation/ledger.py`

`require_attestation` is the FIRST call of every turn: it mints a signed `UsageAttestation` binding WHO (OperatorIdentity: OS login + git name/email + Ed25519 key fingerprint + hostname), WHEN (wall `at` plus a non-decreasing monotonic anchor) and WHAT (action/target/phase), all secret-redacted, then returns an `AttestationVerdict`. Append-only, hash-chained, Ed25519-signed; `verify_ledger` re-checks the whole presented chain plus an out-of-band pinned head/count to catch tail-truncation or wipe.

FEATURES

mint signed `UsageAttestation` BEFORE any gated action

WHO/WHEN/WHAT binding + F3 secret redaction at mint time

append-only hash-chained Ed25519 ledger; `verify_ledger` with pinned

Live Claude think-step binder

`integration/vigil_integration/live/think_claude.py`

Replaces the F2 injected think-thunk with a real Claude call: wraps every attacker-influenceable input (prior tool output, prior-action digest) in a random-nonce untrusted boundary (`safety.prompt_safety.wrap_untrusted`), masks secrets via the F3 `redact_tool_args`, makes ONE structured `client.messages.create` call, and hands the raw text to `react.parse_decision`. Keyless-live via an injected replay (`ReplayThinker`) when no API key exists.

FEATURES

untrusted-context nonce framing + standing UNTRUSTED_OUTPUT_GUIDANCE

injected client (fake in tests / real in prod)

keyless-live replay path

secret-free prompt

TOOLS

anthropic SDK (Claude)

ReAct keystone — `parse_decision` / `classify_edge` / `authorize_edge` / `intake_result`

`integration/vigil_integration/agent/react.py`

The sovereign interposition over redamon's ReAct shape. `parse_decision` parses raw text fail-closed into a typed `LLMDecision` (garbage → safest ASK_USER; a broken `deploy_fireteam` never becomes a deploy). `classify_edge` is pure policy: inert actions pass, target-touching tool calls demand the conjunctive gate at the phase tier, phase-escalation/fireteam demand signed approval, structurally-invalid edges are DENIED. `authorize_edge` routes the edge through the INJECTED conjunctive gate fail-closed. `intake_result` turns every LLM `output_analysis` claim into a LEAD and only mints a FACT when the injected oracle returns a non-whitespace signed evidence ref.

FEATURES

Agent state + typed LLM decision

`integration/vigil_integration/agent/state.py`

The typed run state carried across turns: Phase, the 7-action `ActionType`, `ToolCall`, `OutputAnalysis` (LLM claims), `LLMDecision` (the single structured proposal), `Finding`, and `AgentState` with SEPARATE facts/leads stores. `record_fact` requires a signed `evidence_ref`; `record_lead` forces `status=lead/no-evidence`.

FEATURES

fact/lead store separation baked into types

`Finding._fact_needs_evidence` validator

pydantic + stdlib only, import-clean

TOOLS

pydantic

expected_head/expected_count

monotonic anti-back-dating anchor
(attestation/anchor.py: TPM tpm2_readclock preferred, persisted software counter fallback)

LIVE pure core

TOOLS

Ed25519 signer (injected)

tpm2_readclock / /dev/tpm0
(attestation/anchor.py, optional)

attestation/identity.py, models.py

INVARIANT

Fail-closed DENY on no signer / unbound operator / malformed input / durable-write failure — an action whose attestation is not durably recorded cannot proceed; chain order is (seq, prev_hash) only, so `at`/monotonic are signed DATA never ordering keys (no wallclock/RNG in the chain math).

safety/prompt_safety.py

tools.redact_tool_args (F3)

INVARIANT

A think step emits ONLY a non-authoritative PROPOSAL; no key/client/replay → fail-closed to the safest action; the API key is only forwarded to the SDK constructor, never logged; nothing here authorizes — the proposal must still clear the conjunctive gate.

fail-closed decision downgrade ladder

pure EdgeSpec/EdgeVerdict classification

gate + oracle injected (testable without kernel/framework)

LLM claim → LEAD, oracle-confirm → FACT

LIVE keystone, fully tested

TOOLS

injected conjunctive gate thunk

injected OracleFn thunk

safety/llm_intake.parse_proposal

INVARIANT

The LLM/tools only PROPOSE; no action-bearing edge is ever synthesised from unparseable input; no gate wired → tool call DENIED; any gate/oracle error is a DENY / mints no fact; an `allowed=False` but outcome==allow` malformed gate can never present as allow. This is the load-bearing anti-hallucination seam.

INVARIANT

OutputAnalysis fields (esp. exploit_succeeded) are PROPOSALS that flow only into leads; facts grows only through the oracle path; a Finding model validator refuses status='fact' with an empty evidence_ref at the TYPE level, closing the deserialization/checkpoint path.

Phase → **WARDEN tier machine**

integration/vigil_integration/agent/phases.py

Maps the offensive kill-chain phase onto WARDEN kernel tiers (informational → A1, exploitation → A2, post_exploitation → A3) and floors any destructive/high-blast tool at A3. can_transition enforces monotone, one-step escalation (refuses downgrade/skip).

FEATURES

monotone one-step phase ladder

destructive A3 floor

stdlib only

TOOLS

(none — pure policy)

Conjunctive gate

integration/vigil_integration/conjunctive_gate.py

Composes the target-touching authorization as a conjunction over one signed spine: CRUCIBLE authority (killswitch → validity window → scope → destructive → live-destructive → budget) AND WARDEN tool-tier gate AND (for destructive) the m-of-n threshold. First-failure-wins; ALLOW only if in-envelope AND WARDEN auto AND quorum-satisfied, QUEUE if WARDEN needs owner approval, DENY otherwise. Killswitch has absolute precedence (checked first).

FEATURES

WARDEN tool-tier gate

integration/vigil_integration/warden_gate.py

Classifies an offense tool by CLASS/NAME to a tier A0..A3 via an injectable classifier (default: the sigil-kernel binary subprocess), then applies a raise-only floor (default A2) the same max() way the kernel raises a pin, with an A1 ceiling so offense tools QUEUE for owner approval rather than auto-run.

FEATURES

raise-only tool-NAME floor

injectable classifier

posture knob: staging floor A1 for auto-recon

decision core LIVE/tested; SDK hook DEFERRED

Destruction gate (m-of-n threshold authorization)

integration/vigil_integration/destruction_gate.py

For a destructive/high-blast action only, requires a quorum of DISTINCT trusted authorizers (vigil_core.verify_threshold) that MUST include the owner, with action-binding (engagement/target/blast-class/action_id), a bounded dead-man's-switch validity window, and single-use nonce checked against a caller-supplied is_consumed.

FEATURES

RFC-9591 m-of-n authorization

mandatory owner bound at deployment

action-binding + dead-man's-switch + single-use nonce

INVARIANT

Phase escalation is an authority-tier crossing that must clear the conjunctive gate's signed-operator leg — never the LLM's say-so; a destructive tool floors A3 AND additionally needs the m-of-n destruction quorum. Pure/deterministic.

3-way conjunction (CRUCIBLE $\wedge$ WARDEN $\wedge$ m-of-n)

first-failure-wins, fail-closed

pure core + lazy real-CRUCIBLE wiring

TOOLS

framework.v2 CRUCIBLE authorize_action (lazy — DEFERRED live drive)

warden_gate.decide_tool

destruction_gate

INVARIANT

ANY error in any half is a DENY (never caught-and-continued); the pure composition core is injected-thunk testable; build_offense_gate wires the real CRUCIBLE authorize_action via the governance trust root (a None trust root was flagged as the biggest fail-open).

TOOLS

sigil-kernel binary (subprocess, injectable classifier)

INVARIANT

The floor can only RAISE a tier, never lower it; classification is fail-closed to A3; the SDK hook adapter (Strix Runner) fails safe (only an AUTO decision runs) — that live wiring is DEFERRED because the openai-agents SDK is not vendored.

TOOLS

vigil_core.verify_threshold / crypto core

INVARIANT

Highest-consequence gate: a single prompt-injectable worker cannot self-authorize; the mandatory owner set is fixed at deployment in an immutable DestructionAuthority (not a per-call string a worker could rename); first-failure-wins, any error/malformed input is a DENY; single-use survives restart because is_consumed is re-derived from the spine.

Governed live executor (egress loopback pin + output capture)

integration/vigil_integration/live/executor.py

The live choke point replacing the 'run a tool' thunk. Deny-by-default pipeline BEFORE any spawn: (1) loopback pin — resolve target via getaddrinfo, REFUSE unless every address is IPv4 127.0.0.0/8 (denylist.is_egress_denied is an independent second refusal); (2) authorization via tools.authorize_tool_call through the conjunctive gate; (3) argv-LIST build (never a shell string) reconstructed from validated components only; (4) run, capture full stdout/stderr (ExecResult), hash + redact into a signed record.

FEATURES

loopback-only egress pin (127.0.0.0/8)

Oracle adapter (no-hallucinated-findings pipeline)

integration/vigil_integration/oracle_adapter.py

Drives CRUCIBLE's deterministic oracle: confirm_finding re-runs the pure oracle over the retained oracle_context (baseline/mutated responses, probe rounds, timing, OOB hits) and returns None unless an oracle actually FIRED at $\geq$ high-confidence; a signed FACT additionally requires the bug_class to be a KNOWN oracle-mapped class, else it stays a labelled lead. On confirmation it mints a proof-carrying EvidenceCertificate signed m-of-n.

FEATURES

re-execution not string trust

known-oracle-class honesty gate

EvidenceCertificate binding
oracle_context_digest

Challenge oracle (per-run unforgeable verdict)

integration/vigil_integration/challenge_oracle.py

Generalizes DARPA CGC proof-of-vulnerability: each confirmation issues a FRESH unpredictable challenge (nonce/canary/token) and marks a finding VERIFIED only if the live target satisfies THAT specific challenge; the VERIFIED verdict is HMAC-bound to (kind, token) under a per-run oracle key.

FEATURES

fresh per-run challenge

kernel-minted Verified|Abstain (EG-VAR)

HMAC-bound verdict

TOOLS

secrets, hmac, sha256

Spine → graph projector (read-model)

integration/vigil_integration/graph/projector.py

The one-way projector: the graph is written ONLY by project() over a list of signed SpineRecords. A finding becomes a CONFIRMED node only if the record is a FACT carrying BOTH a non-empty evidence_ref AND a signature_ref; everything else is a LEAD node. Bi-temporal (refute sets invalid_from, never deletes); recon bridges are scope-gated.

FEATURES

projection-only from signed spine records

CONFIRMED $\Leftrightarrow$ oracle-signed (evidence_ref $\wedge$ signature_ref)

bi-temporal, never-delete

scope-gated bridges

argv-list, no shell

full stdout/stderr capture with cap + redaction

fail-closed before subprocess

TOOLS

subprocess
(nmap/nuclei/httpx/ffuf/sqlmap/hydra)

vigil_gateway.denylist.is_egress_denied

tools.authorize_tool_call

INARIANT

Going live changes NOTHING about the contract: LLM proposes, only the conjunctive gate authorizes, and traffic may only reach loopback for validation; a smuggled second host (127.0.0.1@evil.com) can never reach a non-loopback host (client-independent); execution proceeds ONLY on verdict.allowed. LIVE but loopback-pinned for validation.

TOOLS

framework.v2 CRUCIBLE oracle (lazy)

governance m-of-n signer

INARIANT

No assertion-only path — an LLM's say-so never confirms anything; a fire on an unmapped class is a lead, never a fact; the framework imports are lazy so the sovereign env stays offense-free. Live re-drive of a scope-gated baseline+probe pair is DEFERRED (re-runs retained context).

INARIANT

Replay/hallucination impossibility (not improbability) — a response for an old challenge always ABSTAINS; only the deterministic oracle can MINT a Verified token, so no LLM/critic/confidence path can forge it; the value is Verified|Abstain, never a bare boolean. Pure/stdlib, import-clean.

TOOLS

graph/model.py (typed read-model)

injected scope_gate

INARIANT

The single most dangerous fusion (trust-laundering) — enforced deterministically: nothing an LLM/tool asserts reaches the graph except as a spine record whose veracity is re-derived; the graph is INERT (authorizes nothing); deterministic/rebuildable (sorts by (seq, hash), no wallclock). Live Neo4j backend DEFERRED.

Cognition governors (non-authoritative budget layer)

integration/vigil_integration/agent/cognition.py

Scores the run's productivity/stall/loop over the execution_trace and escalates the agent's own governance (green → inject_hint → require_deep_think → require_pivot → block_next_expensive_call). audit_productivity_claim cross-checks the LLM's self-reported progress against the MEASURED state delta (non-empty collection, not a truthy scalar); detect_uniform_response_anomaly flags a short-circuit streak as INCONCLUSIVE not

Detection Mirror (proves the signature)

integration/vigil_integration/detection/base.py

A DetectionOracle reads retained telemetry and either FIRES or stays silent (benign twin must stay silent). A FACT is minted ONLY when the oracle fires AND a signed DetectionCertificate is produced AND reverify_certificate RE-EXECUTES the named oracle over the certificate's embedded evidence and the signature verifies; anything softer degrades to a LEAD.

FEATURES

fires on true signature only; benign twin silent

Remediation (triage + gated codefix)

integration/vigil_integration/remediation/triage.py

The deterministic triage half of CypherFix: 9 fixed queries over the PROJECTED GraphView (zero LLM cost) collect/dedup/prioritize into a draft; a TriageFinding may spawn a remediation ONLY if it is an oracle-confirmed FACT (a CONFIRMED graph node with a signed evidence_ref). codefix.py is the pure, injected, tiered pipeline (clone A1 / edit A2 / build A3 / open-PR A3-destructive + m-of-n) that never git-add-A and re-fires the ORIGINAL oracle before signing 'remediated'.

AIxCC auto-patch loop

integration/vigil_integration/autopatch/loop.py

Layers propose → apply-in-sandbox → build → open-PR → verify on top of the gated codefix boundary and the fsjob path-confinement kernel. The coder LLM only PROPOSES a minimal unified diff, parsed fail-closed (parse_unified_diff) with every path re-validated through is_safe_repo_path AND fsjob lexical confinement.

FEATURES

fail-closed unified-diff parse

fsjob sandbox path confinement

tiered gated stages + m-of-n on PR

deadline-based approval, no auto-accept

NEGATIVE (kills the hallucinated 'vector tested, target safe').

FEATURES 5-signal weighted stall/loop score

honesty audit vs measured delta

uniform-response INCONCLUSIVE guard

deep-think novelty guard

tested-axes semantic ledger

TOOLS hashlib/json/re/collections (stdlib only)

INVARIANT

BUDGET/SCHEDULING only — a verdict/score/anomaly NEVER gates a finding's truth (the oracle's sole job); this module never imports/constructs/mutates a Finding or facts store; every function is total on untrusted tool output (degrades to 'no signal', never raises). Pure/deterministic, LIVE.

reverify = re-run oracle over embedded evidence

FACT/LEAD grading

offline re-verifiable PCF certificate (detection/certificate.py)

TOOLS vigil_core.crypto.verify_one

detection/registry.py resolver

mcp_registry._redact_str (F3)

INVARIANT

Re-execution, not string trust — a recorded/forged certificate whose evidence does not reproduce the fire cannot re-verify; the benign twin firing is a BLOCK (false-positive control); deterministic/total/offense-free (reads telemetry, performs NO egress). This is the defensive mirror of CRUCIBLE's evidence cert. LIVE pure.

FEATURES 9 deterministic graph queries

FACT-only remediation spawn

tiered gated stages, never git-add-A

verify-before-sign via oracle re-fire

TOOLS vigil_integration.graph read-model

injected conjunctive gate + git/sandbox/Claude thunks

INVARIANT

A LEAD can NEVER trigger a codefix (enforced at the TYPE level and the spawn boundary); the per-block approval TIMEOUT auto-REJECTS (inverting redamon's auto-accept); 'remediated' is signed only when the exploit oracle goes SILENT. Deterministic; live git/sandbox/Claude DEFERRED (injected thunks).

TOOLS remediation.codefix normalizers

fsjob (fs/jobs/sandbox)

injected coder LLM + gate + clock

INVARIANT

Confirmed-only entry (a LEAD is never auto-patched however severe/KEV-flagged); deadline approval auto-REJECTS on timeout (deterministic injected clock); only explicit path-validated files are staged; 'remediated' signed only when the fix oracle goes silent. Live git/sandbox DEFERRED.

Observability recorder (emit-only span)

integration/vigil_integration/observability/recorder.py

Passive EMIT-ONLY plane: pure builders (new_span/complete_span/new_observation) turn injected spine identity + already-redacted attributes into typed OTel/Langfuse records; guardrail_observation records a WARDEN block and evaluator_observation records an oracle confirm/refute. SpineTracer pushes each record to an injected sink.

FEATURES

OTel span + Langfuse observation taxonomy

emit-only, no authorization surface

frozen records, deterministic ids

Spine-snapshot checkpoint + live signed-spine binder

integration/vigil_integration/agent/heckpoint.py

Replaces the mutable MemorySaver/Postgres checkpointer with an append-only signed spine snapshot: each turn serialises the whole AgentState into a content-hashed, signed SnapshotRecord (hash over (seq, engagement, prev_hash, state_json)); rebuild replays records deterministically (latest valid (seq,hash) wins, torn tail skipped). live/spine_vigilcore.py binds the injected signer/verify/writer/reader thunks to a real Ed25519 key and an append-only one-record-per-line spine file.

FEATURES

F3 redaction at construction

TOOLS

observability/model.py, identity.py

injected exporter sink (live/otel_export.py
DEFERRED)

INVARIANT

Never authorizes — no function returns allow/deny/tier; a WARDEN outcome or oracle verdict is recorded as an inert redacted string that nothing reads back to gate/mint; records are frozen; total on malformed input (a raising sink is swallowed as a 'dropped' counter, never denies cognition); every attribute key/value F3-redacted. Live OTel collector DEFERRED (injected sink).

append-only signed content-hashed snapshots

two-gate fact-store soundness on rebuild

deny-by-default verification on the read path

live Ed25519 + append-only spine file binder
(LIVE)

TOOLS

vigil_core.crypto (Ed25519), vigil_core.chain

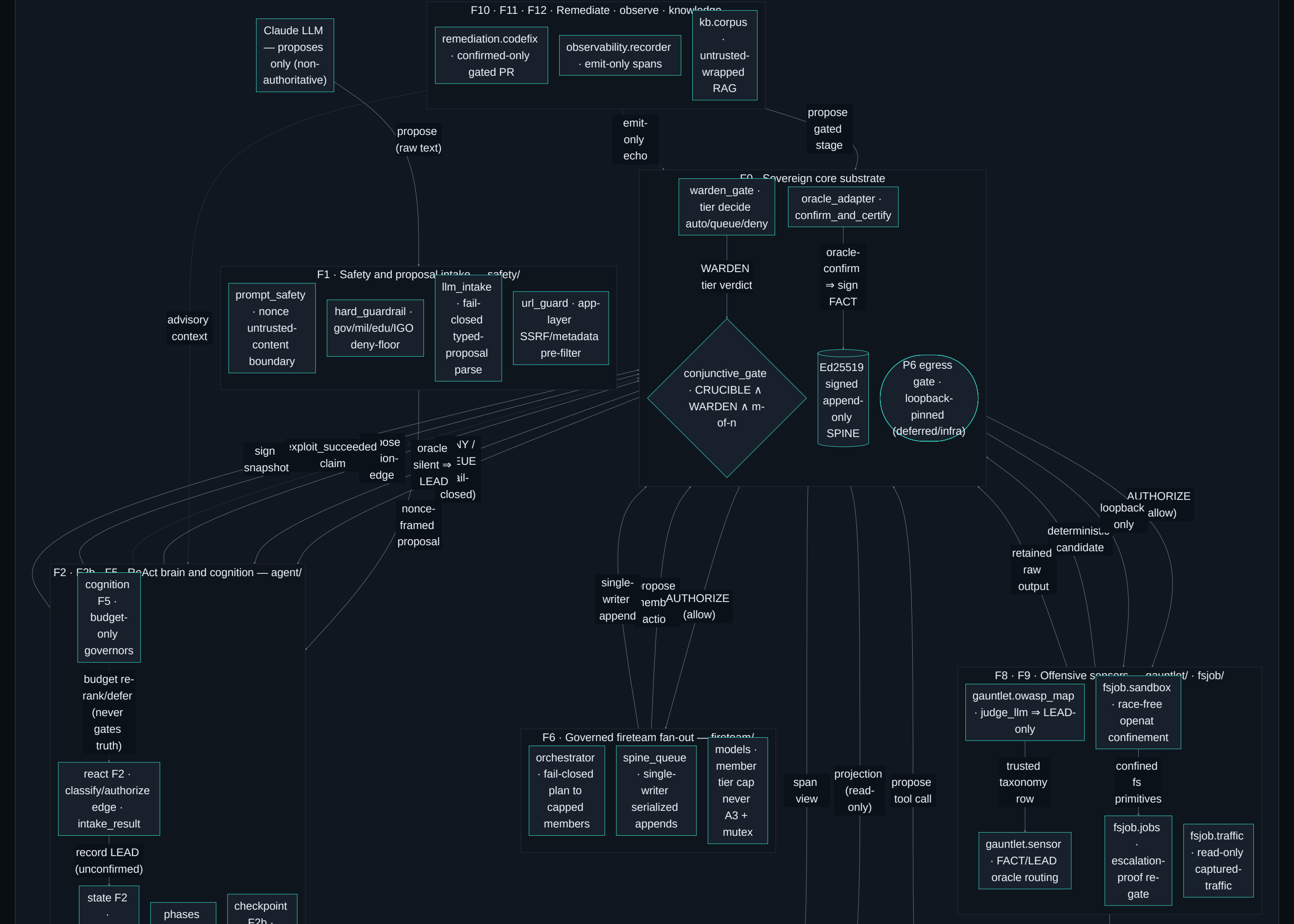
injected signer/verify/writer/reader thanks

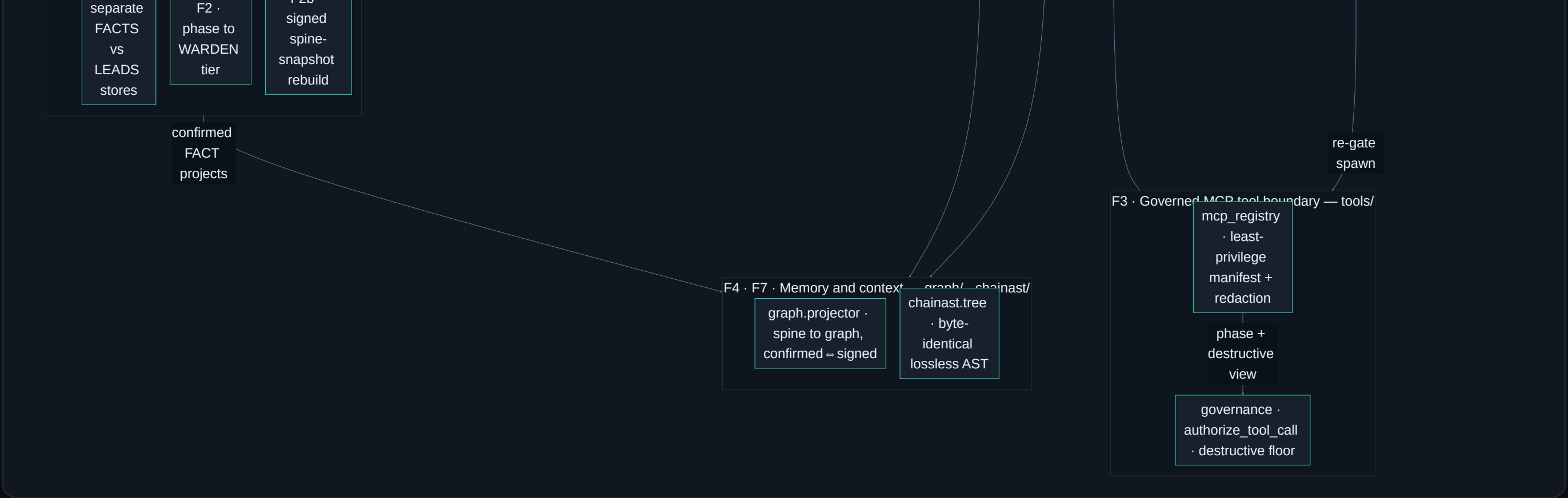
INVARIANT

A Finding in facts round-trips ONLY with its signed evidence_ref intact — _load_state runs the Finding validator AND _facts_store_is_sound (store-membership re-check) so no status spelling ('Fact'/'fact'/'lead') can launder an evidence-less fact into facts[]; DENY-BY-DEFAULT read path (no verifier + no explicit trust_unverified → fresh empty state); deterministic/total, no wallclock/RNG (seq is the temporal coordinate); a durable-write failure surfaces (fail-open on durability forbidden).

VIGIL Fusion Body — sovereign ReAct offense engine, grouped by kill-chain role (integration/vigil_integration/)

L3 is the "body" bolted onto VIGIL's sovereign core. It absorbs redamon (Python AI-hacker) and pentagi (Go) shapes but re-plumbs every action edge through the governing invariant: the LLM, tools, graph, cognition-governors and KB ONLY PROPOSE; the deterministic CRUCIBLE oracle is the SOLE minter of a signed FACT; the conjunctive gate (CRUCIBLE authority $\wedge$ WARDEN tier $\wedge$ m-of-n destruction) is the SOLE authorizer of an action; the P6 egress gate is the SOLE traffic exit (loopback-pinned for live validation); and every durable record lands on an append-only Ed25519 signed-hash SPINE. Every module is pure/injected (gate, oracle, signer, spine writer, subprocess boundary are all callables), fail-closed, deterministic (no wallclock/RNG), and total on untrusted input. Findings are exactly FACT (oracle-confirmed + signed evidence_ref) or LEAD (proposal). Phase → module map: F0 sovereign-core seam (conjunctive_gate/warden_gate/oracle_adapter, plus the P6 egress gate that lives in the gateway package and is exercised only at live-fire); F1 safety/; F2 agent/react+state+phases; F2b agent/checkpoint; F5 agent/cognition; F3 tools/; F4 graph/; F6 fireteam/; F7 chainast/; F8 gauntlet/; F9 fsjob/; F10 remediation/; F11 observability/; F12 kb/. Honesty notes grounded in the code: F3's live MCP client, F9's live job coroutines, F10's live git/sandbox/Claude, F12's live crawl, F11's OTel collector, and the loopback egress enforcement are all DEFERRED to live-fire — the merged slices are the pure decision cores + witnessed provenance, fully tested without a live kernel.





F0 · conjunctive_gate

/home/kali/vigil/integration/vigil_integrat
egration/conjunctive_gate.py

conjunctive_decide composes three orthogonal checks first-failure-wins: CRUCIBLE authority (killswitch → window → scope → destructive → budget) and WARDEN tool-tier decide, and — for destructive actions only — the m-of-n threshold-destruction gate. build_offense_gate wires the real CRUCIBLE authorize_action (loaded verified via the governance trust_root) + the WARDEN classifier, cross-binding the quorum action's target/engagement to the gate's so one ALLOW can't pair scope-check(A) with quorum(B).

- FEATURES
- first-failure-wins conjunction
- strict-is-True destruction check

F0 · warden_gate

/home/kali/vigil/integration/vigil_integrat
egration/warden_gate.py

Raise-only tool-NAME tier floor: an injectable classifier (default the sigil-kernel subprocess, fail-closed to A3) maps a tool name to A0..A3; a default A2 floor is imposed via max() so read-shaped names (http.get) can't auto-run. With offense ceiling A1, every offense tool QUEUES for owner approval; only tiers ≤ auto-bar auto-run.

- FEATURES
- raise-only max() floor
- injectable classifier
- danger-first whole-token classification
- SDK hook wiring deferred (fails safe)
- TOOLS
- sigil-kernel binary (subprocess)

F0 · oracle_adapter

/home/kali/vigil/integration/vigil_integrat
egration/oracle_adapter.py

The no-hallucinated-findings bridge. confirm_and_certify re-runs CRUCIBLE's deterministic oracle over a finding's retained oracle_context; only if an oracle FIRED at ≥high-confidence AND the confirmed bug_class is a known oracle-mapped class does it mint+sign a proof-carrying EvidenceCertificate (m-of-n). certify_to_scitt bridges a FACT to OpenVEX/DSSE/SCITT.

- FEATURES
- oracle-mapped-only signed facts
- proof-carrying certificate binding oracle_context_digest
- OracleKind .value canonicalization
- lazy framework imports
- TOOLS

F1 · hard_guardrail

/home/kali/vigil/integration/vigil_integrat
egration/safety/hard_guardrail.py

A categorically-never denylist for government/military/educational/intergovernmental targets, evaluated before the charter or any gate, with no LLM/network/settings. Folds unicode-dot homoglyphs + NFKC, checks EVERY mainstream client host reading (requests/WHATWG fold-backslash AND httpx) so backslash/userinfo authority-confusion can't smuggle a protected host.

- FEATURES
- client-independent host extraction
- curated IGO exact-domain set + TLD suffix regex
- unicode-dot fold
- runs before charter/gate
- TOOLS
- stdlib re/unicodedata only

target/engagement cross-binding

lazy framework import (import-clean for sovereign side)

TOOLS

CRUCIBLE framework.v2.authority.gate

warden_gate.decide_tool

destruction_gate.authorize_destruction

INVARIANT

ONLY this gate authorizes an action. ALLOW requires CRUCIBLE in-envelope AND WARDEN 'auto' AND (if destructive) strict authorized is True; any error/refusal/unrecognized outcome, or a None trust_root, is a DENY (fail-closed).

INVARIANT

The name floor can only RAISE a tier, never lower it; classification is fail-closed to A3; offense never auto-runs (auto only at/below A1 ceiling).

CRUCIBLE

framework.v2.verify.confirmation/verifier

framework.v2.evidence.certify

scitt.mint_finding_statement

INVARIANT

ONLY a deterministic oracle fire (never an LLM's say-so) mints a FACT; a fire on an unmapped class stays a labelled LEAD; refuses to sign with zero governance signers (fail-closed honesty invariant).

INVARIANT

A pure predicate that can ONLY DENY and cannot be toggled off; blocks if ANY client reading resolves to a protected host; raises HardBlockError fail-closed (never caught-and-continued).

F1 · llm_intake

/home/kali/vigil/integration/vigil_integration/safety/llm_intake.py

Hardened LLM proposal intake: transient/permanent error classifier + exponential backoff retry of a Claude-native zero-arg async callable, once-only param self-heal (drop an unsupported param and retry), and a fail-closed JSON → typed-proposal boundary (fenced/balanced-span extraction + best-effort bracket-stack repair) that returns a caller's safe default on any parse failure.

FEATURES

Claude-native retry (no LangChain)

once-only param self-heal

string-aware JSON repair

RecursionError-safe parse

TOOLS

F1 · url_guard

/home/kali/vigil/integration/vigil_integration/safety/url_guard.py

App-layer SSRF/metadata pre-filter behind (never replacing) the P6 host egress gate: requires http/https, blocks cloud-metadata hostnames, resolves the host FIRST and rejects if any resolved A/AAAA is on the egress denylist (delegated to vigil_gateway.denylist), and refuses plaintext http to a public host. Folds backslash → slash before parsing to match the real client.

FEATURES

resolve-then-classify (defeats DNS-name tricks)

embedded-IPv4 unwrap via gateway

injectable resolver for tests

lazy gateway import

TOOLS

socket.getaddrinfo

F1 · prompt_safety

/home/kali/vigil/integration/vigil_integration/safety/prompt_safety.py

A prepared-statement for LLM context: wraps every attacker-controllable text (tool output, KB pages, captured responses, graph data) in a one-time secrets.token_hex(8) nonce boundary, defangs any attacker-supplied UNTRUSTED_ marker with zero-width spaces, and ships the standing UNTRUSTED_OUTPUT_GUIDANCE directive telling the model to treat marker-bounded text as inert DATA.

FEATURES

unforgeable 64-bit nonce boundary

marker defang

inline single-line variant

label sanitization

TOOLS

stdlib secrets/re

F2 · agent.react

/home/kali/vigil/integration/vigil_integration/agent/react.py

The sovereign ReAct interposition (F2 keystone). parse_decision fail-closed-parses the LLM response into a typed LLMDecision (a broken deploy_fireteam never becomes a deploy; total parse failure pauses for a human). classify_edge/authorize_edge route EVERY action-bearing edge through the injected conjunctive gate at the phase tier; phase-escalation/fireteam-deploy additionally QUEUE for signed approval. intake_result turns output_analysis claims into LEADs and re-fires the injected oracle over retained raw output for an exploit_succeeded claim.

FEATURES

fail-closed action downgrade

edge classification vs authorization split

oracle-interposed intake

stdlib asyncio/json/re; injected validator
(pydantic model_validate)

INVARIANT
Nothing here makes anything true — output is a non-authoritative PROPOSAL; any extraction/validation failure yields the fail-closed default, never executed garbage; word-boundary status regex so '500' doesn't fire on '50000'.

vigil_gateway.denylist.is_egress_denied

INVARIANT
Can ONLY DENY (a pre-filter, not an authority); fail-closed on unparseable URL / unresolvable host / any denied resolved IP; delegates IP classification to the single denylist source of truth (no drift-prone second copy).

INVARIANT
Structural (not advisory) prompt-injection defense — the nonce is unforgeable per call; this HARDENS the proposing LLM and is NOT an authority (nothing inside a boundary is a fact or authorization).

malformed-gate normalization (allowed derived, not trusted)

TOOLS injected gate callable
injected OracleFn (wraps oracle_adapter.confirm_and_certify)
safety.llm_intake.parse_proposal

INVARIANT
The LLM/tools ONLY propose; a FACT is minted only by an oracle confirmation returning a non-empty signed evidence_ref (whitespace/garbage ref mints nothing); no gate wired ⇒ tool call denied; any gate error ⇒ DENY.

F2 · agent.state

/home/kali/vigil/integration/vigil_integration/agent/state.py

Typed ReAct state + the single structured LLMDecision (7 action types). AgentState keeps SEPARATE facts/leads stores: record_lead forces status=lead/no-evidence; record_fact requires a signed evidence_ref. Finding has a model validator (_fact_needs_evidence) enforcing the invariant at the type level, closing the deserialization path.

FEATURES facts/leads separation

7-action decision union

OutputAnalysis claims are proposals only

type-level evidence enforcement

TOOLS pydantic

F2 · agent.phases

/home/kali/vigil/integration/vigil_integration/agent/phases.py

Maps the offensive phase ladder onto WARDEN tiers (informational → A1, exploitation → A2, post_exploitation → A3); a destructive tool floors at A3. can_transition enforces MONOTONE one-step escalation (refuses downgrade/skip); tool_tier applies the destructive floor via TIER_ORDER index.

FEATURES phase → tier table

monotone one-step transitions

destructive A3 floor

pure/deterministic

TOOLS stdlib only

F2b · agent.checkpoint

/home/kali/vigil/integration/vigil_integration/agent/checkpoint.py

Replaces redamon's mutable LangGraph checkpointer with append-only signed spine snapshots: serialize content-hashes (seq, engagement, prev_hash, state_json) canonically and signs via injected signer; rebuild replays records, latest (seq,hash) valid snapshot wins. Deny-by-default read path: refuses to reconstruct without a verifier unless trust_unverified is explicitly set.

FEATURES

deterministic content-hash (no wallclock/RNG)

append-only prev_hash chain

total on torn/forged rows

strict facts-store re-check beyond the type validator

TOOLS

injected signer/verifier/writer/reader thanks

F5 · agent.cognition

/home/kali/vigil/integration/vigil_integration/agent/cognition.py

Non-authoritative cognition governors: 5-signal weighted stall/loop score → escalating BUDGET actions (hint → deep-think → require-pivot → block-next-expensive-call). audit_productivity_claim cross-checks the LLM's 'made progress' against the MEASURED state delta (non-empty collection, not a truthy scalar). detect_uniform_response_anomaly flags short-circuited probe streaks as INCONCLUSIVE (not NEGATIVE), killing hallucinated negatives.

FEATURES honesty audit vs measured delta

uniform-response modal-cluster anomaly

tested-axes semantic dedup ledger

deep-think novelty guard (jaccard + prior-coverage)

TOOLS stdlib hashlib/re/collections

INVARIANT

facts[] grows only through the oracle path; a FACT-status Finding MUST carry a non-whitespace signed evidence_ref (model_validate refuses an evidence-less fact), so a replayed/untrusted record can never construct one.

INVARIANT

Phase escalation is monotone and never on the LLM's say-so — an allowed transition is an authority crossing that must clear the conjunctive gate's signed-operator leg; destructive floors at A3 (which also needs m-of-n).

hashlib/json

INVARIANT

A FACT survives serialise → rebuild ONLY with its signed evidence_ref intact — _load_state runs the Finding validator AND a facts-store soundness re-check (status exactly 'fact' + non-empty evidence) so no case/whitespace status variant can launder an evidence-less fact into facts[]; a lead can never be upgraded.

INVARIANT

Every function is a BUDGET/SCHEDULING governor only — a stall score or anomaly NEVER gates a finding's truth (the oracle's sole job); this module never constructs or mutates a Finding or facts[].

F3 · tools.mcp_registry

/home/kali/vigil/integration/vigil_integration/tools/mcp_registry.py

Pluggable MCP-server manifest layer (pydantic models + cross-server validation + secret redaction + phase views). Sovereign inversions on port: default_phases is least-privilege (informational only, not ALL_PHASES); default_phases_for returns [] (deny everywhere) for an unregistered tool; any server error drops the WHOLE server. Produces the transport config a live MCP client will consume.

FEATURES least-privilege phase defaults

reserved-id/builtin/duplicate collision checks

unified secret redaction

case-insensitive identity

TOOLS

MCP servers (nmap/nuclei/metasploit/playwright reserved); Claude-Agent-SDK MCP client (deferred)

INVARIANT

Deny-by-default: an undeclared tool/phase is refused, not allowed-everywhere; the operator's declared destructive can only

F3 · tools.governance

/home/kali/vigil/integration/vigil_integration/tools/governance.py

The sovereign tool-call boundary. authorize_tool_call runs the fail-closed phase gate FIRST (unregistered/out-of-phase ⇒ DENY before the gate), classifies tier + destructiveness (manifest destructive_view authoritative, a broad known-dangerous-name floor raises it), then routes through the injected conjunctive gate; a destructive call is flagged requires_quorum. redact_tool_args scrubs secrets (bounded-depth) before any spine write.

FEATURES phase-gate-before-gate ordering

raise-only destructive floor

depth-bounded arg redaction

malformed-verdict normalization

TOOLS agent.phases.tool_tier

mcp_registry redaction helpers

injected conjunctive gate

INVARIANT

F4 · graph.projector

/home/kali/vigil/integration/vigil_integration/graph/projector.py

The one-way spine → graph projector — the single most dangerous fusion (trust-laundering), enforced deterministically. project() is the ONLY writer, sorts records by (seq,hash), and re-derives confirmation from signed evidence. Bi-temporal: a refute retires (invalid_from) not deletes. Scope-gated bridges (host/endpoint/port checked client-independently); the graph is inert and authorizes nothing.

FEATURES

deterministic byte-identical GraphView

confirmed vs lead node split

oracle-grounded demotion only

client-independent scope bridges

TOOLS pydantic; injected scope_gate callable

INVARIANT

Confirmed ⇔ oracle-signed: a finding is a CONFIRMED node ONLY if status=='fact' with BOTH a non-empty evidence_ref AND signature_ref; a confirmed fact can be

F6 · fireteam.orchestrator

/home/kali/vigil/integration/vigil_integration/fireteam/orchestrator.py

Governed fan-out/fan-in: validates the plan fail-closed (malformed/oversized/mutex-violating ⇒ refused, spawns nothing), builds each member with a capped tier + deterministic credit/deadline budget, runs the injected member runner under a bounded-concurrency semaphore. A member crash yields an ERROR result (never aborts the wave). All member spine writes drain through the single-writer queue; escalations register PENDING; findings roll up as LEADS.

FEATURES fail-closed whole-plan validation

bounded concurrency

member-crash isolation

single-writer drain

TOOLS

asyncio; injected runner/gate/oracle; SingleWriterSpineQueue

INVARIANT

RAISE blast class; one secret vocabulary (`_is_secret_key`) + one scrubber (`_redact_str`) so redaction can't cover one path but not another. Live MCP client binding is deferred.

Fail-closed everywhere: unregistered/out-of-phase/no-gate/gate-error $\Rightarrow$ DENY; the name floor can only RAISE (never lower) an operator-declared destructive tool; loopback/tier gating subordinates every tool to the sovereign core.

demoted only by an oracle-grounded (signed) refutation; an unauthenticated refute retires only a LEAD (confirmed $\leftrightarrow$ lead split, never launder a claim to a fact).

A member never gets a privileged gate/oracle copy; FACTs are minted solely by the injected oracle via collect; deterministic seq = seq_start+index (no wallclock/RNG); a bad member is isolated.

F6 · fireteam.spine_queue

/home/kali/vigil/integration/vigil_integration/fireteam/spine_queue.py

Serializes all concurrent member spine writes behind ONE injected writer. submit buffers a REDACTED record (F3 scrubber); flush drains in deterministic (seq, member_id, kind) order one at a time; the async write path guards with an `asyncio.Lock` plus a re-entrancy tripwire that raises rather than corrupt the chain. A bad write is isolated (skipped).

FEATURES deterministic drain order

re-entrancy tripwire

per-record write isolation

one F3 redaction path

TOOLS `asyncio.Lock`

`tools.governance.redact_tool_args`

injected writer

INVARIANT

Single-writer append-only serialization — two interleaved appends can never corrupt the Ed25519 hash chain; a member never calls the writer; no writer wired $\Rightarrow$ nothing

F6 · fireteam.models

/home/kali/vigil/integration/vigil_integration/fireteam/models.py

Typed specs/results for the wave. FireteamMemberSpec is frozen with a capped_tier that can never be A3 and a bounded credit; FireteamPlan is validated whole (1..5 members, unique ids, singleton/mutex so two members can't both claim metasploit/burp/sqlmap); parse_fireteam_plan returns None on any malformation. EscalationRequest binds (wave,member,seq) so an approval can't be replayed.

FEATURES A3-forbidden member cap

singleton-tool mutex

frozen tamper-proof specs

deterministic deadline ticks (no wallclock)

TOOLS `pydantic`; `is_destructive_tool` floor

INVARIANT

A sub-agent can NEVER hold A3 (post-exploit/destructive authority is the parent+operator's alone, needing m-of-n); a malformed plan is refused, never partially

F7 · chainast.tree

/home/kali/vigil/integration/vigil_integration/chainast/tree.py

Reversible AST over a contiguous span of signed spine records: parse is a pure/total projection into ChainAST $\rightarrow$ ChainSection $\rightarrow$ {Header,BodyPair}; render flattens back to the exact record list. Because the tree only GROUPS original records (never rewrites), the round-trip is byte-identical by construction. Body-pairs are veracity-tagged; validate reports pentagi invariants without raising; repair is an explicit non-lossless force-mode.

FEATURES total lossless parse

byte-identical render

derived-tags-on-node (never on records)

last-body-pair protection for extended-thinking signature

TOOLS

`pydantic`; `chainast.model`
`normalize/record_confirmed`

INVARIANT

Byte-identical round-trip (re-executability): `to_canonical_bytes(render(parse(x)))==normalize()` a SUMMARIZATION pair is ALWAYS SUMMARY and can NEVER be a FACT even if its records forge status='fact' — a

F8 · gauntlet.sensor

/home/kali/vigil/integration/vigil_integration/gauntlet/sensor.py

The offensive-LLM sensor core: drives an injected subprocess `run_tool` (garak/PyRIT/promptfoo), parses candidates, and routes each by oracle_kind. Deterministic kinds (contains/classifier/regex) re-execute through the injected oracle with a fresh per-run challenge token derived from the injected seed; only a non-empty signed ref mints a FACT. `judge_llm/unmapped` return a LEAD BEFORE any oracle call. Total on malformed tool output.

FEATURES oracle_kind FACT/LEAD routing

deterministic per-run challenge token

trusted-taxonomy chip/id (never raw category)

total on subprocess failure

TOOLS injected GauntletOracle

injected `run_tool` subprocess
(garak/PyRIT/promptfoo/Giskard)

INVARIANT

A judge_llm finding can NEVER auto-promote — routing returns a LEAD before the oracle is consulted, so no adversarial oracle, maxed ASR, or high severity can launder an LLM-judge guess into a FACT;

written (never faked); every record is secret-redacted before touching the spine.

spawned; every member finding is a proposal downgraded to LEAD by collect.

lossy summary must never masquerade as oracle-confirmed.

ASR is a metric, never a promotion signal; no oracle/empty ref ⇒ LEAD (fail-closed).

F8 · gauntlet.owasp_map

/home/kali/vigil/integration/vigil_integrations/gauntlet/owasp_map.py

The OWASP-LLM-Top-10 taxonomy: each probe family maps to (owasp_llm_id, chip, oracle_kind), co-locating WHAT a finding is with HOW it can be confirmed. contains/classifier/regex are deterministic (re-executable → FACT-eligible); judge_llm is non-deterministic (LEAD forever). map_category resolves exact → family → default.

FEATURES deterministic-vs-judge split

family-token reduction

fail-closed judge_llm default

trusted taxonomy constants

TOOLS stdlib re only

INVARIANT

Sovereign inversion of redamon's fail-open: an UNMAPPED category defaults to judge_llm (LEAD-only), never onto the deterministic promotion path — a category we haven't classified can never become a signed fact.

F9 · fsjob.sandbox

/home/kali/vigil/integration/vigil_integrations/fsjob/sandbox.py

The path-confinement kernel: replaces redamon's resolve-then-string-compare (TOCTOU/symlink-race-prone) with a race-free openat walk. A path is validated lexically (no NUL/absolute/escaping ..), then walked one component at a time relative to a pinned real dir fd with O_NOFOLLOW|O_DIRECTORY; any symlink component fails ELOOP and is refused. Every fs mutation executes over the returned dir_fd.

FEATURES race-free openat walk

O_NOFOLLOW every hop

lexical pre-validation + bounds

confined read/write/rename/unlink/listdir primitives

TOOLS stdlib os/stat (openat/dir_fd)

INVARIANT

Sandbox-inescapable by construction — the operation itself (not just a prior check) is confined via pinned fds, so a mid-walk symlink swap is a hard refusal not a followed redirect; deny-by-default (a refusal IS the safe outcome), DoS-bounded, total.

F9 · fsjob.jobs

/home/kali/vigil/integration/vigil_integrations/fsjob/jobs.py

Governed background job runner (on-disk meta, status lifecycle, crash recovery). job.spawn RE-CHECKS the target tool through the exact sovereign boundary a direct call clears (authorize_tool_call → phase gate then conjunctive gate). Deterministic ids, secret-free meta, signed spine events; a spawn whose signing fails rolls back the meta. recover_on_boot flips orphaned jobs to 'interrupted'.

FEATURES

re-gate through authorize_tool_call

deterministic content-addressed job id

signing-fail rollback

fail-closed crash recovery

TOOLS fsjob.sandbox primitives

SpineEventLog (injected signer)

tools.authorize_tool_call/redact_tool_args

INVARIANT

Escalation-proof backgrounding — the agent can NEVER background a tool it couldn't call directly (a permissive gate can't rescue an out-of-phase tool; no gate ⇒ deny); crash recovery fails closed (never silently 'done'); each transition is a signed spine event.

F9 · fsjob.traffic

/home/kali/vigil/integration/vigil_integrations/fsjob/traffic.py

Read-only captured-traffic sensors (search/get/grep/sitemap/params) over an injected STATIC corpus of HTTP transactions — deliberately non-live (no proxy/replay/fuzz/network). grep is a literal fixed-string match (ReDoS-free); all outputs are row/size bounded; headers/bodies scrubbed via the one F3 path; to_llm_digest frames rows through the F1 untrusted boundary.

FEATURES read-only static-corpus tools

literal ReDoS-free grep

one F3 redaction path

F1-framed LLM digest

TOOLS

stdlib urllib.parse; safety.wrap_untrusted; tools.redact_tool_args

INVARIANT

Sensors, not authorities — everything returned is attacker-influenced data → a LEAD only, never a fact; secret-free (Authorization/Cookie masked wholesale); the dangerous active proxy_replay/proxy_fuzz tools are NOT implemented and would need WARDEN tier + conjunctive gate + egress gate.

F10 · remediation.codefix

F11 · observability.recorder

F12 · kb.corpus

/home/kali/vigil/integration/vigil_integration/remediation/codefix.py

The gated destructive codefix pipeline (CypherFix re-plumbed). run_codefix: sovereign confirmed-only check → CLONE/branch(A1) → per-file EDIT(A2, each gated + explicit approval) → sandbox BUILD(A3) → OPEN PR(A3 destructive + m-of-n quorum, staging ONLY explicit validated paths, never git add -A) → VERIFY. Every side effect is an injected callable; steps are secret-scrubbed + deterministic.

FEATURES oracle-confirmed-fact-only entry

timeout⇒reject approval

never git add -A (path validation)

fix-verification oracle before signing

TOOLS

injected
gate/clone/write/build/open_pr/quorum/approve/exploit_ (git, tree-sitter, GitHub PR, sandbox — live deferred)

INVARIANT

Confirmed-only (a LEAD can never fix) via may_spawn_remediation; THE CRITICAL FLIP — the per-block approval TIMEOUT auto-REJECTS (inverting redamon's auto-accept), only an explicit 'approve' applies; 'remediated' is signed ONLY after the original exploit oracle goes SILENT on the patched build and a signer mints a cert (type-level enforced).

/home/kali/vigil/integration/vigil_integration/observability/recorder.py

Emit-only passive recorder + spine-bound builders (new_span/complete_span/new_observation) that turn injected spine identity + redacted attributes into typed records with deterministic ids. Two first-class provable-engine observations: guardrail_observation on a WARDEN block, evaluator_observation on an oracle confirm/refute. SpineTracer holds an injected sink and swallows sink errors (tracked as dropped).

FEATURES

append-only span close (model_copy)

deterministic spine-bound ids

redacted attributes

best-effort swallowed sink

TOOLS

pydantic; injected exporter sink (OTel/Langfuse collector deferred)

INVARIANT

EMIT-ONLY / never authorizes — no function returns an allow/deny/tier; recording a WARDEN outcome or an oracle verdict changes no gate and mints no fact; a confirm echo does NOT mint a FACT (the oracle path does); a sink failure never denies cognition.

/home/kali/vigil/integration/vigil_integration/kb/corpus.py

Governed offensive-corpus RAG (redamon tradecraft_lookup reimplemented). Deterministic lexical ranking (jaccard + 0.5*substr_overlap) with an injected LLM tiebreak spent only on a genuinely ambiguous query; the tiebreak may only SELECT among already-ranked candidates. LookupResult has NO raw-content field — the only content-bearing attribute is the F1 [UNTRUSTED]-wrapped block.

FEATURES deterministic lexical ranking

select-only LLM tiebreak (never injects content)

structural no-raw-content result

offline injected corpus (live crawl deferred)

TOOLS

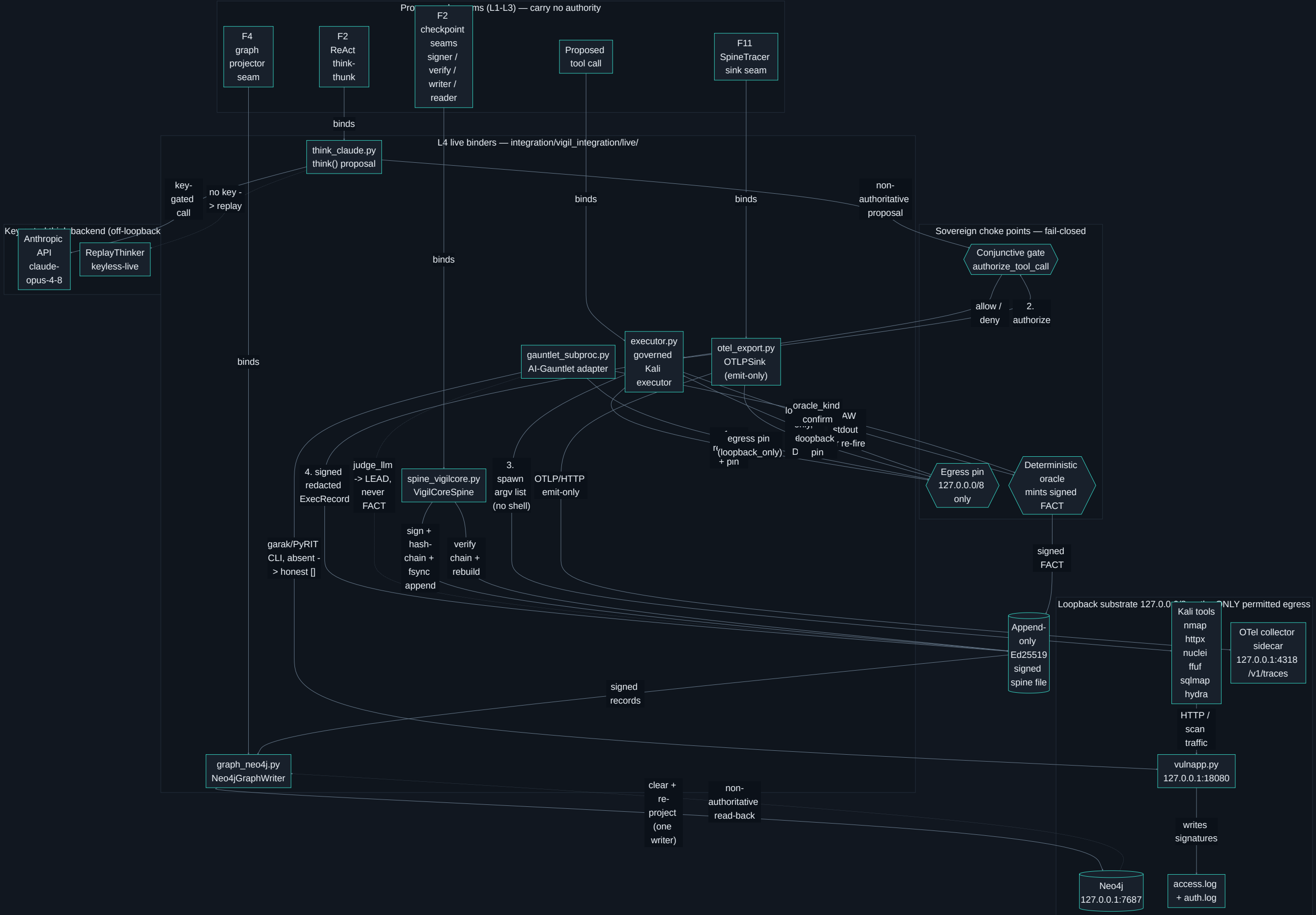
safety.prompt_safety.wrap_untrusted; injected corpus (HackTricks/PayloadsAllTheThings/CVE-PoC) + tiebreak fn

INVARIANT

KB content is a prompt-injection channel — every section body is nonce-wrapped before it can reach a prompt, and it is advisory retrieval context ONLY, never a fact or authorization (a technique still needs the oracle to confirm any resulting finding); total on malformed input.

VIGIL L4 — Live Binders, Sovereign Choke Points, and the Loopback Substrate

L4 is the set of drop-in "live" binders that replace each F1–F12 injected thunk with a real backend, plus the two host sidecars (OTel collector) and the loopback attack target (vulnapp). Every binder is import-clean and unit-testable via injected callables; going live changes NOTHING about the sovereign contract. The LLM/tools/graph/critic only PROPOSE (think_claude emits a non-authoritative LLMDecision; the gauntlet/executor emit candidates); only the deterministic CRUCIBLE oracle mints a signed FACT (re-firing over the executor's RAW output and confirming a gauntlet oracle_kind); only the conjunctive gate (authorize_tool_call) authorizes an action; only the egress gate lets traffic out, and for this validation it is HARD-PINNED to loopback (127.0.0.0/8) at three independent binders (executor resolve+pin, gauntlet loopback_only_egress, OTLPSink endpoint_is_loopback). Every persisted record lands on the append-only Ed25519 signed hash-chain spine file via VigilCoreSpine, and the graph binder mirrors that spine into Neo4j as a pure one-way projection where Confirmed FACT and Lead are physically distinct labels. Honesty notes: the executor supports nmap/nuclei/httpx/ffuf/sqlmap/hydra only — there is NO msf/metasploit builder (an unknown tool is denied); garak/PyRIT are NOT installed on this box, so the gauntlet honestly returns [] rather than a fabricated finding; there is no docker-compose in-repo — the OTel sidecar is a single config file and the loopback port 18080 is the validation convention (vulnapp's own default flag is 8080).



live.executor — governed
live Kali-tool executor

integration/vigil_integration/live/ex
ecutor.py

execute() replaces the F3/F6 'run a tool'
thunk with a real subprocess spawn, running
a deny-by-default 4-stage pipeline that
refuses BEFORE any subprocess: (1)
LOOPBACK PIN — derive the target from
tool_args and resolve via
socket.getaddrinfo, refusing unless EVERY
resolved address is IPv4 loopback
(127.0.0.0/8); a non-loopback, unresolvable,
IPv6 ::1, metadata, or smuggled-second-
host target dies here; (2) AUTHORIZATION
— authorize_tool_call routes phase-
>WARDEN tier AND the injected conjunctive
gate AND, for a destructive tool, the m-of-n
threshold-destruction leg, proceeding only
on verdict.allowed; (3) ARGV BUILD+RUN
— a per-tool builder emits an argv LIST
(never a shell string) reconstructing the
target from VALIDATED components only,
run through the injected run (default
subprocess_runner, shell=False,
timeout+cap); (4) SIGNED RECORD —
hash the RAW streams, write a signed, F3-
redacted ExecRecord, and return RAW
stdout/stderr to the caller so the
deterministic oracle can re-fire over it.

FEATURES

- IPv4-only loopback pin via getaddrinfo
(_resolve_loopback); IPv6 ::1 explicitly refused
- vigil_gateway.denylist.is_egress_denied
consulted as an INDEPENDENT second
refusal (never the allow decision)
- per-tool argv builders that refuse smuggled
hosts / unsafe or missing options (_Pinned
reconstruction)

live.graph_neo4j — live Neo4j
binder for the F4 read-model

integration/vigil_integration/live/gr
aph_neo4j.py

Neo4jGraphWriter mirrors the F4 signed-
spine projection into a real Neo4j database
(injected zero-arg session_factory;
production wiring lambda: driver.session()
against 127.0.0.1:7687). rebuild_from_spine
CLEARs the engagement partition then re-
projects a list of SpineRecord via
graph.project — the ONE writer — so Neo4j
always equals the in-memory projection of
the same spine. Reads (query_confirmed /
query_leads / run_triage) return non-
authoritative retrieval context; run_triage
executes the 9 deterministic F10 triage
Cypher queries (TRIAGE_CYPHER)
matching only :ChainFinding:Confirmed
active nodes, then a Python
correlate/dedup/prioritize pass mirroring
remediation.triage.

FEATURES

- clear + re-project via _CLEAR_CYPHER then
batched UNWIND node/edge upserts
- REMOVE :Confirmed / :Lead before SET so a
stale confirmation label can never survive a
lead re-projection
- strict _FINDING_PROP_KEYS allowlist +
_RESERVED_PROP_KEYS
(system/provenance props always win)
- _coerce_scalar drops unstorable values
(nested maps, out-of-int64, non-finite floats) so
one poisoned prop degrades to a smaller
mirror, not an all-or-nothing driver reject
- evidence_ref written ONLY for a confirmed
FACT node
- 9 named F10 TriageCypher queries +
_dedup_by_ref + _priority_key

live.gauntlet_subproc — live AI-
Gauntlet subprocess adapter

integration/vigil_integration/live/ga
untlet_subproc.py

Wires the F8 offensive-LLM sensor to a
REAL garak/PyRIT (also giskard/promptfoo)
command-line invocation without importing
either framework — they run out-of-process
behind an INJECTED run_tool(argv)-
>envelope. run_gauntlet_report builds an
argv LIST, egress-checks the target, runs
the CLI once, interprets the envelope
(available must be exactly True), then
parses+routes every candidate through the
already-hardened gauntlet core
(parse_adapter_output + route_candidate)
rather than re-implementing routing. On this
validation box garak is NOT installed, so
run_tool honestly reports available:False
and the adapter returns [] — a fail-closed
HONEST no-signal.

FEATURES

- build_argv emits an argv list (no shell, no
interpolation); redacted_argv for logs/spans
- _interpret_envelope reads report aliases
(report/stdout/output/jsonl/raw/results) totally
- ssrf_egress_gate ADAPTER that inverts
is_egress_denied's (denied,reason) polarity
exactly once — the ONLY correct way to wire
the general SSRF gate (never inject
is_egress_denied raw, which would turn every
DENY incl. 169.254.169.254 into an ALLOW)
- _target_host folds backslash->slash to match
the real client and defeat userinfo tricks
- LiveGauntletReport carries honest
available/egress flags +
fact_count/lead_count/overall_asr

TOOLS

live.otel_export — live
OTLP/HTTP exporter sink

integration/vigil_integration/live/ot
el_export.py

OTLPSink is the drop-in for the F11
observability recorder's injected SINK seam
(SpineTracer). It converts each spine-
bound, redacted Span/Observation to an
OTel ReadableSpan and hands it to a lazily-
built OTLP/HTTP exporter pointed at the
collector (default http://127.0.0.1:4318,
normalized to /v1/traces), best-effort. The
exporter is injectable so tests use an in-
memory spy; a real OTLPSpanExporter is
built only when none is injected AND the
endpoint is loopback.

FEATURES

- endpoint_is_loopback: DNS-free loopback gate
(127.0.0.0/8, ::1, or 'localhost' literal; a
127.0.0.1.evil look-alike => False)
- deterministic ids via
observability.identity.derive_trace_id/derive_span_id
- _otlp_scalar flattens values to OTLP-native
scalars, reducing foreign objects to a stable
type token (never str(obj) with a memory
address) to preserve spine determinism
- count-based (no-clock) consecutive-failure
circuit breaker (max_consecutive_failures,
default 3)
- opentelemetry imported lazily/optionally =>
import-clean and inert where absent

TOOLS

- opentelemetry-exporter-otlp-proto-http
(OTLPSpanExporter),
opentelemetry.sdk.trace.ReadableSpan (lazy)
- reuse seams: observability.identity,
observability.model
(Span/Observation/redact_attributes/coerce_*),
tools.mcp_registry._redact_str

F3 redaction: `_redact_arg_list` + tool-specific secret-position masking (e.g. `hydra -p password`) + `_redact_str` over captured streams

ExecRecord commits `stdout_sha256/stderr_sha256` over the RAW streams; deterministic seq/now coordinates (no wallclock/RNG)

DEFAULT_TIMEOUT 120s +
DEFAULT_OUTPUT_CAP 1,000,000
chars/stream

injected gate/run/signer seams => fully unit-testable without a live kernel or real tool

TOOLS

nmap

nuclei

httpx

ffuf

sqlmap

hydra

subprocess.run (shell=False, argv list)

NOTE: no msf/metasploit builder exists — an unknown tool is denied (fail-closed)

reuse seams: `authorize_tool_call`,
`vigil_gateway.denylist.is_egress_denied`,
`tools.mcp_registry._redact_arg_list/_redact_str`

INVARIANT

NO subprocess runs unless the target resolves to 127.0.0.0/8 AND the conjunctive gate allows — both, fail-closed (gate deny/None/exception => no spawn). A smuggled second host/URL can never reach a non-loopback host (argv pins the validated loopback host, client-independent). No shell=True, argv is always a list. Every execution is a signed, REDACTED spine record — no credential leaks into it; RAW output is returned only for the oracle, never persisted. Total on malformed/attacker-influenced input (every failure degrades to DENY, never raises). A destructive tool stays behind the m-of-n destruction-authority leg. With no signer wired the call is REFUSED before any

(severity/KEV/exploit/chain-depth/ref)

TOOLS

Neo4j (injected session.run with parameters); the neo4j driver is the caller's, never imported here

reuse seams: `graph.project / GraphView / NodeLabel / EdgeType`,
`graph.query.FindingSummary`,
`remediation.triage (may_remediate, TriageFinding, severity_rank)`,
`tools.redact_tool_args`

INVARIANT

Projection-only: Neo4j is never a parallel source of truth; nothing an LLM or tool asserts reaches Neo4j except as a signed spine record whose veracity is re-derived by the projector, never trusted. Confirmed <=> a DISTINCT :Confirmed Neo4j label derived from `node.is_confirmed` (the projector's oracle-grounded flag), NEVER from a finding field — a Lead is a different label and is physically unable to appear in a confirmed query; a confirmed row must still carry a signed `evidence_ref` or the TriageFinding validator drops it. No authority is read FROM Neo4j (the conjunctive gate never consults it). Every finding-derived value is a bound Cypher PARAMETER; only enum-whitelisted labels/relationship-types are interpolated. MERGE identity is engagement-scoped {id, engagement_id} so partitions never alias. Fail-closed + total (no factory / session error => ok=False, no partial-truth write, never a raise). Secret-free (F3 `redact_tool_args`, one path) and

garak / PyRIT / giskard / promptfoo CLI via injected `run_tool` (out-of-process; NOT installed on this box)

reuse seams: `gauntlet core (KNOWN_TOOLS, GauntletSpec, run_gauntlet, GauntletOracle)`,
`tools.redact_tool_args`;
`vigil_gateway.denylist.is_egress_denied`
imported LAZILY only as `ssrf_egress_gate`'s default

INVARIANT

Egress-PINNED to loopback for validation (`loopback_only_egress` default; DNS-free, so a hostname that could resolve off-loopback is DENIED). A `judge_llm` (LLM-judge, non-deterministic) candidate is ALWAYS a LEAD, returned before any oracle call. ONLY a deterministic `oracle_kind` (contains/classifier/regex) that the injected randomized-challenge oracle CONFIRMS (returns a non-empty signed evidence ref) mints a signed FACT. ASR is computed as a METRIC only, never a promotion signal. Fail-closed / deny-by-default: no oracle, no `run_tool`, a runner error/timeout, an unknown CLI, a malformed envelope, a non-loopback target, or malformed output => [] / all-LEADS, never a FACT and never a raise. Deterministic (probe.seed-derived challenge token, no clock/RNG). Secret-free (F3 `redact_tool_args` over any argv

INVARIANT

EMIT-ONLY — authorizes nothing: `__call__` returns None; no method returns an allow/deny/tier/verdict; the only state is best-effort health counters (exported/dropped/refused/skipped) an actor cannot branch on. EGRESS PINNED to loopback: a non-loopback endpoint disarms the sink entirely (every `__call__` refused before the exporter is touched; a real exporter is never even built) — this is a loopback PIN (inverse of the offense-side SSRF denylist; the collector is trusted host infra, not an offense target). SECRET-FREE: every attribute key+value AND the two free-string fields (span name, status description) are re-scrubbed through the one F3 vocabulary at the export boundary, safe even for a hand-built record. DETERMINISTIC identity: OTel trace/span ids derive from `spine_hash/seq` (no clock/RNG). A DOWN/failing collector never breaks the engine — errors are swallowed (dropped) and a count-based circuit breaker latches the sink silent. Total on a malformed record => skipped, never raises into the recorder.

spawn (an unrecordable call is unprovable).

deterministic (spine seq, sorted writes => byte-identical Cypher+params).

preview; raw evidence handed only to the oracle).

live.think_claude — live Claude think-step binder

integration/vigil_integration/live/think_claude.py

think() replaces the F2 ReAct think-thunk with a real Claude call (anthropic SDK, default claude-opus-4-8). It frames every attacker-influenceable input — the prior tool output / target data (prompt_ctx) and a redacted prior-action digest from the execution trace — in a one-time random-nonce boundary (wrap_untrusted + UNTRUSTED_OUTPUT_GUIDANCE), makes ONE messages.create call asking for exactly one JSON decision, and parses the raw text FAIL-CLOSED via agent.react.parse_decision. Backend selection, deny-by-default: an injected client wins (its own auth), else a resolvable key (api_key arg or ANTHROPIC_API_KEY) builds a real client, else an injected replay (keyless-live via ReplayThinker) supplies scripted decisions, else the safest ASK_USER.

FEATURES

untrusted context nonce-framed (wrap_untrusted / wrap_untrusted_inline); prior-action args F3-redacted before framing

_MAX_CTX_CHARS / _MAX_RESPONSE_CHARS bounds cap regex cost on hostile oversized output (truncation is always safe-down)

_extract_text total over any client/response shape (SDK object, dict, bare string, fake)

ReplayThinker: stateful, deterministic, side-effect-free scripted backend for keyless-live

live.spine_vigilcore — real Ed25519 signed-spine checkpoint binder

integration/vigil_integration/live/spine_vigilcore.py

VigilCoreSpine is the LIVE drop-in for the F2 checkpoint layer's four injected seams (signer / verify_record / writer / reader), binding them to a real Ed25519 keypair (vigil_core.crypto) and a real append-only spine FILE (one canonical-JSON record per line, hash-chained with vigil_core.chain). write_state serializes+signs an AgentState, hash-chains it, signs the chain link, and fsync-appends one line; rebuild reconstructs the AgentState offline-verified; verify() audits the WHOLE file. Two Ed25519 signatures per line with distinct jobs: the RECORD signature (SnapshotRecord.signature_ref) authenticates the state snapshot for offline rebuild; the ENTRY signature (over the vigil_core chain-link entry_hash) authenticates the append-only file chain so a deletion/reorder cannot be hidden.

FEATURES

_repair_torn_tail truncates a non-newline-terminated final line before any append (SIGIL write-repair lesson)

SpineLine binds cert_digest (record digest) -> entry_hash -> entry signature so any record edit forces a fresh unforgeable signature

reader is lenient (torn-tail tolerant) but the sovereign rebuild path re-verifies every record

_append_line does write + flush + os.fsync (durable append-only); errors surface, never swallowed

infra sidecar — OTel collector config

infra/sidecars/otel-config.yaml

The OpenTelemetry Collector configuration for VIGIL-LIVE: an OTLP/HTTP receiver on 0.0.0.0:4318 (container-internal; the container is intended to bind 127.0.0.1:4318 on the host) with traces/logs/metrics pipelines all exporting to the 'debug' exporter (detailed verbosity), i.e. spans/observations logged to stdout. This is the concrete backend OTLPSink egresses to.

FEATURES

OTLP HTTP receiver on :4318

traces + logs + metrics pipelines -> debug (stdout) exporter

no docker-compose in repo — this is a standalone collector config; the sidecar is deployed by host infra

TOOLS

OpenTelemetry Collector

OTLP/HTTP protocol

debug/stdout exporter

infra loopback target — vulnapp

infra/loopback/vulnapp.py

A deliberately-but-SAFELY vulnerable stdlib-only HTTP app, HARD-PINNED to 127.0.0.1 (run on :18080 for validation; the --port flag defaults to 8080). It exposes genuine, controlled vulnerabilities so a live offensive tool confirms a real bug: string-concatenated SQLi over an in-memory FAKE sqlite DB (sqlmap confirms), reflected XSS of the search q, a path-traversal surface that returns a DECOY /etc/passwd string on an escape attempt, and a /login brute-force/spray surface (single fake valid cred admin/secret123). It writes access.log (CLF-ish, one line per request) and auth.log (one line per /login) — the two log files the AEGIS Detection Mirror consumes to prove the attack's signature from the defensive side.

FEATURES

controlled SQLi + reflected XSS at /search; path-traversal decoy at /file; brute-force surface at /login

discoverable path tree (real 200 vs absent 404) to feed forced-browsing / scanner oracles a 404-rate + path-entropy signal

writes access.log + auth.log under --logdir (thread-locked)

ThreadingHTTPServer, HTTP/1.1

TOOLS

confirmed against by the executor's Kali tools: sqlmap, nuclei, ffuf, hydra, httpx, nmap

stdlib http.server + in-memory sqlite3 (fake rows only)

(degrades to safest ASK_USER when exhausted)

anthropic SDK imported lazily so fake/replay runs need no SDK

TOOLS

Anthropic Claude Messages API via anthropic.Anthropic (claude-opus-4-8, max_tokens 4096) — key-gated

ReplayThinker scripted decisions (keyless-live fallback)

reuse seams: agent.react.parse_decision, agent.state (ActionType/AgentState/LLMDecision), safety.prompt_safety, tools.redact_tool_args

INVARIANT

A think step only ever emits a NON-AUTHORITATIVE proposal — nothing it outputs is a fact or an authorization; a well-formed LLMDecision must still clear the conjunctive gate (agent.react.authorize_edge) before anything runs, and any malformed/garbage/oversized response is downgraded to the SAFEST action (an inert ASK_USER human pause), never an action-bearing edge synthesised from garbage. Secret-free: the API key is only ever forwarded to the SDK client constructor — never logged, never placed in a span/spine record, never in the inspectable request; exception paths log the type name only. Deterministic prompt assembly (sorted keys, bounded

verify() whole-file audit reuses vigil_core.chain.verify_chain from genesis

TOOLS

vigil_core (ChainEntry, append_entry, digest_payload, sign, verify_one) + vigil_core.chain.verify_chain — real Ed25519 crypto and hash-chain

reuse seams: agent.checkpoint (serialize, rebuild_from, head_hash, SnapshotRecord), agent.state.AgentState

INVARIANT

A Finding in AgentState.facts round-trips off the real spine ONLY with its signed evidence_ref intact — rebuild_from re-runs the Finding validator + _facts_store_is_sound on every record, so a forged snapshot (however validly signed) smuggling a 'fact' with an empty/whitespace evidence_ref under ANY status spelling is SKIPPED and never materializes. A torn tail (partial last line from a crash) is repaired/dropped; the good prefix survives. A TAMPERED record (bad record sig, broken hash-chain link, reordered/deleted line, unexpected field) is DETECTED (extra='forbid' + entry sig + record sig + vigil_core chain recompute), never silently trusted. Deterministic + spine-safe (no wallclock/RNG/uuid; temporal coordinate is the injected checkpoint seq + monotonic chain seq); append-only (a recorded line is never mutated). Fail-closed: a bad key / signer outage => writer REFUSES with SpineWriteError — never an unsigned or fabricated line. Secret-free: the private key lives only in process memory. Honest scope limit: a bare append-only file cannot distinguish a tail-truncation rollback from 'not yet written' (anti-rollback = a signed head high-water mark, out of this binder's scope) — but a rollback can only REMOVE records, never ADD an evidence-less fact,

INVARIANT

Internal-only telemetry sink and TRUSTED host infrastructure (not an offense target — the inverse posture from the offense-side loopback denylist). No external exporter is configured, so telemetry never leaves the box; secret-free by construction (the records were already F3-scrubbed and

INVARIANT

Binds 127.0.0.1 ONLY. It never reads real files, runs no real shell, and serves only FAKE in-memory rows; a traversal attempt returns a decoy so the signature fires while leaking nothing real. It is a TARGET, not a service — it wields nothing (no outbound calls, no privileged actions). It exists to give both the offensive oracle and the defensive

truncation, no clock/RNG). Never raises; always returns an LLMDecision.

so the fact/evidence invariant holds under it.

OTLPSink re-scrubs at the export boundary).

Detection Mirror a real, reproducible signature with zero real-data risk.

vigil_integration.live package init

integration/vigil_integration/live/___init__.py

The LIVE bindings package. Its docstring declares that each module is a drop-in for exactly one F1–F12 injected seam (Kali executor, Neo4j writer, garak/PyRIT gauntlet, OTLP exporter, Claude think-step, vigil_core signed-spine checkpoint). Exports are wired by the unified engine (VIGIL-LIVE §12 WS-2) after each binder lands; a direct import (from vigil_integration.live.executor import ...) always works. In the current tree the binders are exercised by the live test suite and referenced by autopatch/loop.py.

FEATURES one module per injected seam

import-clean: each binder loads without a live kernel/sidecar/tool via injected callables

engine-level export wiring deferred to WS-2

TOOLS

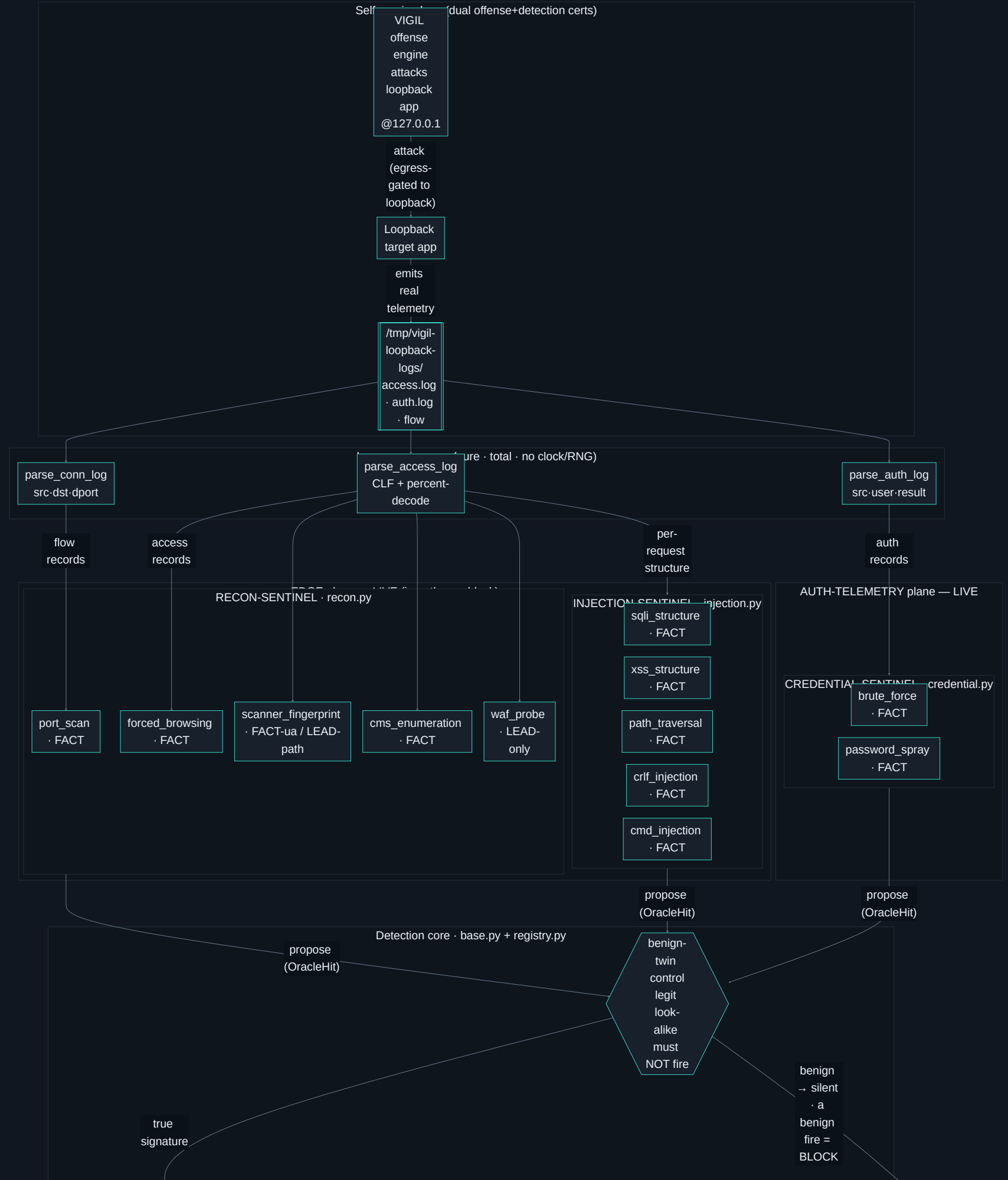
namespace package for the six live binders

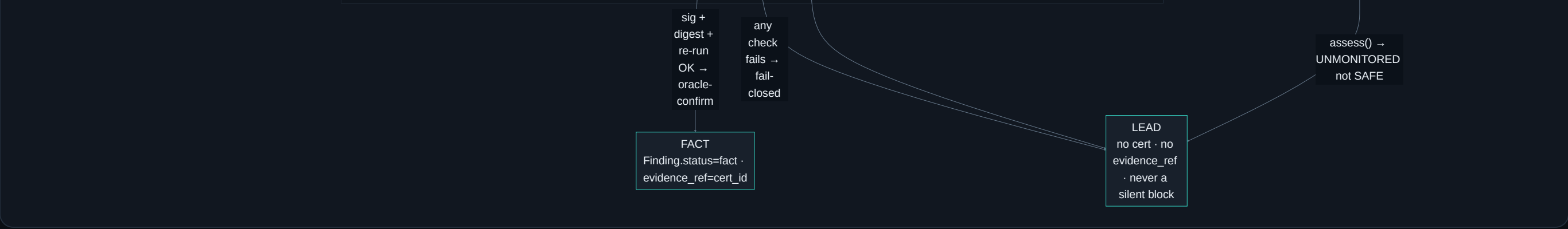
INVARIANT

Going live changes NOTHING about the sovereign contract: the LLM/tools only PROPOSE, only the deterministic oracle mints a signed FACT, only the conjunctive gate authorizes an action, and only the egress gate — pinned to loopback for validation — lets traffic out. The package is a persistence/backend layer, not an authority layer.

L5 — The AEGIS Detection Mirror (the defensive dual)

L5 is VIGIL's defensive dual: for every offensive tool, a DETERMINISTIC detection oracle that PROVES its use against telemetry the systems you protect actually produced — minted as a re-runnable, signed PCF certificate, not an alert. It lives entirely in `integration/vigil_integration/detection/` (9 modules) and inherits the sovereign invariant of the offensive side: the oracle only PROPOSES a fire; a detection becomes a FACT ONLY when the signed `DetectionCertificate` RE-VERIFIES offline (Ed25519 signature + evidence digest + a LIVE oracle RE-RUN over the embedded evidence — proof by re-execution, not string trust); everything softer degrades to a LEAD, never a silent block. Every oracle ships a MANDATORY benign twin (a legitimate look-alike that must NOT fire; a benign twin that fires is a BLOCK). Defensive-only by construction: the Sentinels READ telemetry, wield no tool, and perform NO egress. GROUND TRUTH vs the architecture doc: TWO of the three planes are LIVE — the EDGE plane (`RECON-SENTINEL`: `port_scan/forced_browsing/scanner_fingerprint/cms_enumeration/waf_probe` over `access+conn` logs; `INJECTION-SENTINEL`: `sqli/xss/path_traversal/crlf/cmd` over `access` logs) and the `AUTH-TELEMETRY` plane (`CREDENTIAL-SENTINEL`: `brute_force/password_spray` over `auth.log`) — 12 registered oracle classes, all import-clean and tested. The `TELEMETRY` and `EGRESS` planes (`C2`, `IDENTITY-GRAPH`, `CLOUD`, `SESSION-PHISH` Sentinels) are honest LEAD-only STUBS: no `NetFlow/DNS/DC/CloudTrail/IdP` telemetry is ingested, so `fact_possible=False` by construction — a quiet plane is reported UNMONITORED, never SAFE. Honest doc-vs-code gaps: the doc's `ssti/xxe/deserialization/jwt_tamper` injection oracles and a standalone `credential_stuffing` oracle are NOT implemented (only 5 injection + 2 credential oracles exist); the seven `.claude/agents/*-sentinel.md` files are design-time Claude Code subagents documented in the AEGIS doc, not runtime code. The self-proving loop is real: VIGIL offense attacks the loopback app (127.0.0.1, egress-gated) → the app emits `access.log/auth.log` under `/tmp/vigil-loopback-logs/` → the detection oracle re-reads those real logs and proves the attack's signature, yielding a DUAL certificate pair (offense evidence cert + detection PCF cert), exercised by `test_detection.py::TestLiveSubstrate`.





detection package (public surface)

/home/kali/vigil/integration/vigil_in
tegration/detection/__init__.py

Exports the whole Detection Mirror: the 12 live oracle classes, the DetectionOracle base + Grade/OracleHit/Detection, the DetectionCertificate + crypto helpers, the parsers, the registry run_* helpers + facts()/leads() filters, and the 4 LEAD-only telemetry stubs. Its docstring states the covered planes plainly: EDGE (recon+injection) and AUTH-TELEMETRY (credential) are live; egress/directory/cloud/session are honest LEAD-only stubs.

FEATURES

- single import surface for all detection
- planes-covered honesty note in docstring
- __all__ enumerates live oracles vs stubs

TOOLS

vigil_core (crypto seam)

vigil_integration.agent.state.Finding (F2 typed finding)

DetectionOracle base + FACT gate + re-verification (base.py)

/home/kali/vigil/integration/vigil_in
tegration/detection/base.py

Defines the deterministic oracle base: Grade(FACT/LEAD), OracleHit (a fire + its exact evidence window), Detection (typed result), the windowed()/group_by() rate primitives (axis = records' own ts when all present, else in-group seq — never a wallclock), and detect() — the FACT gate: evaluate() proposes an OracleHit, then a FACT is minted ONLY if grade is FACT AND a signer+verify_key are wired AND the signed certificate re-verifies; otherwise it degrades to a LEAD with an honest note. reverify_certificate() is the anti-hallucination core: ALL of (1) valid Ed25519 signature, (2) recomputed evidence digest == cert's, (3) the named oracle RE-RUN over the cert's own redacted evidence FIRES the same signature family — proof by re-execution, not string trust.

FEATURES

- FACT/LEAD grading
- monotone sliding window over ts-or-seq axis
- re-execution re-verification (sig + digest + live re-run)
- _safe_evaluate total wrapper

PCF detection certificate (certificate.py)

/home/kali/vigil/integration/vigil_in
tegration/detection/certificate.py

The signed, offline-re-verifiable proof object — the defensive mirror of CRUCIBLE's evidence certificate. DetectionCertificate is a pydantic model carrying the oracle id, signature family, verdict, redacted evidence + its digest, reproduction params, injected seq, key_id, and Ed25519 signature. build_certificate() scrubs+digests evidence; sign_certificate() signs the domain-tagged canonical bytes (model minus signature) with an INJECTED signer, fail-closed; verify_certificate_signature() checks it via vigil_core.verify_one; cert_id is the sha256 of the signing bytes (content-addressed → Finding.evidence_ref).

FEATURES

- Ed25519 signature over canonical JSON
- content-addressed cert_id
- F3 secret redaction before signing
- append-only (unsigned input never mutated)
- reproduction params for exact re-run

TOOLS

vigil_core.evidence_signing_bytes /
digest_payload / sha256_hex / verify_one

Oracle registry + operating surface (registry.py)

/home/kali/vigil/integration/vigil_in
tegration/detection/registry.py

ORACLE_CLASSES catalogues all 12 live oracle classes keyed by stable name; resolve_oracle() instantiates a fresh stateless oracle by name for OFFLINE re-verification (thresholds are class constants, so the re-run is identical to the mint). Plane-level helpers parse a log source and run its oracle set on a disjoint seq band so cert ids never collide: run_access_detections (9 edge oracles), run_conn_detections (port_scan), run_auth_detections (brute_force/password_spray), run_all_detections (all three). facts()/leads() partition the results.

FEATURES

- name → class catalogue
- offline oracle resolver for re-verify
- per-plane run_* helpers on disjoint seq bands
- facts()/leads() partition

TOOLS

base.DetectionOracle

logs.py parsers

the recon/injection/credential oracle modules

INVARIANT

The module boundary itself is the honesty boundary — what is exported as an oracle is wired and testable; what is exported as a stub can never mint a FACT. A detection is FACT only when the oracle fires AND a signed cert re-verifies offline; every oracle ships a benign twin; oracles are pure/deterministic/total/secret-free/offense-free.

per-oracle reproduction params embedded in the cert

TOOLS certificate.py (build/sign/verify/redact)

logs.py parsers (for evidence re-parse)

registry.resolve_oracle (lazy, offline re-run)

INVARIANT

Only the oracle proposes; only a re-verified signed cert promotes a fire to FACT. Fail-closed and total throughout: no signer, a signer outage, an un-reproducible cert, an unknown oracle, or attacker-shaped telemetry all degrade to LEAD/no-signal and NEVER raise, never a silent block, never a false re-verify. No clock/RNG; seq is injected.

vigil_integration.tools.mcp_registry._redact_str (F3 redactor)

INVARIANT

Signed via the shared vigil_core seam so signer and verifier hash byte-identically — any tamper to evidence/verdict/oracle breaks the signature. Secret-free: evidence passes through the ONE F3 redactor (_redact_str) BEFORE entering the signed payload, so no credential lands on the spine. Fail-closed: no/raising/blank signer → None (caller degrades to LEAD). Deterministic: same evidence+oracle+seq+signer → byte-identical cert. Owns model+crypto only (no oracle import → no cycle).

INVARIANT

Total (a non-str log → []), deterministic (distinct injected seq per oracle), offense-free (reads telemetry, wields nothing, NO egress). An unknown oracle name resolves to None → fail-closed (can never re-verify). Wiring signer+verify_key is what lets a FACT-grade fire become a signed re-verifiable cert; without them the fire degrades to a LEAD, never a silent block.

Telemetry parsers (logs.py)

/home/kali/vigil/integration/vigil_integration/detection/logs.py

Turns three retained log sources into typed, ordered records the oracles reason over: parse_access_log (Apache/nginx CLF, splitting method/route/query and carrying both the raw percent-encoded target and its unquote-decoded form), parse_auth_log (loopback auth.log: src/user/result), parse_conn_log (flow: src/dst/dport/proto). Timestamps are parsed FROM the log's own string via a pure Howard-Hinnant days-from-civil computation to epoch seconds.

FEATURES

CLF access parser with raw+decoded target

auth.log and conn/flow parsers

pure epoch-seconds timestamp parse (no datetime)

ReDoS-resistant key=value tokenizer

RECON-SENTINEL — perimeter-reconnaissance oracles (recon.py)

/home/kali/vigil/integration/vigil_integration/detection/recon.py

Fires at the earliest kill-chain stage ('they mapped us') over edge/flow telemetry. port_scan: one src sweeping >=15 distinct dports in a window (FACT). forced_browsing: one src hitting >=12 distinct 404 paths (FACT). scanner_fingerprint: a self-identifying scanner User-Agent (nuclei/nikto/sqlmap/masscan/wpscan/ffuf/...) → FACT, else a burst of >=3 distinct infra/secret discovery paths (/ .git, /.env, /server-status, admin) → LEAD. cms_enumeration: >=5 distinct CMS/plugin paths (WPScan) (FACT). waf_probe: a wafw00f UA or >=3 distinct attack classes from one src → LEAD-only per doctrine.

FEATURES port-spread over flow log

INJECTION-SENTINEL — in-path structural oracles (injection.py)

/home/kali/vigil/integration/vigil_integration/detection/injection.py

The RAMPART edge family — recognises a hostile injection CONSTRUCT (not a keyword) in each request's percent-decoded (and, for traversal/crlf, raw) target, firing FACT on the first matching record. sqli_structure (tautology / UNION-SELECT / quote-break / comment / time-fn), xss_structure (<script> / curated event-handler on...= / javascript: call / vector tag), path_traversal (../ or encoded escape or sensitive absolute path), crlf_injection (encoded/raw CR/LF/NUL), cmd_injection (shell substitution or separator+real binary/arg). Structural detectors are module-level pure functions exported via ATTACK_DETECTORS for reuse by waf_probe.

FEATURES

CREDENTIAL-SENTINEL — authentication-attack oracles (credential.py)

/home/kali/vigil/integration/vigil_integration/detection/credential.py

Fires at the credential-access stage over auth.log. brute_force: one ACCOUNT accrues >=8 auth failures in a window (per-account velocity — Hydra/Medusa/Patator hammering one user), FACT. password_spray: one SOURCE fails against >=8 distinct accounts, each <=2 failures, in a window (broad+shallow source-fanout fingerprint), FACT. Evidence for spray is only the shallow-account failures constituting the fanout.

FEATURES per-account failure velocity (brute)

cross-account source-fanout (spray)

shallow-failure evidence selection

explicit password-not-in-log honesty note

total on garbage lines

TOOLS

stdlib re

urllib.parse.unquote

IN VARIANT

NO CLOCK / NO RNG — the ordering axis is the line index (seq) plus the timestamp parsed out of the log text itself (reading data, not a wallclock). TOTAL on malformed/attacker-influenced input: an unparseable line degrades to a record with raw+seq set and empty fields (matches no signature), never raises. Percent-decoding exposes true structure so %27 cannot hide a payload; ReDoS-hardened tokenizer (str.split before anchored per-token match).

distinct-404 density

self-identifying scanner-UA (FACT) vs discovery-path burst (LEAD)

CMS/plugin path walking

waf_probe LEAD-only

TOOLS

MIRRORS: Nmap, masscan, RustScan, ffuf, gobuster, dirsearch, feroxbuster, Nuclei, Nikto, WPScan, wafw00f

reuses injection.ATTACK_DETECTORS for the waf_probe multi-class composite

IN VARIANT

Fires on the true recon rate/spread only; the benign twin stays silent — an uptime monitor (one port/path), a robots-respecting crawler (200s, few 404s), a normal CMS visitor (2-3 paths). curl/python-requests/Googlebot are deliberately absent from the scanner-UA list (legit clients). A path pattern is a LEAD; a tool that names itself is a FACT. Pure/deterministic/total; windows from records' ts/seq.

5 structural classes (sqli/xss/traversal/crlf/cmd)

decoded+raw target scan

curated event-handler & binary lists to kill false positives

16KB ReDoS cap

pure detectors reusable by waf_probe

TOOLS

MIRRORS: sqlmap, Ghauri, Dalfox, XSSStrike, path-traversal/CRLF tooling, command-injection

stdlib re only

IN VARIANT

Benign-twin discipline is a SAFETY property here — these run in-path (can block) so a false positive is an outage: the word 'select' in prose, O'Reilly, escaped <script>, onboarding=, report..2024.pdf, hex '0a', a lone ';'cat' must NOT fire. Scans the percent-decoded target so %27 cannot hide a payload; scan prefix capped at 16 KB (ReDoS defense) → over-long param degrades to no-signal. FACT is minted only via the base oracle's signed-and-re-verified cert path. Doc-vs-code: ssti/xxe/deserialization/jwt_tamper are NOT implemented (only these 5 classes).

TOOLS

MIRRORS: Hydra, Medusa, Patator, credential-stuffing kits

base.group_by / windowed over auth records

IN VARIANT

Benign twins stay silent: a user mistyping 3-4 times (below per-account threshold), a legitimate login surge (many distinct users SUCCEEDING — spray needs cross-account FAILURES; brute needs one account's failures). A deep brute (spread 1) does NOT fire as spray. Honest note recorded on every spray: auth.log retains src/user/result but NOT the password, so it proves the SOURCE-FANOUT fingerprint, not literally 'same password' — stated, not faked. Windows from ts/seq, no clock; total. Doc-vs-code: a standalone credential_stuffing oracle is NOT implemented.

Telemetry + egress LEAD-only stubs — C2 / IDENTITY-GRAPH / CLOUD / SESSION-PHISH Sentinels (telemetry.py)

/home/kali/vigil/integration/vigil_integration/detection/telemetry.py

The four unmonitored detection domains, represented as TelemetryStub objects that name the oracles they WOULD provide

Self-proving loop + adversarial test harness (test_detection.py)

/home/kali/vigil/integration/tests/test_detection.py

Exercises the whole mirror and the self-proving loop: TestLiveSubstrate reads the REAL loopback logs at /tmp/vigil-loopback-logs/access.log & auth.log (written by the loopback target app after VIGIL offense

once telemetry is ingested: C2_STUB (beacon_periodicity, c2_tls_fingerprint, dns_tunnel, exfil_volume — needs NetFlow/proxy/DNS), IDENTITY_GRAPH_STUB (ldap_recon, kerberoast, asrep_roast, dcsync, ticket_anomaly, lsass_access, adcs_abuse — needs DC/LDAP/Kerberos + EDR), CLOUD_STUB (cloud_enumeration, iam_privesc, k8s_attack, container_escape — needs CloudTrail/k8s-audit), SESSION_STUB (session_origin_anomaly, token_replay, oauth_grant_anomaly — needs session/IdP). assess() returns a single honest LEAD with a coverage-gap note naming the absent telemetry.

FEATURES

- 4 stubs covering the telemetry+egress planes
- fact_possible=False deny-by-default
- coverage_note naming absent telemetry
- honest analyst-suspicion folded in as non-authoritative text
- hardest-domain (session) benign-overlap note

TOOLS

- MIRRORS (not yet wired): Cobalt Strike/Sliver/Metasploit, BloodHound/Rubeus/Mimikatz/Certipy, Pacu/kube-hunter, Evilginx2
- vigil_integration.agent.state.Finding

INVARIANT

STATUS: DEFERRED / LEAD-only.
fact_possible=False by construction and available() always False (deny-by-default) — no ingested telemetry, no oracle to fire, so no FACT is possible. Fakes nothing (no synthesized beacons/tickets/API calls). A quiet plane is reported UNMONITORED, never SAFE — 'no log source, no proof'

attacks it @127.0.0.1) and asserts every FACT emitted over the real log re-verifies offline; TestSovereignInvariant is the explicit red-pen pass — it tries to make benign traffic fire, forge a FACT by hand / with the attacker's own key / by cross-oracle evidence swap / by field tamper / by unsigned cert / by unknown-oracle name, and to smuggle a secret onto the spine — each attempt must fail closed.

FEATURES

- live loopback-log substrate test
- benign twin per oracle (must stay silent)
- keyless-forgery / evidence-swap / re-execution-required adversarial cases
- secret-masking-in-cert test
- no-wallclock/no-RNG import guard

TOOLS

- vigil_core.generate_keypair/sign/verify_one
- the detection public surface
- pytest

INVARIANT

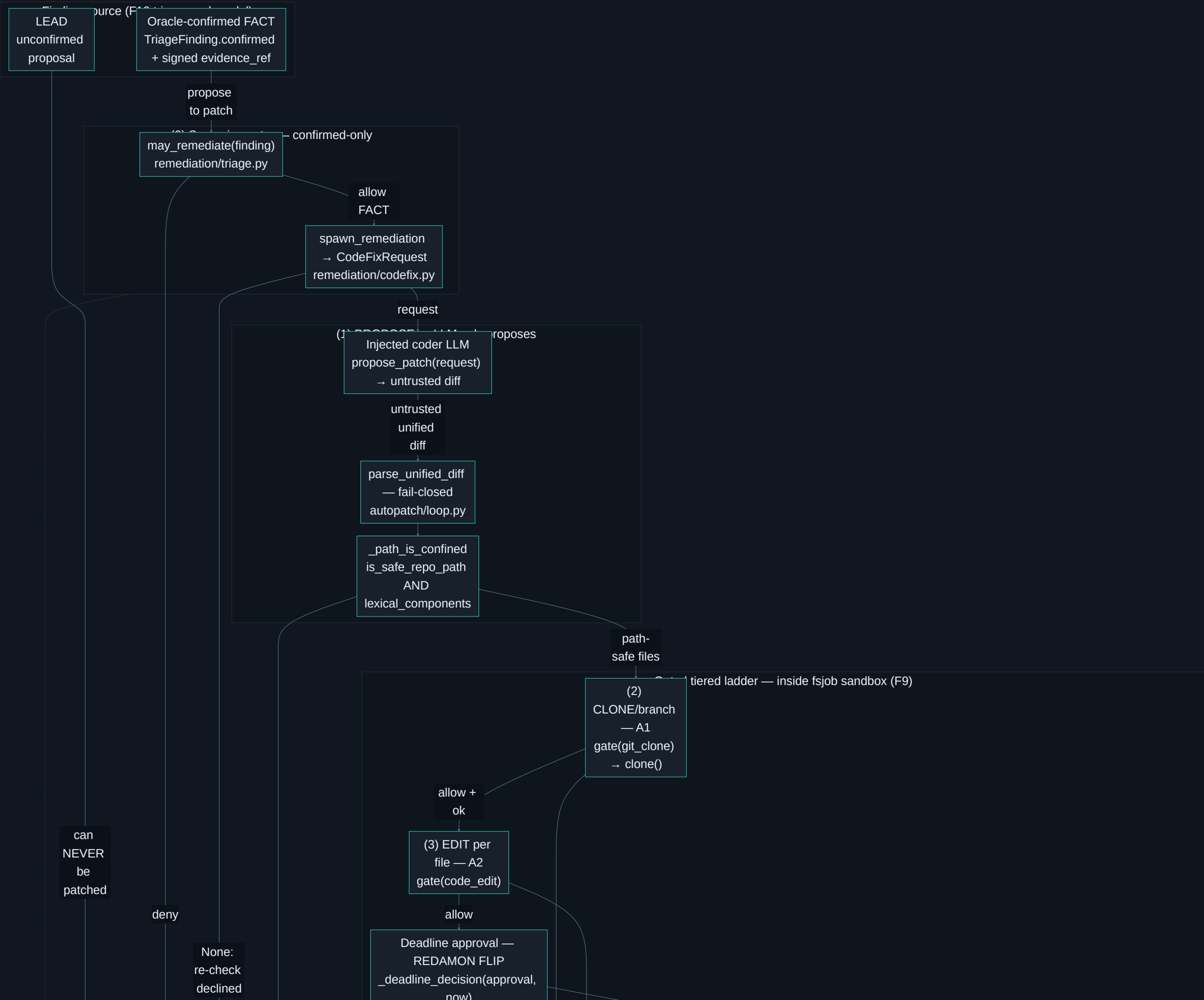
Proves the dual-certificate self-proving loop: offense confirms the attack (offense evidence cert), detection re-reads the resulting real logs and proves the attack's signature (detection PCF cert) — two independent certs over one event. The benign-traffic battery asserts NO benign look-alike across any plane reaches FACT;

stated plainly. When SENSOR-WRIGHT later ingests the telemetry, the named oracles become real fired detections.

no forgery re-verifies under the defender's trust root.

VIGIL L6 — AIXCC Auto-Patch Loop (FACT → gated codefix → fix-verification oracle → signed 'remediated')

Layer L6 (phase 32) is the sovereign auto-patch loop in integration/vigil_integration/autopatch/loop.py, layered on the F10 remediation boundary (remediation/triage.py + remediation/codefix.py) and the F9 fsjob path-confinement kernel (fsjob/sandbox.py). It keeps redamon's AIXCC shape — propose → apply-in-sandbox → build → open-PR → verify — but inverts every fail-open default. Flow: (0) the sovereign entry may_remediate() refuses fail-closed unless the finding is an oracle-confirmed FACT with a signed evidence_ref — a LEAD (however severe/KEV) can NEVER be patched; spawn_remediation re-checks. (1) the injected coder LLM ONLY PROPOSES a minimal unified diff, parsed fail-closed and path-validated twice (is_safe_repo_path AND lexical_components) so no /dev/null, absolute, flag, or '..' path is ever staged. (2) clone/branch=A1, (3) per-file edit=A2 each conjunctive-gated AND deadline-approved — THE REDAMON FLIP: an injected logical clock decides expiry and now()>deadline (or no clock/no approval/non-approve) auto-REJECTS the edit (redamon auto-accepts on timeout). (4) sandbox build=A3 egress-gated. (5) open-PR=A3 destructive AND an explicit m-of-n quorum, staging ONLY the explicit approved files — never 'git add -A'. (6) the single fix-verification ORACLE re-fires the ORIGINAL exploit against the PATCHED build; only oracle SILENCE plus a signed certificate mints 'remediated' (type-enforced: remediated requires a signed evidence_ref AND an opened PR); the exploit still firing → 'still-vulnerable', a missing/erroring/silent-no-cert oracle → 'unverified'. Every stage that denies/fails degrades to _refuse() (fail-closed), every step is a deterministic _Recorder row scrubbed through the F3 redaction path (no wallclock/RNG/uuid; remediation_id derives from signed provenance). All side effects (gate/oracle/LLM/clone/build/open_pr/quorum/approval/now) are injected callables, so the whole loop is total and runs with no live git/build/network/LLM. VERIFIED at runtime: a LEAD yields status refused-not-confirmed, remediated=False.





autopatch loop (the AlxCC auto-patch orchestrator)

integration/vigil_integration/autopatch/loop.py

autopatch(finding, *, gate, oracle, propose_patch, clone, build, open_pr, quorum, approval, now, ...) drives the whole loop in fixed order: (0) sovereign confirmed-only check + spawn_remediation, (1) propose+parse, (2) clone A1, (3) per-file edit A2 gated+deadline-approved, (4) build A3, (5) PR A3-destructive+quorum, (6) verify. Every deny/fail returns _refuse() (fail-closed); success returns a PatchResult whose model_validator makes remediated=True IMPOSSIBLE without a signed evidence_ref AND an opened PR. Total on malformed input, deterministic (no wallclock/RNG; remediation_id derives from the finding's signed provenance via _derive_remediation_id).

FEATURES

parse_unified_diff + _path_is_confined (untrusted-diff parser)

integration/vigil_integration/autopatch/loop.py

parse_unified_diff(text) turns the LLM's untrusted unified diff into path-validated PatchFile proposals, FAIL-CLOSED: non-string / blank / >1MB / >500 files refused; each '--- ' / '+++ ' header pair resolved to a concrete repo path (add/modify/delete, /dev/null handled via a sentinel), duplicates collapsed, never raises. _path_is_confined(path) admits a path ONLY if it clears BOTH the remediation guard is_safe_repo_path AND the fsjob lexical kernel lexical_components (belt-and-suspenders, two independent refusals).

FEATURES byte/file-count bounds on the diff

git-meta segment attribution for per-file diff_text

dual path validation (remediation guard + fsjob

_deadline_decision (the redamon auto-accept FLIP)

integration/vigil_integration/autopatch/loop.py

Decides one file's edit approval under an INJECTED logical clock. Returns 'approve' ONLY when the injected approval callable returns decision=='approve' AND now() <= its carried deadline. Every other branch REJECTS fail-closed: no approval mechanism, no clock, an unusable clock value, an approval with no numeric deadline, now()>deadline (window EXPIRED, regardless of decision), or a raising approval callable.

FEATURES

injected clock (logical time, no wallclock)

expiry wins over decision (past-deadline approve still rejected)

bools rejected as clock/deadline values

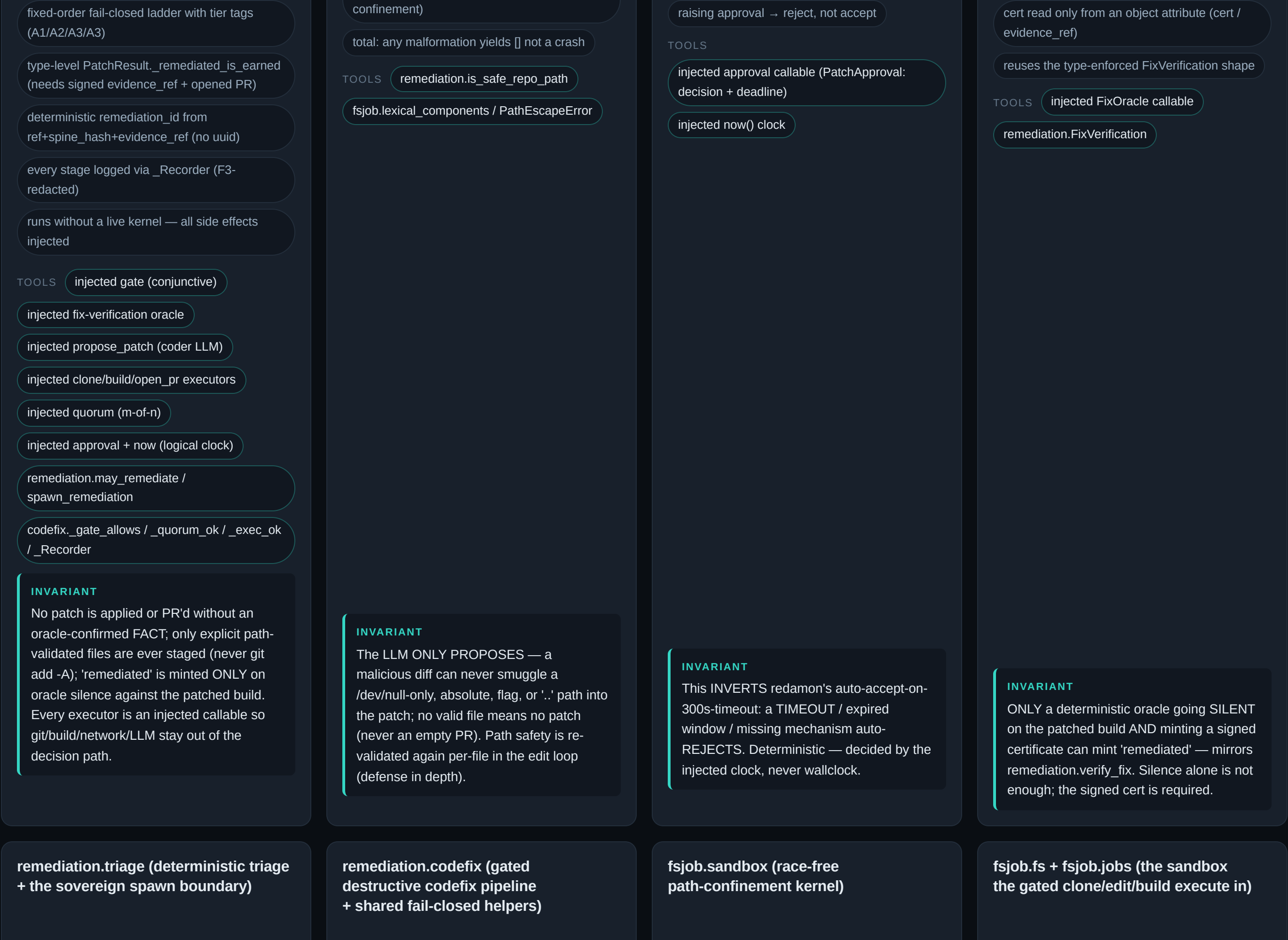
verify_patch (fix-verification oracle adapter)

integration/vigil_integration/autopatch/loop.py

Re-fires the ORIGINAL exploit via the single injected oracle(request, patched_build) and maps its verdict to a FixVerification: no oracle / oracle raises / no usable verdict → 'unverified'; exploit STILL fires (fired True or a truthy exploit ref) → 'still-vulnerable'; oracle SILENT (fired False) with a signed cert attribute → 'remediated'; silent with no cert → 'unverified'. _oracle_fired/_oracle_cert are fail-closed (an ambiguous verdict is never read as a false 'silent'; a bare string is an exploit ref, never a certificate).

FEATURES

default OracleVerdict.fired=True (empty verdict = fail-closed still-vulnerable)



integration/vigil_integration/remediation/triage.py

9 fixed queries (TRIAGE_QUERIES, asserted ==9) over the F4 projected GraphView read-model at zero LLM cost gather ground-truth CONFIRMED finding nodes, then dedup + severity-prioritize into a RemediationDraft (confirmed facts in .findings, unproven proposals SEPARATELY in .leads).

TriageFinding._confirmed_needs_evidence refuses a confirmed finding with no signed ref at the type level; may_remediate(finding) is THE sovereign gate returning (allowed, reason): a non-TriageFinding, a LEAD, or a confirmed finding lacking a signed evidence_ref is refused fail-closed.

FEATURES

- 9 pure-Python queries over the signed-spine GraphView (not raw Cypher)
- confirmation re-derived from node.is_confirmed + signed evidence_ref, not from anyone's word
- deterministic dedup + priority key (severity, KEV, exploit, chain depth, ref)
- total: a torn node / non-GraphView degrades to a smaller/empty draft

TOOLS

vigil_integration.graph GraphView / GraphNode / EdgeType (F4 read-model)

integration/vigil_integration/remediation/codefix.py

The pipeline the auto-patch loop is built on and reuses. spawn_remediation constructs a CodeFixRequest ONLY from a confirmed fact. run_codefix drives clone A1 / edit A2 (timeout=REJECT) / build A3 / PR A3-destructive+m-of-n / verify. autopatch reuses its exact fail-closed normalizers: _gate_allows (derives allowed from BOTH .allowed is True AND .outcome=='allow'; no gate/error/queue/deny → DENY), _quorum_ok (approved must be strictly True), _exec_ok (executor .ok must be strictly True), _Recorder (deterministic seq + F3 redaction), the TIER_* constants (A1/A2/A3/A3), _BULK_TOKENS, and verify_fix. is_safe_repo_path refuses non-string/empty/bulk/flag/absolute/traversal paths.

FEATURES

- stage → WARDEN-tier ladder (TIER_CLONE A1 / TIER_EDIT A2 / TIER_BUILD A3 / TIER_PR A3)
- fail-closed gate/quorum/exec/approval normalizers
- verify_fix: exploit-oracle silence + RemediationSigner before 'remediated'
- secret-free: every payload scrubbed via redact_tool_args
- render_untrusted_finding wraps attacker-influenced text as inert data (F1)

TOOLS

- injected gate / clone / write_file / build / open_pr / quorum / approve_block / exploit_oracle / sign_remediated
- safety.parse_proposal / wrap_untrusted (F1)
- tools.governance.redact_tool_args (F3)

integration/vigil_integration/fsjob/sandbox.py

The single load-bearing confinement primitive the auto-patch loop leans on for path safety. lexical_components validates a path lexically (no NUL/absolute/'..'-'-escape, bounded length/component count) and returns confined components; resolve_within / walk_to_parent perform a race-free openat walk, opening each hop O_NOFOLLOW|O_DIRECTORY relative to a pinned real dir fd, so a mid-walk symlink swap fails ELOOP instead of being followed. ANY symlink component (intermediate or final) is refused. All mutation primitives (write/read/unlink/rename/mkdir/listdir) run over the safe fd, not a re-resolved string.

FEATURES

- openat walk with O_NOFOLLOW (symlink-race-proof)
- lexical pre-validation before touching disk
- DoS bounds (component count/length, path chars)
- stdlib-only, no wallclock/RNG/network

TOOLS

os / stat (openat, O_NOFOLLOW, dir_fd)

integration/vigil_integration/fsjob/fs.py

WorkspaceFS (fs.py) exposes reads plus signed, append-only, REVERSIBLE mutations (write/edit/delete/move/mkdir/extract), each a SpineEvent with pre/post content hashes; the jobs/ subtree is PROTECTED from the agent-facing tools; archive extraction is hardened (tar-slip/symlink-member/zip-bomb). JobRegistry (jobs.py) provides escalation-proof spawn: job_spawn RE-checks the target tool through tools.authorize_tool_call → is_tool_allowed_in_phase BEFORE the conjunctive gate, so a tool that could not be called directly can never be backgrounded; deterministic ids, F3-redacted witnessed provenance, fail-closed crash recovery. These are the disposable, egress-gated sandbox surface the loop's injected clone/build/open_pr executors are expected to run over.

FEATURES

- signed reversible SpineEvent mutations with undo stack
- PROTECTED jobs/ subtree (no forged job metadata)
- hardened archive extraction
- escalation-proof job_spawn (phase re-gate then conjunctive gate)
- deterministic ids + fail-closed crash recovery

TOOLS

- fsjob.sandbox kernel
- fsjob.spine (Signer/SpineEventLog/sha256_hex)
- tools.authorize_tool_call / is_tool_allowed_in_phase / redact_tool_args

remediation.triage.may_remediate /
TriageFinding

INVARIANT

Each stage routes through the injected conjunctive gate (no gate wired / error / queue / deny stops the stage); the per-block approval TIMEOUT auto-REJECTS; only explicit path-validated files are staged (never git add -A); 'remediated' is signed ONLY after the original exploit oracle goes silent AND a signer mints a signed cert. CodeFixResult/FixVerification enforce remediated $\Rightarrow$ (status remediated + signed ref + opened PR) at the type level.

INVARIANT

A remediation may be spawned ONLY from an oracle-confirmed FACT carrying a signed evidence_ref; a LEAD can NEVER trigger a codefix. Enforced twice — at the TYPE level and at the spawn boundary. Confirmed facts and leads live in separate fields (anti-trust-laundering). Deterministic ordering, no wallclock/RNG.

INVARIANT

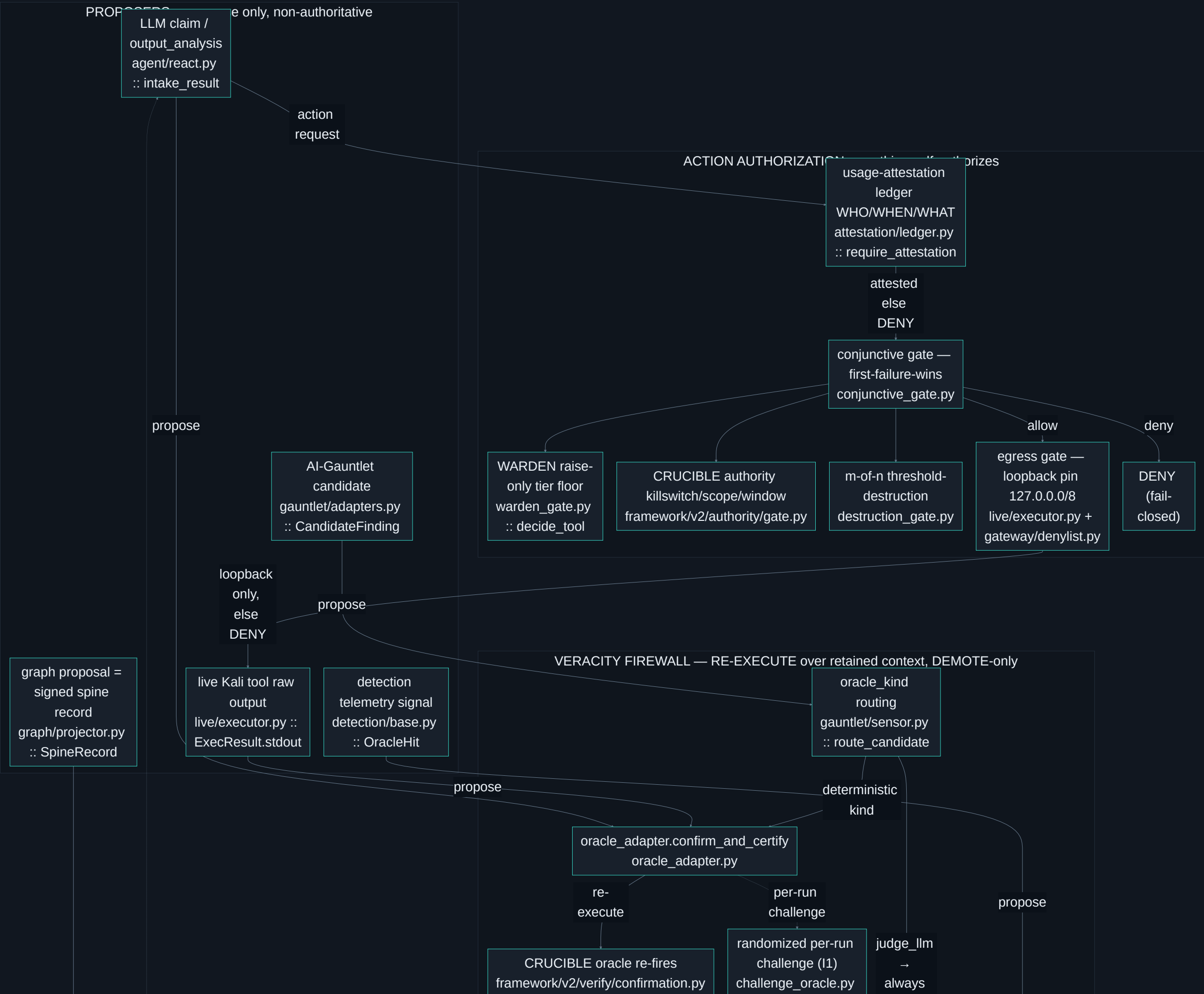
A path can NEVER escape the sandbox root; deny-by-default (a PathEscapeError refusal IS the safe outcome). Replaces redamon's TOCTOU-prone resolve-then-compare with an openat walk, closing the symlink race. autopatch calls lexical_components as its second, independent path refusal.

INVARIANT

Every fs mutation is a signed, reversible spine event (no unsigned mutation); a job can NEVER be backgrounded if a direct call would be refused for the current phase; every public method is total and returns a structured result (never raises). Live subprocess execution is a deferred slice — this governs lifecycle + provenance.

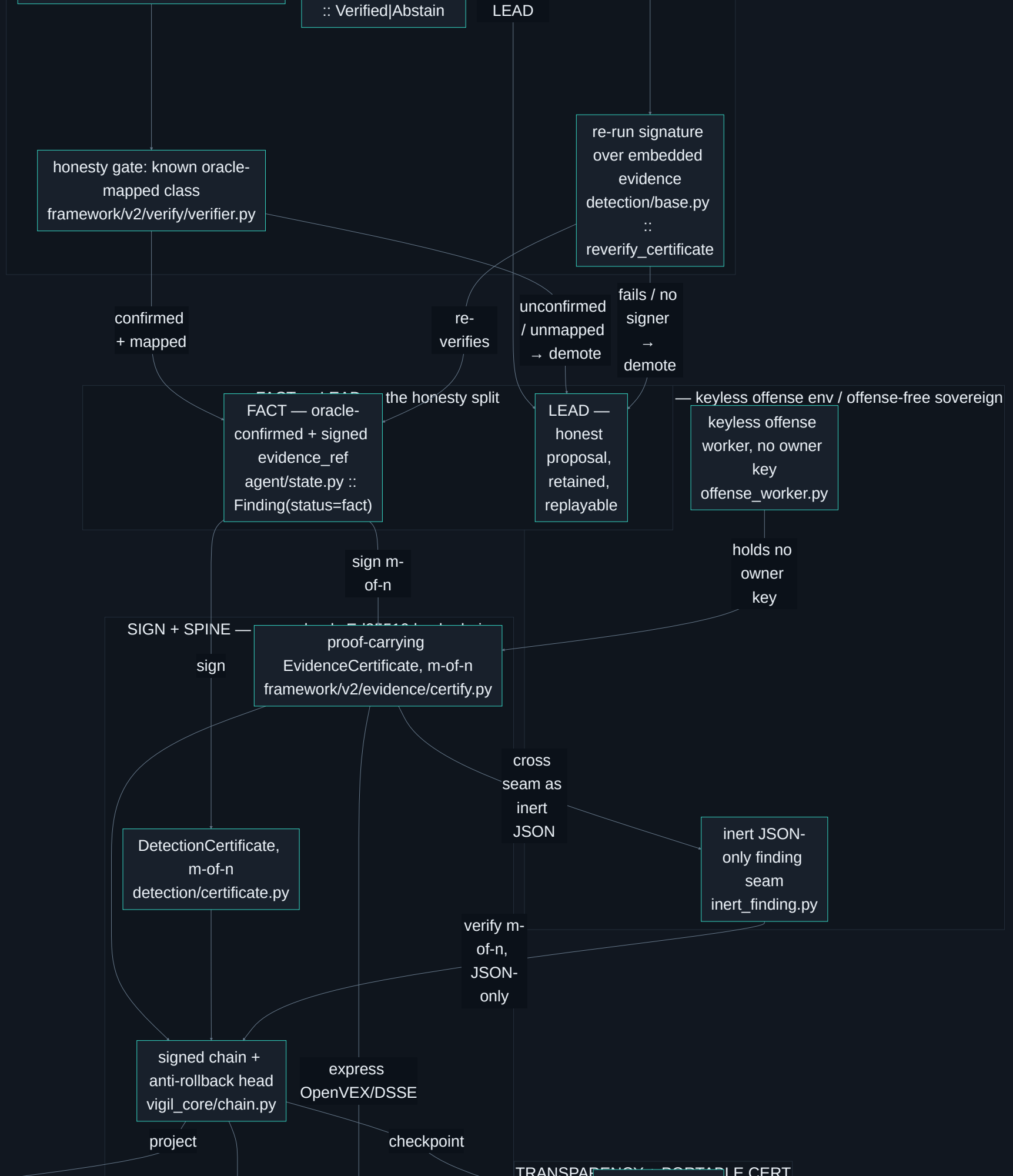
VIGIL L7 — The Veracity Firewall & Trust Boundaries: how a claim becomes a signed FACT or stays a LEAD

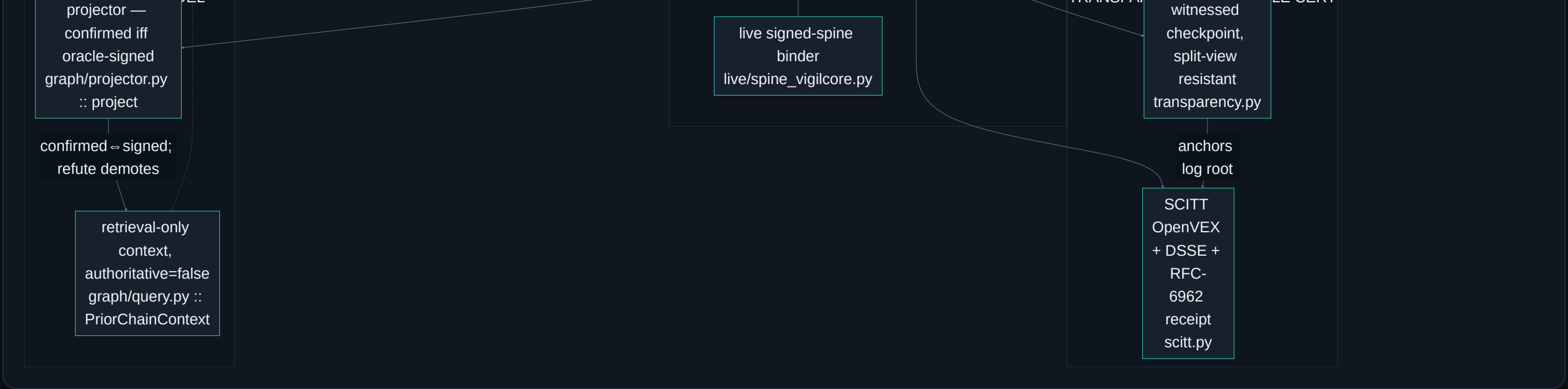
Every signal in VIGIL — an LLM's exploit claim, a live tool's raw output, an AI-Gauntlet result, a detection signature, a graph node — enters as a non-authoritative PROPOSAL. The veracity firewall RE-EXECUTES a deterministic oracle over the retained context (never trusts a string): CRUCIBLE's oracle re-fires, an I1 randomized per-run challenge makes replay/hallucination structurally impossible, gauntlet routes judge_llm results to LEAD before any oracle call, and detection re-runs its own signature over the certificate's embedded evidence. The firewall is DEMOTE-only — it can turn a claimed fact into a LEAD but never promote by assertion. A CONFIRMED signal additionally requires a known oracle-mapped class (honesty invariant), then is minted as an m-of-n signed proof-carrying certificate, appended to the append-only Ed25519 hash-chain SPINE, exposed via a split-view-resistant witnessed transparency checkpoint, and expressed as an offline-forever SCITT (OpenVEX/DSSE/RFC-6962) statement. The FACT-vs-LEAD split reappears everywhere: react.IntakeResult, gauntlet fact/lead, detection Grade.FACT/LEAD, and graph confirmed/lead nodes. Orthogonally, action authorization runs its own conjunctive gate (WARDEN tier $\wedge$ CRUCIBLE authority $\wedge$ m-of-n destruction) plus a loopback-pinned egress gate; nothing self-authorizes, and every gated action is preceded by a fail-closed usage-attestation ledger entry (WHO/WHEN/WHAT). The offense worker is keyless (the owner signing key never crosses into it) and confirmed findings cross to the sovereign side only as inert JSON verified by m-of-n — the two structural trust boundaries.



context
only, no
authority

propose





ReAct veracity interposition (intake_result / authorize_edge / parse_decision)

/home/kali/vigil/integration/vigil_integrat

The sovereign re-plumb of redamon's ReAct loop. parse_decision fail-closes a raw LLM response into a non-authoritative LLMDecision (a broken deploy never becomes a deploy; total parse failure pauses for a human). classify_edge/authorize_edge route every target-touching edge through the injected conjunctive gate; escalation/fireteam edges QUEUE for signed operator approval. intake_result turns every analysis.findings claim into a LEAD and, on an exploit_succeeded claim, fires the injected deterministic oracle over the retained raw output — only a signed evidence ref yields a FACT.

- FEATURES
- fail-closed decision downgrade
 - IntakeResult.facts vs IntakeResult.leads split

oracle_adapter — no-hallucinated-findings pipeline (confirm_and_certify / certify_to_scitt)

/home/kali/vigil/integration/vigil_integrat

Bridges a proposed finding (carrying its retained oracle_context) into CRUCIBLE's authority: (1) confirm_finding re-runs the pure deterministic oracle over the retained baseline/mutated/probe/timing/OOB context — returns None unless an oracle actually fired at high confidence; (2) honesty invariant — a signed FACT requires the confirmed bug_class to be a known oracle-mapped class, else it stays a labelled lead; (3) mints a proof-carrying EvidenceCertificate binding oracle_context_digest and signs it m-of-n. certify_to_scitt bridges only a FACT to an OpenVEX 'affected' SCITT statement (a lead is refused).

- FEATURES
- AdapterResult(status=fact|lead)
 - OracleKind .value stored (not repr) so verify_certificate accepts it

CRUCIBLE deterministic oracle (confirm_finding)

/home/kali/vigil/engine/crucible/framework/v2/verify/confirmation.py

The deterministic authority the adapter drives. Re-executes the pure oracle for the finding's class over the retained oracle_context and returns a confirmation (bug_class, confirmed_by oracle kind, confidence) only when an oracle genuinely fires at >= high confidence, else None.

- TOOLS
- framework/v2 oracle registry (BUG_CLASS_ORACLES)

Honesty gate — known-oracle-mapped bug class (is_known_bug_class / normalize_bug_class)

/home/kali/vigil/engine/crucible/framework/v2/verify/verifier.py

normalize_bug_class canonicalizes a claimed/confirmed class; is_known_bug_class returns true only for classes present in BUG_CLASS_ORACLES (a class VIGIL actually has a deterministic oracle for).

injected gate + injected OracleFn (testable without kernel/framework)

TOOLS

safety/llm_intake.parse_proposal

conjunctive gate (injected)

OracleFn (wraps oracle_adapter.confirm_and_certify)

IN VARIANT

The LLM only PROPOSES. An output_analysis assertion (incl. exploit_succeeded) is never a fact; with no oracle wired nothing can be a fact (fail-closed). A whitespace/garbage evidence ref mints no fact. This is the load-bearing anti-hallucination seam of the fusion.

fail-closed on empty signers

TOOLS

framework/v2/verify/confirmation.confirm_finding

framework/v2/verify/verifier.is_known_bug_class + normalize_bug_class

framework/v2/evidence/certify.build_certificate + sign_certificate

scitt.mint_finding_statement

IN VARIANT

There is no assertion-only path — an LLM's say-so never confirms. confirm_and_certify refuses to run with zero governance signers (won't label an unsigned cert a fact). A fire on an unmapped class is retained as a lead, never a signed fact. framework imports are LAZY so the module stays import-clean for the offense-free sovereign env.

IN VARIANT

Re-execution, not string trust — the confirmation is derived from the retained evidence reproducing the finding, never from the finding's own claim. It is the single point of truth-minting; the veracity firewall demotes when it does not fire.

IN VARIANT

A confirmed fire on an unmapped class must NOT become a signed fact — claiming otherwise would be the very hallucination the system exists to kill; such a fire is retained as a labelled lead.

Randomized-challenge oracle — I1 replay/hallucination impossibility (challenge_oracle)

/home/kali/vigil/integration/vigil_integration/challenge_oracle.py

Generalizes DARPA CGC proof-of-vulnerability to web/API classes. Each confirmation issues a FRESH unpredictable challenge (nonce-echo / canary-leak / oob-token / value-control); the finding is VERIFIED only if the LIVE target satisfies THAT specific per-run token. A verdict is kernel-minted (Verified|Abstain) and HMAC-bound to (kind, token) under a per-run oracle key so no LLM/critic can forge 'Verified'; verify_minted checks the MAC.

FEATURES

AI-Gauntlet oracle_kind routing (route_candidate / run_gauntlet)

/home/kali/vigil/integration/vigil_integration/gauntlet/sensor.py

Drives a red-team tool (garak/pyrit/giskard/promptfoo) via an injected subprocess boundary, parses candidates, and routes each by its TRUSTED-taxonomy oracle_kind: contains/classifier/regex → deterministic (the injected oracle re-executes over the retained output with a fresh per-run challenge token; only a non-empty signed evidence ref mints a FACT); judge_llm/unmapped → returns a LEAD BEFORE any oracle call.

FEATURES

Gauntlet adapter — untrusted report normalizer (parse_adapter_output)

/home/kali/vigil/integration/vigil_integration/gauntlet/adapters.py

Folds four red-team tools' native JSON reports into one normalized CandidateFinding schema (tolerant field aliases across garak/PyRIT/Giskard/promptfoo). evidence is the retained raw output the oracle re-executes over — transient, never persisted; safe_preview is the only sanctioned display path.

TOOLS

safety/llm_intake.extract_json

safety/prompt_safety.wrap_untrusted_inline

tools.redact_tool_args (one F3 secret

DetectionOracle base — FACT/LEAD grading + offline re-verification (detect / reverify_certificate)

/home/kali/vigil/integration/vigil_integration/detection/base.py

The deterministic heart of a Sentinel signature. evaluate fires (OracleHit) or stays silent over retained telemetry; detect mints a FACT only if the resolved grade is FACT AND a signer+verify key are wired AND the signed DetectionCertificate RE-VERIFIES (signature + evidence-digest match + a live RE-RUN of the named oracle over the certificate's own redacted evidence produces the same signature family). Anything softer degrades to a LEAD.

FEATURES

Grade.FACT/LEAD

BUG_CLASS_CHALLENGE map
(sqli/xss/ssti → nonce-echo, idor/lfi → canary-leak, ssrf/rce/xxe → oob-token, memory_safety → value-control)

constant-time value compare

MintedVerdict.mac

IN VARIANT

verify(new_challenge, response_for_old_challenge) is always ABSTAIN — replay/hallucination is impossible by construction, not merely improbable. The value is Verified|Abstain, never a bare boolean an agent could assert; a degenerate low-entropy token can never mint Verified. Pure stdlib, no framework import; token from secrets (injectable for tests).

deterministic per-run challenge_token from injected seed (no RNG/wallclock)

GauntletResult.fact_count / lead_count

type-level FACT construction via Finding model

TOOLS

gauntlet/owasp_map.map_category + DETERMINISTIC_KINDS

injected GauntletOracle (wraps I1 challenge + CRUCIBLE confirm/certify)

injected RunTool subprocess boundary

IN VARIANT

A judge_llm result can NEVER auto-promote — the promotion path is structurally unreachable for it, so no adversarial oracle, maxed ASR, or high severity band can launder an LLM-judge guess into a fact. ASR is a metric, never a promotion signal. bug_class/owasp_id come only from the trusted taxonomy row, never the attacker-influenced raw category. Total on malformed input.

vocabulary)

IN VARIANT

TOTAL on attacker-influenceable input — a malformed/adversarial/truncated report degrades to no candidates, never a raise. The raw category is used ONLY to look up a trusted taxonomy row and never becomes part of an emitted record. No red-team framework is imported (deps + egress isolated behind the subprocess boundary).

Detection.is_fact requires certificate present

windowed()/group_by() rate axis from records, not clock

TOOLS

detection/certificate build/sign/verify

detection/registry.resolve_oracle

detection/logs parsers

IN VARIANT

Fires on the TRUE signature only — the benign twin does NOT fire (a benign twin that fires is a BLOCK). Re-execution, not string trust: a recorded/forged certificate whose evidence doesn't reproduce the fire cannot re-verify. Deterministic/total (windows from records' own ts/seq, injected seq, no wallclock/RNG); offense-free (reads telemetry, performs NO egress). Every method degrades malformed input to no-signal.

DetectionCertificate — signed, offline-re-verifiable PCF (defensive mirror of the evidence cert)

/home/kali/vigil/integration/vigil_integration/detection/certificate.py

Every FACT-grade detection is minted as a DetectionCertificate: redacted telemetry evidence + the oracle that fired + signature family + verdict + Ed25519 signature over the vigil_core domain-tagged canonical payload (minus the signature). cert_id is the content-address used as Finding.evidence_ref.

TOOLS

vigil_core.evidence_signing_bytes / verify_one

Finding — the type-level FACT/LEAD invariant

/home/kali/vigil/integration/vigil_integration/agent/state.py

The shared finding record across react/gauntlet/detection. status is the veracity: LEAD = proposal; FACT = oracle-confirmed carrying a signed evidence_ref. record_lead strips evidence_ref; record_fact refuses an empty ref. A model_validator enforces at construction time that a FACT-status finding carries a non-whitespace evidence_ref.

FEATURES

_fact_needs_evidence model_validator

Graph projector — one-way spine → graph, confirmed iff oracle-signed (project)

/home/kali/vigil/integration/vigil_integration/graph/projector.py

The single most trust-laundering-prone fusion, enforced deterministically. The graph is written ONLY by project() over signed SpineRecords. A finding projects to a CONFIRMED node only if the record is a FACT carrying BOTH a non-empty signed evidence_ref AND a signature_ref (_is_confirmed); everything else — including a bare status='fact' with no evidence — is a LEAD. Bi-temporal: a refute sets

Graph query — inert, non-authoritative read-model (PriorChainContext)

/home/kali/vigil/integration/vigil_integration/graph/query.py

Cross-session retrieval: surfaces a prior engagement's confirmed findings and failure lessons to warm-start the next run. confirmed_findings is built only from GraphView.confirmed_findings (a lead physically cannot appear there); leads sit in a separate, explicitly-unproven field; only active (non-retired) nodes are returned.

tools/mcp_registry._redact_str

separate facts/leads stores on AgentState

invalid_from (never deletes). Recon bridges to a host are created only if the injected scope_gate admits every client-independent reading of the host.

FEATURES

ConfirmationStatus.CONFIRMED/LEAD

_is_oracle_grounded_refutation (demote-only-by-re-execution)

scope-gated bridges, fail-closed on unextractable host

total over a malformed record list

INVARIANT

The graph can never launder an unproven claim into a fact; veracity is re-derived from signed evidence, never trusted from a field's claim. A confirmed fact can be DEMOTED only by an oracle-grounded refutation (itself carrying signed evidence_ref+signature_ref) — a bare LLM 'looks like a false positive' can retire only a LEAD. Pure/deterministic ((seq,hash) order) so the same spine rebuilds a byte-identical view; the graph authorizes nothing.

INVARIANT

An analyst RE-RUNS it instead of trusting it — any tamper (evidence/verdict/oracle) breaks the signature. Secret-free: evidence lines scrubbed through the one F3 redactor before entering the cert, so no credential lands on the spine. Deterministic (injected seq, no wallclock/RNG).

INVARIANT

Only agent.react (the oracle-confirmation path) may set FACT. Finding.model_validate({status:fact, evidence_ref:"}) is refused — so a deserialized/replayed/checkpointed record can never construct an evidence-less fact even off the persistence path.

INVARIANT

No consumer may read AUTHORITY from the graph — every result is stamped authoritative=False / retrieval_only=True and the model is frozen so a consumer cannot flip those flags. Any action it suggests still clears WARDEN + the conjunctive gate + the egress gate; being in a prior chain authorizes nothing.

Signed spine chain — append-only Ed25519 hash-chain + anti-rollback head (vigil_core.chain)

/home/kali/vigil/packages/core/vigil_core/vigil_core/chain.py

Each ChainEntry hashes (seq, prev_hash, cert_digest); a SignedChainHead anchors the whole log with a monotonic last_seq and an ABSOLUTE entry_count (base_count of pruned Merkle-committed leaves + live window). verify_chain detects deleted/reordered/tampered entries and seq gaps; verify_head additionally checks the m-

Live signed-spine binder (spine_vigilcore)

/home/kali/vigil/integration/vigil_integration/live/spine_vigilcore.py

The live drop-in for the F2 checkpoint layer's injected trust seams — binds signer/verify/writer/reader to a real Ed25519 keypair and a real append-only hash-chained spine file (one canonical-JSON record per line). Rebuild re-runs the Finding validator + _facts_store_is_sound on every record; a torn tail is dropped and rebuild falls back to the last intact snapshot; a

Witnessed transparency log — split-view resistance (transparency)

/home/kali/vigil/integration/vigil_integration/transparency.py

Adds transparency-log guarantees over the signed spine head so a third party trusts the log without trusting its operator. A Witness countersigns a checkpoint only after verifying it consistently extends the last one it tracked (append-only: count and last_seq only grow, meta-chain links back). CheckpointEmitter maintains the prev_checkpoint_hash chain, is idempotent

SCITT — offline-forever finding certificate (OpenVEX + DSSE + RFC-6962)

/home/kali/vigil/integration/vigil_integration/scitt.py

Expresses a confirmed-finding certificate as an IETF SCITT-style DSSE signed statement carrying an OpenVEX statement (portable vulnerability+product+status vocabulary), plus a Receipt with a real RFC-6962 Merkle inclusion proof binding it to the statement log. verify_anchored_receipt ties the log root to a split-view-resistant I2

of-n head signature and rejects a rollback below the accepted high-water.

TOOLS

vigil_core.canonical.evidence_signing_bytes

vigil_core.crypto.sign / verify_threshold

INVARIANT

Tamper-evident + append-only; deterministic. Vendored verbatim from CRUCIBLE so the offense-minted certificate and the sovereign-appended record share byte-identical chain math. Signing the head is provisioning; the runtime verifies. Version-conditional head payload keeps a v1 head byte-identical so old clients/persisted heads verify unchanged.

tampered/forged/re-linked record is detected.

TOOLS

vigil_core.crypto

vigil_core.chain

agent/checkpoint

INVARIANT

Going live changes NOTHING about the sovereign contract — it does not make anything true or authorize anything; it is a tamper-evident persistence of state the oracle and gate already governed. An adversary who can write the spine file AND holds the signing key still cannot launder an evidence-less fact (the rebuilt Finding validator skips it under any status spelling). Deterministic: no wallclock/RNG/uuid on the write/decision path.

on an unchanged head, and gathers countersignatures atomically. is_split lets any client holding two checkpoints prove a fork after the fact.

FEATURES

domain-separated witness signature

consistent() pairwise append-only check

verify_witnessed / verify_split_view_resistant / verify_log / is_split

INVARIANT

An honest stateful witness never equivocates (never signs two forks). Split-view PREVENTION is a quorum-intersection property that holds ONLY for a strict-majority witness set (2*threshold > n over distinct decoded keys) — verify_split_view_resistant proves that; below it, only per-witness non-equivocation and DETECTION remain. Fail-closed on empty/duplicate/low-order witness keys. vigil_core-only (sovereign-safe).

witnessed checkpoint, so a client/regulator/court verifies a finding offline and forever — no VIGIL service, no network.

FEATURES

OpenVEX affected/under_investigation

m-of-n DSSE envelope

RFC-6962 _mth/_audit_path/verify_inclusion

proof-depth bound vs recursion DoS

INVARIANT

Honesty preserved in the portable form: a CONFIRMED finding is status 'affected'; an unconfirmed lead is 'under_investigation', NEVER asserted affected — and mint_finding_statement bridges only a FACT. expected_root is REQUIRED in verify_receipt (a receipt carries its own root; without pinning to an out-of-band trusted root an attacker self-manufactures one). DSSE PAE domain-separates a statement signature from a raw evidence/checkpoint signature. vigil_core + transparency only.

Conjunctive action gate — WARDEN $\wedge$ CRUCIBLE $\wedge$ m-of-n (conjunctive_gate)

/home/kali/vigil/integration/vigil_integration/conjunctive_gate.py

Every target-touching offense action passes BOTH the CRUCIBLE authority (killswitch $\rightarrow$ validity window $\rightarrow$ scope $\rightarrow$ destructive $\rightarrow$ budget) AND the WARDEN tool-tier gate, plus a THIRD m-of-n threshold-destruction conjunct for destructive/high-blast actions. First failure wins; ALLOW only if in-envelope AND WARDEN auto AND (if destructive) threshold-authorized; QUEUE if only WARDEN needs owner approval.

WARDEN raise-only tier floor (warden_gate)

/home/kali/vigil/integration/vigil_integration/warden_gate.py

Classifies an offense tool by NAME to a tier A0..A3 via an injectable classifier (default: the sigil-kernel binary, fail-closed to A3), then raises to a floor (default A2) with max(). AUTO only if tier<=A1 AND tier<=ceiling (default A1) — so every live-target offense tool QUEUES for owner approval; a staging/twin posture may lower the floor so recon auto-runs while destructive/exec stays A3 $\rightarrow$ queue.

CRUCIBLE authority — killswitch / scope / window / budget (authority.gate)

/home/kali/vigil/engine/crucible/framework/v2/authority/gate.py

authorize_action decides whether THIS target/action is inside the signed engagement envelope right now (killswitch has absolute precedence, then validity window, scope, destructive/live-destructive flags, budget). load_authority_for_gate loads the authority VERIFIED against the governance trust root.

TOOLS

framework/v2/authority/models.ActionRequest

Threshold-destruction gate — m-of-n owner-inclusive quorum (destruction_gate)

/home/kali/vigil/integration/vigil_integration/destruction_gate.py

For a destructive/high-blast action, authorize_destruction requires an owner-inclusive m-of-n quorum-signed authorization bound to exactly this action (target + engagement), single-use, with a dead-man's-switch — orthogonal to WARDEN's tier approval.

build_offense_gate cross-binds the quorum-authorized action to the SAME target+engagement the CRUCIBLE half scopes.

FEATURES

- CrucibleResult/DestructionOutcome duck-types (pure, injectable)
- target/engagement cross-bind on the destruction leg

TOOLS

- framework/v2/authority/gate.authorize_action + load_authority_for_gate
- warden_gate.decide_tool
- destruction_gate.authorize_destruction

INVARIANT

Fail-closed conjunction: ANY error in any half is a DENY (never caught-and-continued); only an explicit WARDEN 'auto' may ALLOW; a strict `is True` check on destruction.authorized bars a truthy-but-not-bool from opening the irreversible gate. A None trust_root is refused (would load the authority unsigned). framework import is lazy — never runs in the sovereign env.

FEATURES

- ToolDecision(auto|queue|deny)
- cached kernel_classifier subprocess

INVARIANT

The floor can only RAISE a tool's tier, never lower it. Empty/unknown name or garbage classifier output → DENY (A3). The SDK hook adapter is fail-safe: only an AUTO decision runs, anything else raises WardenDenied. Import-clean — imports neither SIGIL nor any offense engine.

governance TrustRoot

INVARIANT

Loaded verified/fail-closed — a tampered scope/window/destructive flag is rejected at load. Its killswitch step is the absolute stop and is checked first by the conjunctive gate.

INVARIANT

An autonomous worker must never perform an irreversible action without the owner-inclusive quorum, even if WARDEN would auto-allow the class. The mandatory owner is bound into an immutable DestructionAuthority (a free owner_key_id would let a worker self-authorize).

Governed live executor — the loopback egress gate (executor.execute)

/home/kali/vigil/integration/vigil_integration/live/executor.py

Replaces the 'run a tool' thunk with a real subprocess spawn of a Kali tool (nmap/nuclei/httpx/ffuf/sqlmap/hydra) under a strict deny-by-default pipeline: (1) resolve the target with getaddrinfo and REFUSE unless every resolved address is IPv4 loopback (127.0.0.0/8) — before authorization, before any spawn; (2)

Egress denylist — metadata/link-local/reserved conscience (gateway.denylist)

/home/kali/vigil/gateway/vigil_gateway/denylist.py

is_egress_denied classifies an IP against always-denied ranges (127/8 loopback, 169.254.0.0/16 link-local INCLUDING 169.254.169.254 cloud IMDS, private ranges, IPv6 fe80::/10, and IPv4-mapped/compat v6 forms unwrapped so

Keyless offense worker — the offense/sovereign trust boundary (offense_worker)

/home/kali/vigil/integration/vigil_integration/offense_worker.py

Runs an offense engagement in env-offense (where framework.*/strix.* are importable) as an engagement-scoped, ceiling-bounded process holding NO owner signing key. Construction refuses an owner_key outright. emit_finding_envelope serializes a CONFIRMED finding to the inert JSON

Inert finding seam — offense → sovereign as JSON-only signed data (inert_finding)

/home/kali/vigil/integration/vigil_integration/inert_finding.py

The sovereign side's receiver for a confirmed finding crossing the FATAL-2 seam. validate_inert_finding parses the inbound blob with json.loads ONLY (never pickle/eval/yaml), size-bounded and strictly shaped; verify_signature re-verifies the m-of-n CRUCIBLE-governance signature with

authorize via the same conjunctive gate; (3) build a host-pinned argv LIST (never a shell string) reconstructed from validated components only; (4) write a signed, REDACTED ExecRecord while returning RAW output for the oracle.

FEATURES

- _resolve_loopback IPv4-only pin (rejects ::1, metadata, private)
- per-tool argv builders refusing smuggled hosts/unsafe options
- ExecRecord stdout_sha256 commits to raw output

TOOLS

- gateway/denylist.is_egress_denied (independent second refusal)
- tools.authorize_tool_call
- tools/mcp_registry._redact_arg_list / _redact_str
- subprocess.run(shell=False)

INVARIANT

No byte of a subprocess spawns unless the target is loopback AND the gate allows — both fail-closed. A smuggled second host/URL (127.0.0.1@evil.com) resolves non-loopback and dies at the pin, and the argv pins the validated loopback host so it can never reach a non-loopback host (client-independent). No shell=True ever. Every execution is a signed redacted spine record — no credential leaks; the RAW streams go only to the oracle, never the spine. Refuses to run with no signer wired (an execution must be recordable).

::ffff:169.254.169.254 / ::169.254.169.254 can't evade the v6 check).

INVARIANT

Consulted by the executor as an INDEPENDENT second refusal for the precise deny reason on the non-loopback branch — never the allow decision (it would 'allow' a public IP; loopback is the only allowed set, enforced separately). Blocks SSRF/credential-theft pivots to cloud metadata.

envelope that crosses to the sovereign side (refuses to emit an unsigned finding).

FEATURES

- DEFAULT_CEILING=A2
- can_sign_governance() structurally False
- duck-types SignedEvidence

INVARIANT

The owner signing key NEVER crosses into the offense process — so a governance event (killswitch release, promotion, approval) the worker forges carries sig=None and never verifies (verified fail-closed by sigil.governor.authn.verify_signed). This structurally stops the powerful offense process from becoming a confused deputy that escalates its own authority. Imports neither framework nor strix.

vigil_core.verify_threshold over the exact evidence_signing_bytes CRUCIBLE signed.

FEATURES

- MAX_ENVELOPE_BYTES DoS bound
- RecursionError-guarded parse
- ValidatedFinding.to_spine_payload (pure JSON)

INVARIANT

Inertness is structural — no code path can execute whatever a compromised offense worker sends. Depends on vigil_core alone (never framework/strix), which is what lets the personal core stay offense-free-by-construction while still ingesting oracle-confirmed findings. Two-anchor trust: the m-of-n cert signature (anchor 1) + the owner-signed spine head (anchor 2, added on append). Fail-closed on anything off.

Usage-attestation ledger
— non-repudiation
WHO/WHEN/WHAT (attestation.ledger)

/home/kali/vigil/integration/vigil_in
tegration/attestation/ledger.py

The always-on ledger the engine calls
(require_attestation) BEFORE any
engagement/gated action: mints a signed
UsageAttestation FIRST binding WHO
(OperatorIdentity: OS login + git name/email
+ key fingerprint + hostname), WHEN
(injected wall 'at' + a monotonic
TPM/software anchor that never decreases),
WHAT (action/target/phase) — every free
string secret-redacted at mint — then
requires a durable writer. verify_ledger re-
checks contiguity, each hash, non-
decreasing monotonic anchor, operator
binding, and every signature;
expected_head/expected_count pins catch
a truncated tail / total wipe.
ledger_who/ledger_when replay it.

FEATURES append-only hash-chained JSONL

- monotonic back-dating rejection
- distinct-operator non-repudiation roster

- TOOLS
- vigil_core.crypto.verify_one
 - vigil_core.canonical.evidence_signing_bytes
 - attestation/anchor.read_monotonic_anchor
 - tools/mcp_registry._redact_str

INVARIANT

Fail-closed on durability: no attestation can
be minted (no signer / unbound operator /
malformed input / write failure) ⇒ the
engine gets a DENY and cannot proceed
— a gated action never runs on an
attestation that wasn't durably recorded.
The signer's key_id must equal the bound
operator fingerprint (identity ↔ signature

binding). Chain order is (seq, prev_hash) alone — 'at'/monotonic are signed DATA, never ordering keys (no wallclock/RNG touches the chain). Tamper-evident: reordered/interior-deleted/forged entries fail.

VIGIL L8 — Proof Lifecycle: FACT → remediation certificate → continuously re-proven attestation series

An oracle-confirmed FACT crosses the FATAL-2 seam (`inert_finding.py`) as inert signed DATA, then becomes a remediation claim strictly by **re-execution**. Deterministic, confirmed-only triage may spawn a gated codefix; `prove_driver.py` re-drives the original exploit against a live authorized target and lets the *original oracle* — never the orchestrator, which only sequences — mint exactly one of four signed states. REMEDIATED is reachable through a single narrow path that embeds a re-executable `RemediationCertificate` (negative proof) or, on the differential channel, its own re-refutable round evidence. `verify_prove_certificate` re-runs that math offline and is **demote-only**: a pre-fix false-REMEDIATED is demoted at re-execution (invariant 3), never promoted. Signed ticks chain into an anti-rollback attestation series, witnessed and optionally RFC 3161-anchored, and re-proven on a cadence by the reprove loop. The certs, firewall, and orchestration are BUILT and offline-verifiable; “continuously re-proven against a live target” is a CAPABILITY gated on a reachable chartered target — with silent-case freshness honestly F1, a local durable floor, and local-TSA/operator-independence residuals named.

error_signature re-drive
remediation/live_adapter.py

boolean matched-decoy + origin
re-drive
remediation/differential_adapter.py

ASan crash-confirm + patch-
synth (R3)
remediation_binary/asan_repair.py

Continuous re-proof
loop (A2)
remediation/reprove.py

injected
adapter

injected
channel
+ Host-
pinned
origin

fix-by-
silence +
structural
+ fuz;
each cycle re-
runs
prove_remediation

Inert FACT
seam (offense-
>sovereign)
inert_finding.py

prove_driver
— 4-state
orchestrator

inert
signed
FACT-
as-DATA
(m-of-n
verified)

Confirmed-only
triage
remediation/triage.py

confirmed
FACT
may

REMEDIED
(1 narrow
path)

RemediationCertificate v2
(negative proof)
remediation/remediation_cert.py

embedded

signs 4
states

one
witnessed
tick per
cycle



optional
sidecar
supersedes
median T

External
RFC3161
anchor (A1)
time_anchor.py

prove_driver — four-state orchestrator

integration/vigil_integration/remediation/prove_driver.py

Turns “we patched it” into a signed REMEDIATED / STILL_VULNERABLE / INCONCLUSIVE / REFUSED statement.

Gates

downgrade/auth/PoP/identity/budget/positive-control, re-fires the ORIGINAL oracle over fresh bytes, and reaches REMEDIATED by ONE narrow path;

verify_prove_certificate re-executes the whole judgement offline.

INVARIANT

The orchestrator only sequences — the oracle over fresh bytes decides. Re-execution is demote-only: a pre-fix false-REMEDIED is demoted, never promoted (invariant 3). Live re-drive is a CAPABILITY behind an injected LiveTargetAdapter.

RemediationCertificate v2 — negative proof

integration/vigil_integration/remediation/remediation_cert.py

Domain vigil-remediation-cert-v2: silent==fixed only with positive-control fires + patched-silent + liveness + bound digests + a whole-cert Ed25519 signature. Offline re-verifiable by re-execution; every control is fail-closed at mint AND re-checked at verify.

INVARIANT

A stripped control breaks authenticity — you cannot forge a “fixed” by dropping a check. Cross-bound to the outer prove-cert (finding / class / freshness nonce / evidence digest) so a foreign cert cannot be spliced in.

differential_adapter — matched-decoy + direct-to-origin (R1/R2)

integration/vigil_integration/remediation/differential_adapter.py

Drives baseline / true / false_a / false_b matched-decoy rounds (metacharacter-identical) judged by the existing boolean-inference (SPRT) + WAF-closure oracles, plus an R2 Host-pinned *direct-to-origin* re-drive that bypasses a sanitizing edge. Every send is a scope-gated fetch.

INVARIANT

A REMEDIATED refute must be genuine CLOSURE (zero-tolerance attribution: no deterministic true≠false_a diff on any judged round), enforced at BOTH mint and re-execution. Origin re-drive can only DEMOTE an edge-only fix; a truncated observation gates to INCONCLUSIVE. Freshness is honestly F1.

live_adapter — error_signature re-drive

integration/vigil_integration/remediation/live_adapter.py

Re-drives the original exploit through CRUCIBLE's gated HttpExecutor; a FIRING trial earns genuine F2 (the fresh challenge nonce lands in the matched error line), a silent trial is F1. Single response-side channel.

INVARIANT

“Against the target now” is a CAPABILITY gated on a chartered, reachable target + the gated executor; other channels are deferred, not silently assumed.

fix_oracle — oracle-backed fix verification

integration/vigil_integration/remediation/fix_oracle.py

attestation_log — anti-rollback re-proof series

integration/vigil_integration/remediation/attestation_log.py

reprove — continuous re-proof service (A2)

integration/vigil_integration/remediation/reprove.py

attestation_witness — timed witnessed quorum

integration/vigil_integration/remediation/attestation_witness.py

`build_fix_oracle /`
`build_fix_signer` earn “remediated” ONLY by re-firing the SAME oracle over the patched build's re-captured bytes and signing on genuine silence; every unsound path (no re-drive, request-side channel, unbuildable context, unsigned) RAISES to unverified.

INVARIANT

No re-execution, no remediation. The re-drive capability is injected by the live pipeline; the pure verifier never fabricates a silence.

A hash-chained, monotonic series of re-proof ticks. `append_tick` admits via `verify_prove_certificate` → `build_chain` → governance m-of-n `sign_head` → `verify_head` + `check_highwater` → advance the floor upward-only.

INVARIANT

A rolled-back / truncated series is refused (`entry_count` primary, `last_seq` secondary). Honest limit: the durable high-water floor is a LOCAL, unsigned 0600 file — off-box anti-rollback is a deploy residual.

`run_reprove` loops a cadence over the retained corpus; each cycle re-runs a REAL `prove_remediation` over live bytes (not a hollow tick) and appends a witnessed tick. Cadence / clock / nonce are injectable; nothing wallclock/RNG enters the signed math. Ships as `vigil_reprove` + a `systemd` timer.

INVARIANT

A running cadence is an operating property only once an operator ENABLES the timer on a host — BUILT + shippable, not DEPLOYED. Freshness is only as current as the last fire, bounded to the retained corpus.

A strict-majority (split-view-resistant) quorum time-co-signs the attestation head, yielding a median no-later-than T; a present RFC 3161 anchor supersedes it. Domain-separated signatures, non-equivocation enforced.

INVARIANT

CAPABILITY for independence: distinct keys ≠ distinct operators; the median is sound only if the presented quorum is a strict majority of honest, independent parties — a deployment/social fact, marked.

inert_finding — the FATAL-2 two-env seam

`integration/vigil_integration/inert_finding.py`

The sovereign-side receiver: a confirmed FACT crosses as INERT signed DATA — `json.loads-only` (never `pickle/eval`), 256 KiB-bounded, strictly shaped, m-of-n governance signature re-verified with `vigil_core.verify_threshold` over the exact signed bytes.

INVARIANT

Inertness is structural — no code path executes what a compromised offense worker sends. Depends on `vigil_core` alone (never `framework/strix`), which is

triage — deterministic confirmed-only

`integration/vigil_integration/remediation/triage.py`

Nine fixed queries over the signed-spine read-model at zero LLM cost; a `TriageFinding` may spawn a remediation ONLY as an oracle-confirmed FACT with a signed `evidence_ref` — a LEAD is refused at the type level AND at the spawn boundary.

INVARIANT

Deterministic and total on malformed input. The live Neo4j read backend is the

codefix — gated destructive pipeline

`integration/vigil_integration/remediation/codefix.py`

Stages map to WARDEN tiers (`clone A1 / edit A2 / build A3 / open-PR A3 + m-of-n`); the per-block approval TIMEOUT auto-REJECTS (inverts `redamon`'s auto-accept), never `git add -A`, and “remediated” is signed ONLY after the original oracle goes silent.

INVARIANT

CAPABILITY: the pure injected pipeline is BUILT and gated; the live `git / sandbox /`

asan_repair — ASan binary remediation (R3)

`engine/crucible/framework/v2/remediation_binary/asan_repair.py`

Compiles C under `gcc -fsanitize=address`, crash-confirms via the `sanitizer_signal_oracle`, rewrites an unbounded `strcpy`→`strncpy`, and accepts ONLY on oracle silence + an out-of-band SIGABRT-absence + a length-sweep fuzz + a STRUCTURAL no-unbounded-copy enumeration + preserved benign functionality.

INVARIANT

The oracle stays the sole crash authority; the child cannot forge silence (out-of-band SIGABRT + tamper denylist). Residual: pattern-only synthesis; the structural check ENUMERATES `stdlib` calls (`strcpy/strcat/gets/sprintf/scanf %s/...`), it is not an exhaustive semantic proof; symbolic synthesis (`angr/z3`) is research-gated →

what keeps the sovereign core offense-free by construction.

deferred slice (H2); the embedded read-model is the default.

Claude executors are a later, off-by-default slice (see L11).

SYNTHESIS_UNAVAILABLE, never a fabricated patch.

VIGIL L9 — Assurance & Zero-Trust: external time anchor

· witness quorum · channel-binding · standalone verifiers

This layer turns a signed bundle's soundness from a CAPABILITY into an *externally re-checkable* property: an RFC 3161 external time bound (A1), a deployable witness quorum (A3), and a TLS channel-binding / notary producer-binding (Z1), attached alongside the transparency checkpoint and re-verified by two verifiers that import **zero VIGIL code** (stdlib + one Ed25519 library). Every module here is FATAL-2-clean, so it runs in the sovereign env. The load-bearing honesty: all three mechanisms are BUILT + tested but default to same-host / software authorities, so each is a CAPABILITY with a named independence residual — and the one thing NEITHER verifier does is re-fire the oracle. The oracle over real target bytes stays the sole minter; a standalone PASS is authenticity + binding + structure + chain + quorum + time, not a re-minted fact.





A1 — RFC 3161 external time anchor

`integration/vigil_integration/time_anchor.py`

Mints a real RFC 3161 token over `checkpoint_hash` via the system `openssl ts`; `genTime` is read ONLY from the signature-covered token (`ts - token_out` defeats the unsigned-status backdate); self-verifies on mint; the token is a sidecar that keeps the chain byte-identical.

INVARIANT

A present-but-unverifiable anchor FAILS CLOSED (never a silent downgrade to the median). Residual: the default `LocalTSA` is self-signed on the same host — proves the MECHANISM only; real no-later-than-T independence needs a third-party `RemoteTSA` / `OpenTimestamps`, unexercisable offline.

A3 — deployable witness co-signer

`integration/vigil_integration/witness_service.py`

A loopback/tunnel-bound co-sign transport (`bind_ok` refuses public/unspecified binds); N independently-keyed processes co-sign; anti-equivocation is delegated to `Witness.cosign` (append-only or 409). Fail-closed body-cap + anti-CSRF/rebind guard + a producer-pin so a bogus first checkpoint cannot poison the tip. Ships as `vigil witness serve|submit`.

INVARIANT

CAPABILITY: N-witness LOCAL deploy is proven and a third party CAN run one — but distinct keys ≠ distinct operators; genuine independence needs THIRD-PARTY operators, and at `threshold==1` equivocation is DETECTABLE, not prevented.

Z1 — channel-binding (zkTLS) evidence

`integration/vigil_integration/channel_binding.py`

An RFC 5705 exporter binding ties response bytes to ONE TLS session; a notary co-signs the domain-separated (session, response-hash) tuple; a producer-free offline verifier rejects a swapped body / wrong session / unpinned notary. Live capture via `openssl s_client` is network-gated (the parser is unit-tested).

INVARIANT

The pinned notary key is the sole trust anchor. Residual: the default SOFTWARE `LocalNotary` that VIGIL runs can be handed a fabricated (session, bytes) tuple — this proves the verifier SHAPE + mechanism, NOT producer-unforgeability, which needs MPC-TLS/TLSNotary (toolchain absent) + a third-party notary (`RemoteNotary` seam).

Standalone verifier — verify_pcf.py

`docs/proof-carrying-finding/verify_pcf.py`

Imports NO VIGIL code (`stdlib` + `cryptography`). Checks the out-of-band fingerprint pin, m-of-n Ed25519 authenticity, `oracle_context` binding, path-confined artifact re-hash, and the hash-chain + signed head + anti-rollback high-water; - - `prove-standalone` asserts no VIGIL module is even importable.

INVARIANT

Residual: it does NOT re-fire the oracle — reproduction (silence/fire) needs `python -m framework.v2 evidence verify`. A standalone PASS is authenticity + integrity, not a re-minted fact.

Standalone verifier — verify_vf.py

docs/proof-carrying-
finding/verify_vf.py

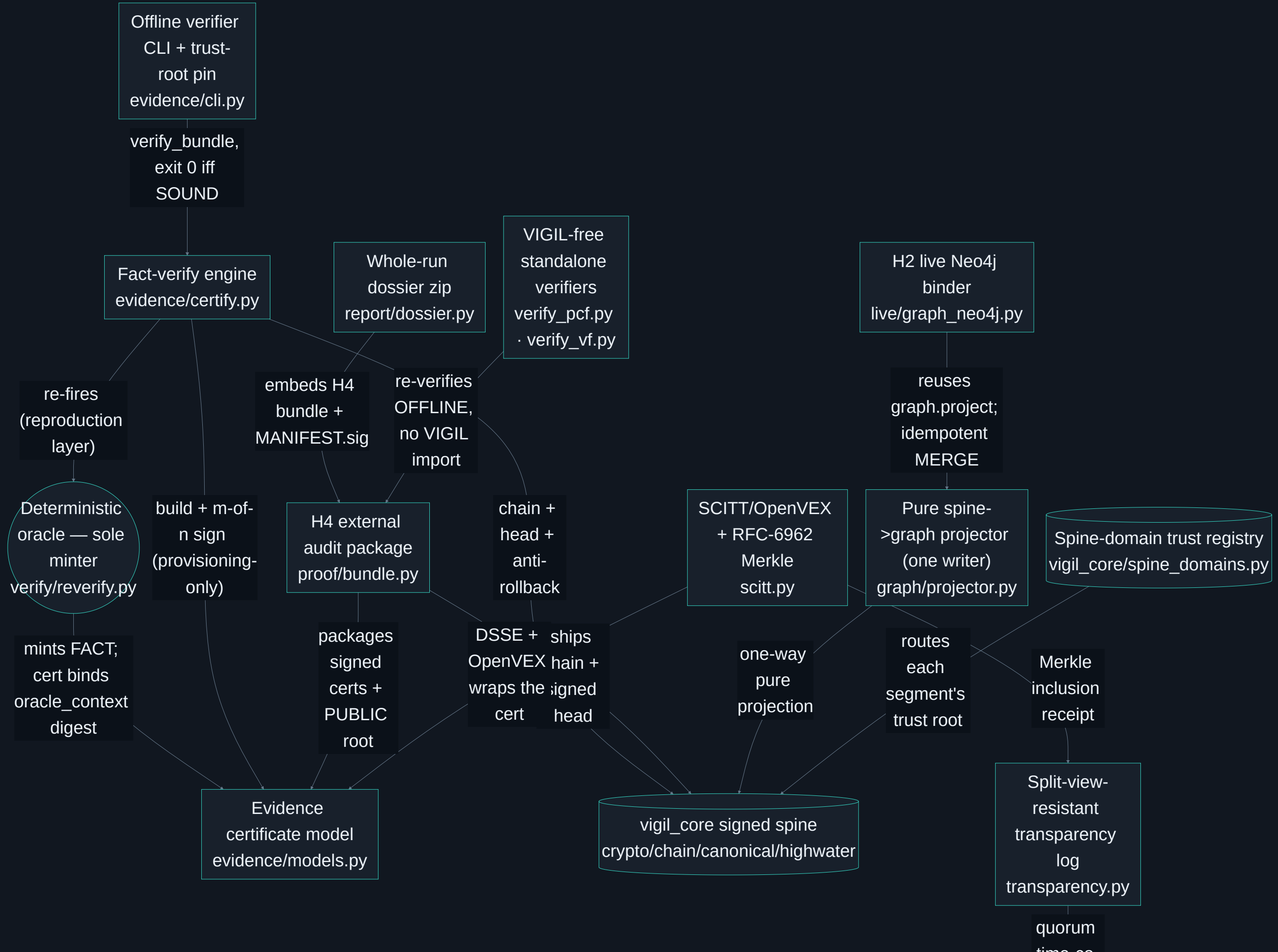
A VIGIL-free re-derivation of the whole remediation lifecycle — prove-cert authenticity, REMEDIATED cross-binding, the attestation chain/head/anti-rollback floor, the strict-majority witness-quorum median T, the A1 anchor genTime SUPERSEDING the median (via openssl ts, still VIGIL-free), and the Z1 channel-binding. Byte-parity with the in-tree producers is proven by a differential test.

INVARIANT

A single flipped byte anywhere flips the verdict to NOT SOUND. The one residual it shares with verify_pcf: it NEVER re-fires the oracle — that is the sole fact-minting authority, and the firewall stays demote-only.

VIGIL L10 — Evidence & Audit: the substrate that authenticates and transports facts (it never mints them)

The deterministic oracle over real target bytes (re-executed by `verify/reverify.py`) is the sole fact-minter; every layer here binds, signs, chains, projects, or re-verifies what the oracle already proved, and the veracity firewall only ever demotes. The flagship is the H4 external audit package (`proof/bundle.py`): a self-contained directory a third party re-verifies OFFLINE with the VIGIL-free `verify_pcf.py` + `verify_vf.py`. Honest limits hold throughout — the trust-root-fingerprint pin FAILS CLOSED on a mismatch (an absent pin is reported UNPINNED / not-anchored, the other layers still run); the offline verifier never re-fires the oracle; the live Neo4j binder needs a provisioned service; and hardware attestation is a stub on commodity silicon.



time-co-
signs
head ->
T

Attestation
quotes +
witness time-
bound
attest/provider.py

Evidence certificate model + fact-verify engine

engine/crucible/framework/v2/evidence
/models.py · certify.py

The deterministic (no-wallclock, seq-ordered) typed cert binding a finding's oracle_context digest + a per-file sha256 artifact manifest. verify_bundle runs five fail-closed layers — m-of-n authenticity · oracle_context binding · artifact re-hash · oracle REPRODUCTION · claim re-grounding — plus cert-set ↔ chain binding and the anti-rollback high-water.

INVARIANT

Signing is provisioning-only; the runtime only verifies (demote-only). Additive fields drop from the canonical bytes when empty, so every prior signature stays byte-identical.

Offline verifier CLI + trust-root pin

engine/crucible/framework/v2/evidence
/cli.py

python3 -m framework.v2 evidence
verify/certify/keygen/pcf-*. --
trust-root-fingerprint REFUSES on
a MISMATCH (exit 2); an ABSENT pin prints
[WARN] UNPINNED and still exercises the
other layers.

INVARIANT

An honest correction to the H4 spec framing: the pin is authenticity-anchoring — without it authenticity is NOT anchored, but the structural/binding/chain layers still run. (The *external-audit-package* verifier, by contrast, fails closed with no pin — see the H4 card.)

H4 external audit package + standalone verifiers

integration/vigil_integration/proof/b
undle.py · docs/proof-carrying-
finding/verify_pcf.py · verify_vf.py

export_bundle assembles a self-
contained directory (evidence-bundle · trust-
root · reverifiable · evidence/ tree ·
README · HOW-TO-VERIFY · TRUST-
ROOT-FINGERPRINT.txt) that a third party
re-verifies with stdlib + one Ed25519 library;
--prove-standalone asserts
framework/vigil_core/strix are unimportable,
and a differential test proves byte-parity with
the in-tree verifiers.

INVARIANT

FATAL-2: every framework import is
function-local; symlinks pruned; action_id
path-confined; only PUBLIC keys written.
Residual: the standalone verifier NEVER
re-fires the oracle — reproduction needs
the open-source VIGIL verifier as a
separate step.

Whole-run dossier package

engine/crucible/framework/v2/report/d
ossier.py

build_dossier emits one deterministic
zip: reports + the H4 proof bundle + secret-
scrubbed logs + the spine chain + drift +
MANIFEST.json + an m-of-n
MANIFEST.sig.json + the trust-root
fingerprint. Sorted entries, fixed epoch, path-
confined, symlinks never followed.

INVARIANT

Deterministic and reproducible: the same
run produces a byte-identical dossier; the
manifest signature makes any post-hoc
edit detectable.

vigil_core signed spine + domain registry

H2 live Neo4j binder + pure projector

SCITT / OpenVEX / Merkle + transparency + attestation

packages/core/vigil_core/vigil_core/
(crypto · chain · canonical · models
· highwater · spine_domains)

The one signed-spine substrate: Ed25519 +
m-of-n threshold, canonical JSON,
ChainEntry/SignedChainHead with a
monotonic anti-rollback high-water,
TrustRoot; spine_domains.py maps
each signed segment (sovereign
head/floor/witness, offense finding-anchor,
engagement spine, usage ledger,
blackboard) to who signs it, its owner-rooted
flag, and its domain tag.

INVARIANT

Namespace-pure — imports NO
framework/strix/sigil, so the FATAL-2
boundary is structural and v1 heads verify
byte-identically. Honest: only the offense-
finding-anchor is owner_rooted today;
the spine/ledger owner-tie is a later slice
(S6/S7).

integration/vigil_integration/live/gr
aph_neo4j.py · graph/projector.py

The projector is the ONE deterministic,
projection-only writer (confirmed/lead split,
spine-hash provenance, bi-temporal) — a
projection authorizes nothing.
Neo4jGraphWriter.rebuild_from_spine
reuses it: a scoped DETACH-DELETE clear
+ an idempotent engagement-scoped
MERGE, parameterized Cypher, a
Confirmed label only from oracle-grounded
is_confirmed, F3-redacted and fail-
closed.

INVARIANT

BUILT + residual: the neo4j driver is
INJECTED, not imported — it needs a
provisioned Neo4j + a wired session
factory or a rebuild is a no-write (tested
against a faithful fake session). The legacy
graph/store.py Neo4j store remains a
gated stub; the embedded store is the
default.

integration/vigil_integration/scitt.p
y · transparency.py ·
engine/crucible/framework/v2/attest/p
rovider.py

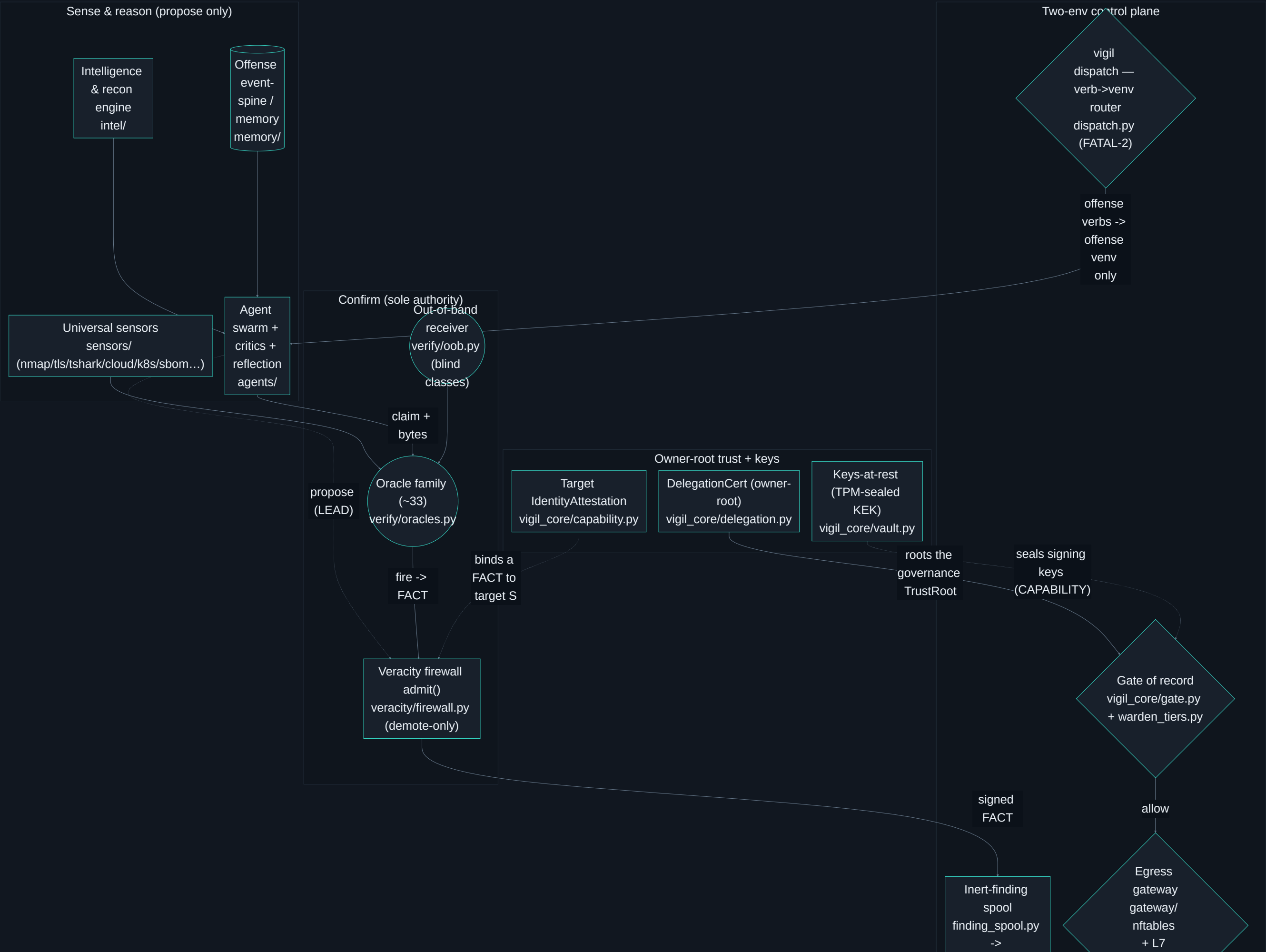
scitt emits a DSSE signed statement +
OpenVEX (confirmed=affected,
lead=under_investigation) + a real RFC-
6962 Merkle inclusion receipt;
transparency is a witnessed append-only
log; the
SoftwareAttestationProvider signs
integrity+origin quotes and a strict-majority
quorum time-co-signs the head.

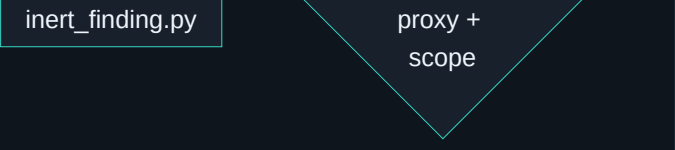
INVARIANT

Residual: split-view PREVENTION is
CONDITIONAL on a strict majority (2t>n)
— at threshold=1 only per-witness non-
equivocation + after-the-fact detection hold;
and a software quote is integrity+origin,
NOT hardware confidentiality (SEV-
SNP/TDX are stubs, H1).

VIGIL L11 — The Offense Plane & Control: the oracle family, the intel/sensor inflow, the egress gateway, and the boundary that keeps offense and sovereign apart

Everything above rests on an offense engine that only ever PROPOSES and a control plane that keeps it structurally separated from the owner's authority. The Master map collapses the whole oracle family into one node; here it expands: ~33 deterministic sole-minters (`verify/oracles.py`) plus the out-of-band receiver (`verify/oob.py`) for blind classes, fed by universal sensors and the intelligence engine, filtered by the general veracity firewall, and driven by a blackboard-coordinated agent swarm with critics and reflection. Underneath sits the two-env control plane: the `dispatch` super-CLI that routes offense verbs into the offense venv and never the sovereign one, the filesystem-only inert-finding spool that is the only thing crossing the boundary, the `nftables` + `filtering-proxy` egress gateway, and the owner-root trust chain (`delegation`, `capability`, `vault`, `warden` tiers). This is the substrate the seven upper layers assume.





The oracle family — ~33 sole-minters

engine/crucible/framework/v2/verify/oracles.py · verify/oob.py

The full set the Master map collapses into one node: differential / timing / boolean / predicate / achieved_state / side_effect / reflection / system_prompt / prompt_injection / honeypot / cred-stuffing / sanitizer / DOM / error_signature / oob_callback / service_reachability / tls_weakness / weak_crypto / version_range / policy_path / cloud / mesh / cicd / mobile / email / identity-posture / SQL-cmd-NoSQL-breakout / JWT + SAML forgery. oob.py is the 127.0.0.1 ephemeral receiver — the only sound minter for blind classes (SSRF, OOB-SQLi, blind XXE, deserialization).

INVARIANT

A claim is a FACT only when one of these fires over real target bytes; nothing else promotes. Each is a pure, deterministic, re-executable program with a mandatory passing negative control.

Veracity firewall — the general anti-hallucination choke

engine/crucible/framework/v2/veracity/firewall.py

admit() passes every Claim through one choke that RE-EXECUTES each cited ground and labels it CONTRADICTED / UNSUPPORTED / etc. (with claims.py / consistency.py / tokens.py). Distinct from the remediation-side prove_driver firewall of L8 — this is the general one backing the L7 discussion.

INVARIANT

Demote-only: it can strip a claim of grounding but never promote one. A non-re-firing finding grants zero grounded fact / node / edge / path.

Intelligence & reconnaissance engine

engine/crucible/framework/v2/intel/

The reason-over-intel layer: a one-substrate Observation model, entity resolution, gated live collectors (crt.sh / DoH / RDAP), a VOI ReconPlanner, and a world-model Beta-belief projection where promote.py mints url-bearing endpoints for discovery.

INVARIANT

Collectors are egress-gated; intel produces LEADS, never facts — discovery stays inert until an oracle confirms.

Universal sensors — the sensing inflow

engine/crucible/framework/v2/sensors/base.py

The producers that FEED the oracles: nmap, tls_cert, tshark, web_scanner, cloud_live / gcp / azure, k8s_live / runtime, sbom, android_manifest, mesh, email_auth, fuzz. Each normalizes into the one Observation model as a provenance-tagged lead.

INVARIANT

A sensor emits leads, never facts; live cloud/k8s sensors are gated. The oracle over the sensor's captured bytes is what confirms.

Agent swarm + critics + reflection

engine/crucible/framework/v2/agents/(critics.py · reflection.py · cognitive_refusal.py · scope_gate.py · egress_guard.py)

The blackboard-coordinated bodies behind the OODA loop — recon / hypothesis / exploit / reporter agents — with multi-critic scoring, reflection, cognitive refusal, and

Offense event-spine / memory

engine/crucible/framework/v2/memory/(store · recall · recorder · priors · postmortem · fleet)

The append-only offense-side substrate the telemetry projection and the graph projector read from — recall, priors, postmortems, fleet coordination. Distinct from the vigil_core sovereign signed spine.

Proof Studio — reproduce-from-raw PCF minting

integration/vigil_integration/proof/engine.py · content_gate.py

The FRONT of the proof lifecycle: a Strix PoC plus executor-captured NON-LLM bytes are re-driven so the oracle confirms them into a signed, replayable proof-carrying finding (else an honest LEAD). A

VIGIL-LIVE unified OODA engine

integration/vigil_integration/live/engine.py

The WS-2 attestation-first loop spine: attest → think (F2) → conjunctive gate → governed execute → oracle re-fire → FACT/LEAD → graph → cognition → observability → spine checkpoint. Loopback-proven; framework-free itself

per-agent scope/egress guards, all writing to one append-only, provenance-linked event spine.

INVARIANT

Critics / reflection may only advise, re-rank, defer, or abstain — never promote a finding or override the oracle. Coverage is never silently gated out.

INVARIANT

Append-only and deterministic in the learning/reward math (no wallclock/global-RNG); learning re-ranks and defers, it never mints or auto-labels a silent oracle a false positive.

fail-closed content gate refuses to store/replay/mint evasion / persistence / wipe / worm / cred-exfil PoC classes before minting.

INVARIANT

The oracle judges the bytes; the model only points. Provenance is reproduced (executor bytes), never the LLM's word.

(framework imports are function-local in the wiring).

INVARIANT

Every target-touching edge crosses the gate before execute; every run that enters the loop persists an owner-rooted, offline-verifiable spine segment.

R4 live external-tool runner

integration/vigil_integration/live/external_tool.py

Runs a REAL external tool through a ScopeGate (gateway denylist ^ charter scope) and the docker + nftables egress topology, then lets the oracle adjudicate the tool's output into a FACT via an independent gated handshake. A present tool (nmap) proves the path against a loopback listener.

INVARIANT

The allow-set is ONLY the charter-authorized IPs — the runner does not self-authorize the target's own resolved IP (no wildcard → private bypass); the IMDS/link-local floor is unliftable. CAPABILITY: garak/PyRIT live-fire is tooling-gated, no such FACT minted.

Authorization gates — approval broker + nonce ledger

integration/vigil_integration/live/approval_broker.py · approval_token.py · nonce_ledger.py

A file-backed pending-approval queue plus an owner-signed single-use token inbox satisfies the WARDEN human leg (queue → allow for exactly one action); a durable ATOMIC (O_CREAT|O_EXCL) nonce ledger is the serialization point for destruction authorizations — exactly one winner under concurrency.

INVARIANT

A non-AUTO action queues rather than proceeds; a single-use token / nonce cannot be double-spent even under a concurrent race.

Destructive-leg quorum + live codefix executors

integration/vigil_integration/live/destruction_provision.py · codefix_runner.py

destruction_provision generates + signs the m-of-n DestructionAuthority quorum for vigil patch --open-pr; codefix_runner is the first live caller of the pure autopatch library — real Claude diff, git clone, git apply on the clone.

INVARIANT

CAPABILITY: real separation-of-duties needs co-signer keys off-box (solo default = a single owner key, still single-use / window / action-bound). The live PROPOSE/CLONE/APPLY legs are BUILT non-destructive; the OPEN-PR leg is gated OFF by default.

The two-env control plane — dispatch · proxy · verify

integration/vigil_integration/dispatch.py · uiproxy.py · live/spine_verify.py · governance_identity.py

dispatch is the pure-stdlib, exec-only static verb → venv router — the FATAL-2 boundary rendered in code (offense verbs never resolve into the sovereign venv). uiproxy (vigil up) federates the sovereign cockpit + offense api + console at one origin behind a never-public bind. vigil verify is a public-key-only, inert-bytes-only offense-spine segment verifier over an owner-signed delegation.

INVARIANT

Offense and sovereign code never share a process; the proxy reaches no target; vigil verify reports owner_rooted honestly and cannot verify the CRUCIBLE DB-projection chain.

Egress gateway — the physical two-env boundary

gateway/vigil_gateway/ (nftables.py · proxy.py · docker.py · scope_source.py · denylist.py)

Owner-root trust chain — delegation · capability · vault · WARDEN tiers · gate-of-record

packages/core/vigil_core/vigil_core/ (delegation.py · capability.py · vault.py · warden_tiers.py · gate.py)

Read planes & one-way projections — console · telemetry · learn-drain · knowledge-sync

engine/crucible/framework/v2/console/ · integration/vigil_integration/

AEGIS runtime-defense engine (the mirror's engine)

engine/crucible/framework/v2/aegis/ (gateway · guard · middleware · boundary · oob_correlator · rampart · response_policy · actor_graph)

Two layers: an nftables L3/L4 deny-default ruleset + a host-side L7 filtering forward proxy + docker sandbox network pinning + charter scope resolution. The denylist is the single source of truth for always-denied ranges (metadata/link-local/loopback/private).

INVARIANT

CAPABILITY: enforcement is real only when the docker + nftables topology is provisioned; the metadata/link-local floor is never liftable by scope. It is the physical layer beneath the WARDEN/authority logical gates.

The owner-root DelegationCert ties the free-floating CRUCIBLE governance TrustRoot to the OWNER (role/scope/expiry-bounded); capability is an owner-signed target IdentityAttestation (SPKI/acceptable-identity pin) that closes proof transplant/target-swap; vault seals keys at rest under a TPM-sealed KEK; warden_tiers is the one A0–A3 classifier-of-record (a byte-faithful port of the Rust kernel, pinned to golden vectors); and gate.py is the one conjunctive gate-of-record shared byte-identically by both processes.

INVARIANT

Since S6 (2026-07-25) conjunctive_decide lives in vigil_core/gate.py as the sole gate-of-record; the integration-side conjunctive_gate.py now only re-exports it. Vault is a CAPABILITY — DISABLED until Vault.provision seals a KEK to this machine's TPM (plaintext-0600 until then).

(telemetry.py · learn_drain.py · knowledge_sync.py)

The loopback-only read+SSE Ops Console the proxy federates at /offense/*; a one-way metrics projection over the signed spine (FACTs vs LEADs, refusals, by-kind); the offense-side consumer of the owner-signed learn-grant seam (re-derives a lead from the offense's OWN intel, mints no fact, bumps no priors); and the operator-gated knowledge → GitHub sync.

INVARIANT

All read-only / widen-nothing: the console mints nothing, telemetry projects only, learn-drain is non-circular (fail-closed Ed25519 verify), and committing knowledge makes nothing a FACT.

The offense-side embeddable provable-firewall itself — distinct from the L5 detection-mirror adapters. It re-uses the same oracle-then-sign discipline to turn runtime telemetry into signed detection facts on their own chain.

INVARIANT

Defensive-only by construction (adds no offense capability); a detection is a FACT only when its defensive oracle fires over data a real system produced.